

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP

Phân tích mối liên hệ về tình trạng giao
thông tại các nút giao thông quan trọng
ở TP.HCM

Ngành: Khoa học máy tính

Hội đồng: 11L Đồ án tốt nghiệp

GVHD1: PGS.TS. Trần Minh Quang

GVHD2: ThS. Đỗ Thanh Thái

—o0o—

SVTH 1: Nguyễn Lê Quốc Khánh (2211522)

SVTH 2: Nguyễn Minh Khang (2211452)

SVTH 3: Nguyễn Hữu Khánh (2211521)

TP. HỒ CHÍ MINH, THÁNG 4/2026

Lời cam đoan

Chúng tôi xin cam đoan rằng toàn bộ nội dung trong báo cáo này là kết quả của quá trình nghiên cứu, tìm hiểu và làm việc nghiêm túc của nhóm. Các nội dung trích dẫn từ tài liệu, công trình nghiên cứu, nguồn dữ liệu và các hình ảnh minh họa đều được ghi rõ nguồn hoặc đã được xử lý phù hợp với mục đích học thuật.

Báo cáo không sao chép hoặc sử dụng bất kỳ nội dung nào từ các công trình khác mà không trích dẫn theo đúng quy định. Các phân tích, đánh giá, biểu đồ và kết luận được trình bày trong báo cáo được xây dựng dựa trên những dữ liệu mà nhóm đã thu thập, xử lý và phân tích trong quá trình thực hiện đề tài.

Chúng tôi hoàn toàn chịu trách nhiệm trước nhà trường và giảng viên hướng dẫn về tính trung thực, tính nguyên bản cũng như độ chính xác của toàn bộ nội dung trong báo cáo này. Nếu phát hiện bất kỳ sai phạm nào liên quan đến tính trung thực học thuật, chúng tôi xin chịu mọi hình thức kỷ luật theo quy định.

Tác giả

Nguyễn Lê Quốc Khánh

Nguyễn Hữu Khánh

Nguyễn Minh Khang

Lời cảm ơn

Trước hết, nhóm chúng tôi xin được gửi lời cảm ơn sâu sắc đến thầy Quang và thầy Thái - những giảng viên đã tận tình hướng dẫn, định hướng và hỗ trợ chúng tôi trong suốt quá trình thực hiện đề tài. Sự chỉ dẫn tận tâm của quý thầy, từ việc định hình hướng nghiên cứu, góp ý kỹ thuật đến việc hoàn thiện nội dung báo cáo, đã giúp nhóm mở rộng kiến thức, tiếp cận đúng phương pháp và phát triển dự án một cách có hệ thống.

Bên cạnh đó, nhóm xin trân trọng cảm ơn các thầy cô trong Khoa đã cung cấp nền tảng kiến thức vững chắc thông qua chương trình học và các buổi giảng dạy, giúp nhóm có đủ năng lực để thực hiện dự án nghiên cứu này. Đồng thời, chúng tôi xin được cảm ơn những người bạn đã hỗ trợ nhóm trên con đường hoàn thành dự án.

Nhóm cũng xin gửi lời cảm ơn đến các thành viên đã luôn hợp tác, hỗ trợ và đồng hành trong suốt quá trình triển khai. Tinh thần làm việc nghiêm túc, sự trao đổi kỹ thuật liên tục và ý thức trách nhiệm của từng thành viên là yếu tố quan trọng giúp nhóm hoàn thành đúng tiến độ và mục tiêu đề ra.

Chúng tôi cũng ghi nhận sự hỗ trợ gián tiếp từ các nguồn tài nguyên học thuật, công cụ phần mềm, nền tảng dữ liệu mở và các công cụ phân tích đã được sử dụng trong quá trình thực nghiệm. Những nguồn tài nguyên này đã giúp nhóm mở rộng khả năng nghiên cứu và triển khai các ý tưởng một cách hiệu quả hơn.

Một lần nữa, nhóm xin gửi lời cảm ơn chân thành đến tất cả những sự hỗ trợ quý báu nói trên.

Tác giả

Nguyễn Lê Quốc Khánh

Nguyễn Hữu Khánh

Nguyễn Minh Khang

Tóm tắt

Tình trạng giao thông đô thị là một trong những vấn đề trọng yếu tại các thành phố lớn. Đặc biệt tại thành phố Hồ Chí Minh, lưu lượng phương tiện giao thông liên tục tăng, gây ra áp lực lớn lên mạng lưới giao thông hiện hữu. Báo cáo này trình bày quá trình nghiên cứu, thu thập và phân tích dữ liệu nhằm đánh giá tình trạng giao thông trong đô thị, đồng thời khám phá mối quan hệ giữa các đoạn đường và khu vực trong mạng lưới giao thông.

Trong dự án này, dữ liệu giao thông được thu thập từ các nguồn số hóa hiện có, trải qua quá trình tiền xử lý nhằm chuẩn hóa, làm sạch và cấu trúc hóa dữ liệu để phục vụ cho phân tích. Sau đó, nhóm sẽ tiến hành khảo sát nhiều phương pháp phân tích và mô hình hóa khác nhau, từ các kỹ thuật phân tích truyền thống cho đến các mô hình học sâu hiện đại, nhằm tìm ra hướng tiếp cận phù hợp cho việc nhận diện xu hướng, đánh giá mức độ ùn tắc và phát hiện mối liên hệ giữa các đoạn đường.

Hệ thống được đề xuất hướng tới việc tích hợp kết quả phân tích vào một nền tảng trực tuyến về giao thông hiện có, giúp người dùng và cơ quan quản lý dễ dàng theo dõi tình trạng giao thông, xem các biểu đồ trực quan và đánh giá các chỉ số quan trọng theo thời gian và không gian.

Kết quả nghiên cứu không chỉ cung cấp một quy trình phân tích mang tính tổng quát, có khả năng mở rộng sang nhiều loại dữ liệu khác nhau, mà còn đưa ra các gợi ý về hướng tiếp cận mô hình hóa, giúp hỗ trợ cho các nghiên cứu chuyên sâu hơn trong tương lai. Dự án góp phần minh chứng tiềm năng của dữ liệu số và các kỹ thuật phân tích hiện đại trong việc nâng cao hiệu quả quản lý giao thông đô thị.

Mục lục

1	Giới thiệu	1
1.1	Động cơ của đề tài	1
1.2	Mục tiêu	3
1.3	Phạm vi	4
1.3.1	Phạm vi không gian	4
1.3.2	Phạm vi thời gian	4
1.3.3	Phạm vi kỹ thuật	4
1.3.4	Giới hạn của đề tài	5
1.3.5	Đối tượng thực hiện và thụ hưởng	5
1.4	Ý nghĩa	6
1.4.1	Ý nghĩa thực tiễn	6
1.4.2	Ý nghĩa khoa học	6
1.5	Cấu trúc báo cáo	8
2	Kiến thức và công nghệ nền tảng	10
2.1	Tổng quan về phân tích tương quan giao thông đô thị	10
2.1.1	Đặc điểm giao thông đô thị và các thách thức	10
2.1.1.a	Đặc điểm giao thông tại TP.HCM	10
2.1.1.b	Hiện tượng lan truyền tắc nghẽn	11
2.1.2	Phân tích tương quan không gian–thời gian	11
2.1.2.a	Tương quan không gian (Spatial Dependence)	11
2.1.2.b	Tương quan thời gian (Temporal Dependence)	12
2.1.2.c	Mô hình hóa tương quan động	12
2.2	Lý thuyết đồ thị và Graph Neural Networks	14
2.2.1	Biểu diễn mạng giao thông dưới dạng đồ thị	14
2.2.1.a	Định nghĩa đồ thị giao thông	14

2.2.1.b	Đặc trưng nút và cạnh	14
2.2.2	Graph Convolutional Networks (GCN)	14
2.2.2.a	Phép tích chập trên đồ thị	14
2.2.2.b	Tính chất và ưu điểm của GCN	15
2.2.3	Graph Attention Networks (GAT)	15
2.2.3.a	Multi-head Attention	16
2.3	Mô hình học chuỗi thời gian	16
2.3.1	Recurrent Neural Networks (RNN)	16
2.3.1.a	Kiến trúc RNN cơ bản	16
2.3.1.b	Vấn đề Vanishing/Exploding Gradients	17
2.3.2	Long Short-Term Memory (LSTM)	18
2.3.2.a	Kiến trúc LSTM	18
2.3.3	Gated Recurrent Unit (GRU)	18
2.3.3.a	Kiến trúc GRU	18
2.3.4	So sánh LSTM và GRU	19
2.4	Mô hình Transformer và BERT	20
2.4.1	Kiến trúc Transformer	20
2.4.1.a	Cơ chế Self-Attention	20
2.4.1.b	Multi-Head Attention	20
2.4.1.c	Positional Encoding	21
2.4.2	BERT: Bidirectional Encoder Representations from Transformers	21
2.4.2.a	Kiến trúc BERT	21
2.4.2.b	Chiến lược Pre-training của BERT	22
2.4.2.c	Fine-tuning BERT	22
2.4.3	Ưu điểm của BERT trong phân tích chuỗi thời gian	22
2.4.4	Áp dụng BERT cho dữ liệu giao thông	23
2.4.4.a	Tokenization cho chuỗi thời gian	23

2.4.4.b	Spatial-Temporal BERT	23
2.4.4.c	BERT-GNN Hybrid Model	24
2.4.5	So sánh BERT với các phương pháp khác	24
2.5	Công nghệ nền tảng và framework	25
2.5.1	Python và hệ sinh thái Deep Learning	25
2.5.1.a	Python trong phân tích dữ liệu và AI	25
2.5.1.b	PyTorch	25
2.5.2	Kiến trúc Kappa cho xử lý dữ liệu streaming	25
2.5.2.a	Tổng quan về Kappa Architecture	25
2.5.2.b	Apache Kafka làm backbone	26
2.5.2.c	So sánh Kappa với Lambda Architecture	27
2.5.2.d	Ứng dụng Kappa trong hệ thống giao thông	27
2.5.3	TomTom Historical Traffic Stats API	28
2.5.4	Cơ sở dữ liệu và lưu trữ	29
2.5.4.a	Lưu trữ dữ liệu dạng cột với Apache Parquet	29
2.5.4.b	Ứng dụng Parquet trong hệ thống giao thông	29
2.5.5	API và Web Development Stack	30
2.5.5.a	FastAPI	30
2.5.5.b	Streamlit	31
2.5.6	Containerization và Orchestration	31
2.5.6.a	Docker	31
2.5.6.b	Kubernetes	32
2.6	Tóm tắt stack công nghệ	33
2.7	Kết chương	34
3	Công trình liên quan	36
3.1	Các nghiên cứu liên quan	36

3.1.1	Mô hình hóa cơ chế lan truyền tắc nghẽn giao thông	36
3.1.1.a	Mô hình lan truyền dạng sóng	36
3.1.1.b	Mô hình lây lan và quá trình lan truyền	36
3.1.1.c	Xác định tuyến và nút ảnh hưởng	37
3.2	Các mô hình học sâu trên đồ thị giao thông	37
3.2.1	Đồ thị tĩnh và các mô hình GCN truyền thống	37
3.2.2	Học quan hệ động và Attention trên đồ thị	37
3.2.3	Transformer cho biểu diễn đồ thị	38
3.3	Các hệ thống và nền tảng giao thông thực tế	38
3.4	Khoảng trống nghiên cứu	38
4	Phương pháp đề xuất	40
4.1	Tổng quan phương pháp và động cơ lựa chọn mô hình	40
4.1.1	Định hướng giải pháp	40
4.1.2	Quy trình xử lý chính	41
4.2	Thu thập và chuẩn bị dữ liệu	42
4.2.1	Nguồn dữ liệu: TomTom Historical Traffic Stats	42
4.2.1.a	Lý do lựa chọn TomTom API	42
4.2.1.b	Cấu trúc dữ liệu TomTom	42
4.2.2	Lựa chọn khu vực nghiên cứu	43
4.2.2.a	Tiêu chí lựa chọn	43
4.2.2.b	Khu vực được chọn	44
4.2.3	Quy trình thu thập dữ liệu	44
4.2.3.a	Kiến trúc Data Pipeline	44
4.2.3.b	Các bước thu thập	44
4.2.3.c	Error Handling và Retry Strategy	46
4.2.3.d	Tối ưu hóa hiệu suất	47

4.2.4	Tiền xử lý dữ liệu	47
4.2.4.a	Xử lý missing values	47
4.2.4.b	Outlier detection và xử lý	48
4.2.4.c	Data validation	48
4.2.4.d	Conversion và chuẩn hóa đơn vị	49
4.2.4.e	Lưu trữ trung gian	50
4.2.5	Pipeline tích hợp	50
4.3	Mối tương quan giữa các tuyến trong giao thông đô thị	52
4.4	Xây dựng vùng giao thông động	53
4.4.1	Phát hiện cụm ứ tắc theo snapshot	53
4.4.2	Chiến lược chọn seed vùng	53
4.4.3	Mở rộng vùng theo cấu trúc mạng lưới	54
4.4.4	Lọc vùng theo bán kính không gian	54
4.4.5	Vùng giao thông chức năng	54
4.5	Khai thác tương quan giao thông và xác định tuyến mục tiêu	55
4.5.1	Tương quan chéo có độ trễ trong vùng giao thông	55
4.5.2	Lọc liên kết tương quan theo tiêu chí thống kê và không gian	55
4.5.3	Xác định tập tuyến mục tiêu	56
4.6	Biểu diễn không gian bằng Laplacian Positional Encoding	56
4.6.1	Xây dựng đồ thị không gian trong vùng giao thông	56
4.6.2	Laplacian và phân rã phổ	56
4.6.3	Ý nghĩa của eigenvector bậc thấp trong bối cảnh giao thông	57
4.6.4	Tích hợp Laplacian Positional Encoding vào Transformer	57
4.7	Biểu diễn thời gian bằng Temporal Encoding	58
4.7.1	Mã hóa vị trí tương đối theo bước thời gian	58
4.7.2	Mã hóa chu kỳ theo thời điểm trong ngày và ngày trong tuần	58
4.7.3	Kết hợp Temporal Encoding vào embedding đầu vào	59

5	Kiến trúc mô hình	59
5.1	Đầu vào của mô hình	59
5.2	Attention score bias trong Transformer	60
5.3	Transformer encoder-only theo phong cách BERT	61
5.4	Lớp dự đoán MLP	62
6	Kết quả thực nghiệm và đánh giá	62
6.1	Thiết lập bài toán và tiêu chí đánh giá	62
6.2	Quá trình huấn luyện và hội tụ	64
6.3	Đánh giá định lượng trên tập kiểm tra	67
6.4	Phân tích đường cong PR và ROC	68
6.5	Phân tích ma trận nhầm lẫn	70
6.6	Một số số liệu phản ánh khả năng học và dự đoán của mô hình sau 30 phút	71
6.7	Thảo luận	71
7	Hệ thống đề xuất	74
7.1	Mô tả hệ thống	74
7.1.1	Mục đích của hệ thống	74
7.1.2	Nhóm người dùng mục tiêu	74
7.1.3	Tổng quan	75
7.2	Yêu cầu	75
7.2.1	Yêu cầu chức năng	75
7.2.2	Yêu cầu phi chức năng	76
7.2.3	Yêu cầu dữ liệu	77
7.2.3.a	Loại dữ liệu cần thu thập	77
7.2.3.b	Yêu cầu về định dạng và lưu trữ dữ liệu	78
7.2.3.c	Yêu cầu về chất lượng dữ liệu	78
7.2.3.d	Yêu cầu về cập nhật dữ liệu	79

8	Phân tích & Thiết kế hệ thống	80
8.1	Kiến trúc tổng thể hệ thống	80
8.2	Tầng thu thập dữ liệu (Traffic Data Sources)	81
8.3	Tầng xử lý luồng (Kappa Architecture – Stream Processing Engine)	82
8.3.1	Data Ingestion Layer	82
8.3.2	Stream Consumer & Data Validation	82
8.3.3	Road Topology Builder (OSM) & Map Matcher	83
8.3.4	Feature Engineering	83
8.3.5	Binary .npz Formatter & Time-Series Sync	83
8.4	Tầng lưu trữ lai ghép (Hybrid Storage System)	84
8.4.1	Object Store – AWS S3 (Data Lake)	84
8.4.2	Fast Access & DB Layer	84
8.4.3	Redis Cache Layer	84
8.5	Tầng học máy không gian – thời gian (Spatio-Temporal Learning Engine)	85
8.5.1	Mô hình T-GCN	85
8.5.2	Correlation Jobs	86
8.5.3	Prediction Jobs	86
8.5.4	MLflow Experiment Tracking	86
8.6	Tầng phục vụ (Serving Layer)	86
8.6.1	Backend – FastAPI	86
8.6.2	Nginx / Reverse Proxy	87
8.6.3	Frontend – ReactJS (UTraffic Platform)	87
8.7	Tầng giám sát – DevOps (Monitoring & Experimentation)	87
8.8	Luồng dữ liệu tổng thể	87
8.9	Thiết kế cơ sở dữ liệu	89
8.9.1	Nhóm bảng xác thực người dùng (Authentication)	91

8.9.2	Nhóm bảng topo học đồ thị đường bộ (Road Graph)	91
8.9.3	Nhóm bảng dữ liệu giao thông (Traffic Data)	93
8.9.4	Nhóm bảng học máy và phiên bản mô hình (ML & Model Versioning)	94
8.9.5	Nhóm bảng phân tích tương quan (Correlation Analysis)	96
8.9.6	Chiến lược index và tối ưu hóa	98
8.9.7	Tóm tắt thiết kế cơ sở dữ liệu	98
8.10	Thiết kế giao diện	99
8.10.1	Mục tiêu thiết kế	99
8.10.2	Trang chủ	99
8.10.3	Trang đăng nhập và đăng ký	100
8.10.4	Trang dữ liệu giao thông	100
8.10.5	Trang phân tích tương quan	101
8.10.6	Trang dự đoán lan truyền ùn tắc	101
9	Tổng kết	103
9.1	Kết quả đạt được	103
9.2	Hướng phát triển	103
10	Phụ lục	109
	PHỤ LỤC A. DANH MỤC THUẬT NGỮ VÀ KÝ HIỆU	109
	PHỤ LỤC B. MÔ TẢ CẤU TRÚC DỮ LIỆU TOMTOM	109
	PHỤ LỤC C. SƠ ĐỒ ERD (CHEN NOTATION)	110
	PHỤ LỤC D. DANH SÁCH USE CASE ÁNH XẠ YÊU CẦU	111
	PHỤ LỤC E. CÔNG CỤ, THƯ VIỆN VÀ CẤU HÌNH SỬ DỤNG	111

Danh sách hình vẽ

1	Kiến trúc Kappa với BERT Model Integration	26
2	Kiến trúc Data Pipeline thu thập và xử lý dữ liệu	45
3	Kiến trúc mô hình Transformer encoder-only với Spatial và Temporal En- coding	61
4	Kết quả của Epoch 149	64
5	Kết quả của Epoch 150	65
6	Hàm loss qua các epoch	66
7	Accuracy, Precision, Recall, F1-score trên train, valid, test qua các epoch .	67
8	Precision-Recall curve	68
9	ROC,AUC	69
10	Ma trận nhầm lẫn	70
11	Epoch 149	71
12	Hàm loss qua các epoch	72
13	Precision-Recall curve	72
14	Precision-Recall curve	73
15	Kiến trúc tổng thể hệ thống UTraffic	81
16	Sơ đồ ERD cơ sở dữ liệu hệ thống UTraffic	90
17	Sơ đồ ERD dữ liệu giao thông theo Chen notation	110

Danh sách bảng

1	So sánh LSTM và GRU	19
2	So sánh BERT với RNN/LSTM/GRU	24
3	So sánh Kappa và Lambda Architecture	27
4	Stack công nghệ hệ thống	33
5	Cấu trúc lưu trữ dữ liệu Parquet	50
6	Kết quả định lượng trên tập test (dự đoán 15 phút sau).	67
7	Cấu trúc bảng <code>users</code>	91
8	Cấu trúc bảng <code>refresh_tokens</code>	91
9	Cấu trúc bảng <code>nodes</code>	92
10	Cấu trúc bảng <code>edges</code>	92
11	Cấu trúc bảng <code>traffic_observations</code> (TimescaleDB hypertable)	93
12	Cấu trúc bảng <code>model_versions</code>	95
13	Cấu trúc bảng <code>predictions</code>	96
14	Cấu trúc bảng <code>correlation_snapshots</code>	97
15	Cấu trúc bảng <code>node_correlations</code>	97
16	Tổng hợp các bảng trong hệ thống UTraffic	98

1 Giới thiệu

1.1 Động cơ của đề tài

Trong bối cảnh toàn cầu hóa và đô thị hóa diễn ra mạnh mẽ, Thành phố Hồ Chí Minh (TP.HCM) với vai trò là trung tâm kinh tế lớn nhất Việt Nam đang đối mặt với nhiều thách thức về quản lý đô thị, trong đó vấn đề giao thông là một trong những bài toán nan giải nhất. Theo báo cáo từ Sở Giao thông Vận tải TP.HCM năm 2023, trung bình mỗi ngày thành phố ghi nhận hơn 1.000 vụ ùn tắc giao thông cục bộ với khoảng 1,5 triệu ô tô và hơn 8 triệu xe máy lưu thông trong giờ cao điểm. Tình trạng này không chỉ gây áp lực khổng lồ lên hạ tầng giao thông vốn đã quá tải mà còn tạo ra những hệ lụy nghiêm trọng về kinh tế - xã hội.

Về mặt kinh tế, ùn tắc giao thông làm tăng chi phí logistics, gây chậm trễ trong vận chuyển hàng hóa và giảm năng suất lao động do người lao động mất nhiều thời gian di chuyển. Về mặt xã hội, nó ảnh hưởng trực tiếp đến chất lượng cuộc sống của người dân thông qua việc tăng thời gian đi lại, gây căng thẳng tâm lý, và làm gia tăng ô nhiễm không khí do phương tiện phải hoạt động với hiệu suất thấp trong trạng thái dừng - chạy liên tục.

Một đặc điểm quan trọng nhưng thường bị bỏ qua trong bài toán giao thông đô thị là **hiện tượng lan truyền tắc nghẽn** (cascading congestion). Khi một nút giao thông quan trọng bị tắc nghẽn, nó không chỉ gây ảnh hưởng cục bộ mà còn tạo ra hiệu ứng dây chuyền lan rộng đến các nút giao thông lân cận và xa hơn trong mạng lưới. Hiện tượng này diễn ra do các tuyến đường trong đô thị có mối liên hệ chặt chẽ với nhau: dòng xe từ nút A sẽ di chuyển đến nút B, rồi tiếp tục đến nút C, tạo thành một chuỗi phụ thuộc phức tạp. Việc hiểu rõ và định lượng được các mối liên hệ này là then chốt để có thể dự báo, cảnh báo sớm và đưa ra giải pháp điều phối giao thông hiệu quả.

Tuy nhiên, qua quan sát thực tế tại TP.HCM, nhóm nghiên cứu nhận thấy rằng phần lớn hoạt động điều phối và giảm ùn tắc giao thông hiện nay vẫn mang tính **phản ứng thụ động** hơn là **dự báo chủ động**. Các cơ quan chức năng thường chỉ vào cuộc xử lý khi đã nhận được tin báo về tình trạng mất trật tự hoặc dựa vào kinh nghiệm về những đoạn đường thường xuyên xảy ra tắc nghẽn. Điều này cho thấy mặc dù đã có các hệ thống giám sát và thu thập dữ liệu giao thông như UTraffic, nhưng dữ liệu này chưa được khai thác và phân tích một cách tối ưu để chuyển hóa thành các thông tin hữu ích phục vụ công tác dự báo và ra quyết định.

Chính từ những nhận định trên, nhóm nghiên cứu quyết định hướng đến giải pháp **phân tích mối liên hệ về tình trạng giao thông giữa các nút giao thông quan trọng tại TP.HCM**.

Với việc tích hợp các phương pháp phân tích hiện đại như Graph Neural Networks (GNN), phân tích chuỗi thời gian, và các kỹ thuật Big Data, đề tài hướng đến mục tiêu xây dựng một hệ thống có khả năng:

- Phát hiện và định lượng mối tương quan giữa các nút giao thông trong mạng lưới đô thị.
- Xác định các mối quan hệ nhân quả (causal relationships) để biết nút giao nào là nguồn gốc gây ra tắc nghẽn lan truyền.
- Cảnh báo sớm khi phát hiện nguy cơ lan truyền tắc nghẽn từ một nút sang các nút khác.
- Cung cấp công cụ trực quan hóa giúp các nhà quản lý giao thông có cái nhìn toàn cảnh về mạng lưới giao thông.

Đây là một hướng tiếp cận mang tính chiến lược và lâu dài, góp phần chuyển đổi công tác quản lý giao thông từ mô hình phản ứng sang mô hình chủ động dựa trên dữ liệu và trí tuệ nhân tạo, từ đó cải thiện chất lượng cuộc sống của người dân TP.HCM.

1.2 Mục tiêu

Mục tiêu tổng quát của đề tài là nghiên cứu và phát triển giải pháp phân tích mối liên hệ về tình trạng giao thông tại các nút giao thông quan trọng ở TP.HCM dựa trên dữ liệu thực tế, từ đó cung cấp cơ sở khoa học cho công tác dự báo, cảnh báo sớm và ra quyết định trong quản lý giao thông đô thị. Cụ thể, đề tài hướng đến các mục tiêu sau:

- **Khảo sát và đánh giá các giải pháp hiện có:** Tổng hợp, phân tích các nghiên cứu và giải pháp phân tích, dự báo giao thông đang được áp dụng trong và ngoài nước. Từ đó xác định phương pháp phù hợp với điều kiện thực tế tại TP.HCM, đặc biệt là bối cảnh giao thông hỗn hợp với mật độ xe máy cao.
- **Phát triển mô hình phân tích mối quan hệ không gian - thời gian:** Xây dựng mô hình có khả năng học và định lượng mối tương quan giữa các nút giao thông trong mạng lưới đô thị. Mô hình cần phát hiện được các cặp nút có độ ảnh hưởng cao lẫn nhau, xác định độ trễ lan truyền (propagation delay), và nhận diện các nút giao thông đóng vai trò quan trọng (hub nodes) trong việc gây ra hoặc kiểm soát tắc nghẽn.
- **Xây dựng hệ thống phân tích và cảnh báo:** Phát triển một hệ thống demo tích hợp vào UTraffic với các chức năng:
 - Thu thập và xử lý dữ liệu giao thông thời gian thực.
 - Phân tích và trực quan hóa mối liên hệ giữa các nút giao thông trên bản đồ.
 - Cảnh báo sớm khi phát hiện nguy cơ lan truyền tắc nghẽn.
 - Cung cấp báo cáo phân tích hỗ trợ ra quyết định.
- **Đánh giá hiệu quả trên dữ liệu thực tế:** Kiểm chứng tính khả thi, độ chính xác và hiệu năng của giải pháp đề xuất thông qua việc thử nghiệm trên dữ liệu giao thông thực tế của TP.HCM thu thập từ hệ thống UTraffic.

Với những mục tiêu trên, đề án không chỉ giải quyết vấn đề cụ thể về phân tích giao thông tại TP.HCM mà còn đóng góp vào việc phát triển hệ thống quản lý giao thông thông minh (Intelligent Transportation System - ITS) trong tương lai.

1.3 Phạm vi

Để đảm bảo tính khả thi và hiệu quả trong thời gian thực hiện đồ án, nghiên cứu được giới hạn trong các phạm vi sau:

1.3.1 Phạm vi không gian

Đề tài tập trung vào **các nút giao thông trọng điểm tại khu vực trung tâm TP.HCM**, bao gồm các quận có mật độ giao thông cao và thường xuyên xảy ra tắc nghẽn như Quận 1, Quận 3, Quận 5, Quận 10, và một số tuyến đường huyết mạch nối liền các khu vực này. Cụ thể:

- Số lượng nút giao thông nghiên cứu: 15-25 nút giao thông quan trọng.
- Tiêu chí lựa chọn: Các nút có lưu lượng phương tiện cao, thường xuyên xảy ra ùn tắc, hoặc có vị trí chiến lược (cổng vào/ra khu vực trung tâm, giao lộ lớn).
- Các tuyến đường chính được xem xét bao gồm: đường Võ Văn Kiệt, đường Cách Mạng Tháng Tám, đường Điện Biên Phủ, đường Nguyễn Thị Minh Khai, và các trục đường nối liền chúng.

1.3.2 Phạm vi thời gian

- **Dữ liệu lịch sử:** Phân tích dữ liệu trong khoảng thời gian từ 6 tháng đến 1 năm gần nhất để đảm bảo phản ánh chính xác xu hướng và đặc điểm hiện tại của tình trạng giao thông, đồng thời có đủ dữ liệu để huấn luyện mô hình học máy.
- **Phân tích theo khung giờ:** Tập trung vào các khung giờ cao điểm (7h-9h sáng, 17h-19h chiều) và so sánh với giờ thấp điểm để đánh giá sự khác biệt về mức độ tương quan.
- **Độ phân giải thời gian:** Dữ liệu được thu thập và phân tích với độ phân giải 5-10 phút để cân bằng giữa độ chi tiết và khả năng xử lý.

1.3.3 Phạm vi kỹ thuật

- **Phương pháp phân tích:** Sử dụng các kỹ thuật Big Data (Apache Spark), Deep Learning (Graph Neural Networks, LSTM), và các phương pháp thống kê (Granger Causality Test, correlation analysis).
- **Loại dữ liệu:** Dữ liệu về tốc độ trung bình, mật độ phương tiện, thời gian di chuyển tại các nút giao thông. Không bao gồm dữ liệu cá nhân hoặc dữ liệu nhận dạng phương tiện.

- **Nền tảng tích hợp:** Hệ thống được phát triển như một module mở rộng của UTraffic, tương thích với kiến trúc hiện có.

1.3.4 Giới hạn của đề tài

Đề tài **không** bao gồm:

- Dự báo dài hạn (trên 2 giờ) về tình trạng giao thông.
- Phân tích ảnh hưởng của các yếu tố thời tiết cực đoan, sự kiện đặc biệt (lễ hội, biểu tình).
- Đề xuất giải pháp điều chỉnh hạ tầng vật lý (xây dựng đường mới, cầu vượt).
- Tối ưu hóa chu kỳ đèn tín hiệu (có thể là hướng phát triển tương lai dựa trên kết quả phân tích).

1.3.5 Đối tượng thực hiện và thụ hưởng

- **Đối tượng thực hiện:** Sinh viên ngành Công nghệ thông tin, chuyên ngành Big Data và Deep Learning.
- **Đối tượng thụ hưởng:**
 - Cơ quan quản lý giao thông đô thị (Sở GTVT TP.HCM).
 - Các đơn vị cung cấp dịch vụ giám sát, điều khiển giao thông.
 - Nhà quy hoạch và nghiên cứu về hạ tầng giao thông.
 - Người dân TP.HCM thông qua việc cải thiện tình trạng giao thông.

Việc mở rộng nghiên cứu sang các khu vực khác trong TP.HCM hoặc áp dụng cho các đô thị khác tại Việt Nam sẽ được thảo luận trong phần hướng phát triển tương lai.

1.4 Ý nghĩa

1.4.1 Ý nghĩa thực tiễn

Kết quả nghiên cứu của đề án mang lại những giá trị ứng dụng thực tế quan trọng đối với công tác quản lý và vận hành mạng lưới giao thông đô thị tại TP.HCM:

- **Hỗ trợ điều phối giao thông theo mạng lưới (Network-wide Management):** Thay vì điều tiết cục bộ tại từng nút giao độc lập, giải pháp giúp các cơ quan chức năng có "bức tranh toàn cảnh" về mối liên hệ giữa các nút giao thông. Từ đó nhận diện được nút giao nào là nguyên nhân gốc rễ (root cause) gây ra sự ùn tắc lan truyền sang các khu vực lân cận, để có phương án xử lý ưu tiên và hiệu quả hơn.
- **Cảnh báo sớm và dự báo chủ động:** Hệ thống có khả năng phát hiện các dấu hiệu ban đầu của tắc nghẽn tại một nút và dự đoán khả năng lan truyền sang các nút khác dựa trên mối quan hệ đã được học. Điều này cho phép các cơ quan chức năng can thiệp sớm, giảm thiểu thời gian và mức độ ùn tắc.
- **Cơ sở cho việc đồng bộ hóa tín hiệu đèn (Signal Synchronization):** Việc định lượng được mức độ ảnh hưởng và độ trễ của dòng giao thông giữa các nút là tham số đầu vào quan trọng để thiết lập các làn sóng xanh (Green Wave) hoặc tối ưu hóa chu kỳ đèn tín hiệu tại các giao lộ liên tiếp, giúp dòng xe di chuyển mượt mà hơn.
- **Tối ưu hóa đầu tư hạ tầng giám sát:** Hệ thống giúp xác định các cặp nút giao thông có độ tương quan cao và các nút có tầm quan trọng chiến lược (hub nodes). Thông tin này hỗ trợ quy hoạch lắp đặt thêm cảm biến, camera hoặc nâng cấp hạ tầng tại các vị trí trọng yếu, tối ưu hóa chi phí đầu tư.
- **Cung cấp thông tin hữu ích cho người tham gia giao thông:** Người dùng không chỉ biết điểm nào đang kẹt mà còn hiểu được xu hướng lan truyền của tắc nghẽn, từ đó chủ động chọn lộ trình tránh các trục đường đang chịu ảnh hưởng dây chuyền, tiết kiệm thời gian di chuyển.
- **Nâng cao hiệu quả vận hành hệ thống UTraffic:** Đề tài giúp chuyển hóa dữ liệu thô từ UTraffic thành các tri thức về cấu trúc vận hành của mạng lưới giao thông, tăng giá trị sử dụng của hệ thống hiện có mà không cần đầu tư lớn vào phần cứng mới.

1.4.2 Ý nghĩa khoa học

Về mặt nghiên cứu, đề tài đóng góp những giá trị học thuật trong việc áp dụng Trí tuệ nhân tạo và Big Data vào bài toán Giao thông thông minh (Intelligent Transportation

Systems - ITS):

- **Mô hình hóa sự phụ thuộc không gian - thời gian (Spatio-Temporal Dependency):** Đề án tập trung nghiên cứu và định lượng mối quan hệ phức tạp giữa các nút giao thông trong mạng lưới đô thị. Thay vì chỉ xét khoảng cách địa lý, nghiên cứu đề xuất phương pháp học các mối liên hệ ẩn (latent correlations) dựa trên dữ liệu dòng chảy thực tế, phản ánh đúng bản chất động của hệ thống giao thông.
- **Ứng dụng Graph Neural Networks (GNN) cho giao thông Việt Nam:** Đóng góp vào việc kiểm chứng hiệu quả của các kiến trúc học sâu trên dữ liệu đồ thị (graph-based data) đối với đặc thù giao thông hỗn hợp tại Việt Nam - nơi mà các quy luật di chuyển thường phức tạp và khó nắm bắt hơn so với các nước phát triển do sự đa dạng về loại phương tiện (xe máy, ô tô, xe buýt) và văn hóa tham gia giao thông.
- **Phân tích quan hệ nhân quả trong mạng lưới giao thông:** Áp dụng các phương pháp thống kê tiên tiến như Granger Causality Test để không chỉ phát hiện tương quan mà còn xác định quan hệ nhân - quả giữa các nút giao thông. Đây là hướng nghiên cứu quan trọng nhưng chưa được khai thác nhiều trong các công trình về giao thông tại Việt Nam.
- **Xây dựng dataset giao thông TP.HCM:** Đề tài góp phần xây dựng và công bố một dataset về giao thông đô thị TP.HCM có gắn nhãn về mối liên hệ giữa các nút giao thông, phục vụ cho cộng đồng nghiên cứu trong và ngoài nước.
- **Cơ sở cho hệ thống giao thông thích nghi (Adaptive Traffic Systems):** Việc phân tích thành công mối liên hệ động (dynamic relationships) giữa các nút giao thông sẽ là tiền đề quan trọng để phát triển các hệ thống AI có khả năng tự thích nghi với sự thay đổi cấu trúc mạng lưới giao thông theo thời gian thực, hướng tới mô hình giao thông tự trị (autonomous traffic management).
- **Kết hợp đa phương pháp (Multi-method Approach):** Đề tài tích hợp nhiều phương pháp từ các lĩnh vực khác nhau: Graph Theory, Deep Learning, Time Series Analysis, Statistical Causality, tạo ra một framework toàn diện để giải quyết bài toán phức tạp về giao thông đô thị.

1.5 Cấu trúc báo cáo

Nội dung của đề án được tổ chức và trình bày theo 9 chương chính, cụ thể như sau:

- **Chương 1 - Giới thiệu:** Trình bày bối cảnh thực hiện đề tài, xác định vấn đề cần giải quyết, mục tiêu, phạm vi nghiên cứu và ý nghĩa thực tiễn của việc phân tích mối liên hệ giao thông.
- **Chương 2 - Kiến thức và công nghệ nền tảng:** Cung cấp cơ sở lý thuyết về các mô hình học sâu trên đồ thị (GCN, GAT) và chuỗi thời gian (RNN, LSTM, GRU). Chương này cũng đi sâu vào kiến trúc T-GCN, các cải tiến hiện đại như Dynamic Graph Learning, cùng với hệ sinh thái công nghệ được sử dụng (Python, PyTorch, kiến trúc Kappa với Kafka/Flink, Docker và Kubernetes).
- **Chương 3 - Công trình liên quan:** Khảo sát các nghiên cứu về mô hình lan truyền, các phương pháp Deep Learning với ma trận kề tĩnh và động. Đồng thời, chương này phân tích các hệ thống thực tế như Google Maps, TomTom, UTraffic để xác định khoảng trống nghiên cứu (Gap Analysis) mà đề tài hướng tới giải quyết.
- **Chương 4 - Phương pháp giải quyết:** Đây là chương trọng tâm mô tả chi tiết quy trình khoa học dữ liệu và thuật toán. Nội dung bao gồm:
 - Quy trình thu thập và tiền xử lý dữ liệu từ TomTom API.
 - Trích xuất đặc trưng không gian - thời gian và phân tích tương quan (DTW, Wavelet).
 - Kiểm định quan hệ nhân quả (Granger Causality) để xây dựng đồ thị nhân quả.
 - Xây dựng mô hình dự báo Dynamic Graph Convolutional Network và chiến lược huấn luyện.
 - Thiết kế hệ thống cảnh báo, phát hiện lan truyền tắc nghẽn và quy trình triển khai (Deployment).
- **Chương 5 - Hệ thống đề xuất:** Tập trung vào khía cạnh kỹ thuật phần mềm, mô tả mục đích hệ thống, nhóm người dùng mục tiêu, và đặc tả chi tiết các yêu cầu chức năng, phi chức năng cũng như các yêu cầu khắt khe về dữ liệu đầu vào/đầu ra.
- **Chương 6 - Phân tích & Thiết kế hệ thống:** Trình bày mô hình hóa tương tác người dùng, quy trình nghiệp vụ, và thiết kế chi tiết kiến trúc giải pháp công nghệ (Frontend, Backend) để hiện thực hóa các yêu cầu đã đặt ra.

- **Chương 7 - Hiện thực và kiểm thử:** Mô tả quá trình hiện thực mã nguồn, môi trường cài đặt và các kết quả kiểm thử hệ thống để đảm bảo tính đúng đắn của các chức năng.
- **Chương 8 - Đánh giá hệ thống:** Đánh giá tổng thể hiệu quả của hệ thống sau khi triển khai thực tế hoặc trên môi trường thử nghiệm.
- **Chương 9 - Tổng kết:** Tóm tắt các kết quả đạt được, đối chiếu với mục tiêu ban đầu, nêu rõ tính mới và đóng góp của đề tài, đồng thời đề xuất các hướng phát triển trong tương lai.

Ngoài ra, báo cáo còn bao gồm danh mục tài liệu tham khảo và các phụ lục liên quan đến mã nguồn và kết quả thực nghiệm chi tiết.

2 Kiến thức và công nghệ nền tảng

2.1 Tổng quan về phân tích tương quan giao thông đô thị

2.1.1 Đặc điểm giao thông đô thị và các thách thức

Giao thông đô thị tại các thành phố lớn thường mang tính phi tuyến, đa biến và chịu ảnh hưởng đồng thời của nhiều yếu tố tự nhiên – xã hội. Sự gia tăng mạnh mẽ của phương tiện cá nhân, hạ tầng hạn chế, cùng với tính khó đoán trong hành vi tham gia giao thông khiến việc phân tích và dự báo trở thành một bài toán phức tạp. Hơn nữa, trạng thái giao thông không chỉ bị chi phối tại chỗ mà còn lan truyền theo mạng lưới đường, dẫn đến hiện tượng "tắc nghẽn dây chuyền" (spillover congestion).

Đối với các hệ thống quản lý giao thông thông minh (ITS), việc hiểu rõ bản chất tương tác không gian–thời gian giữa các nút giao là cơ sở để thiết kế mô hình dự báo có độ chính xác cao, hỗ trợ lập kế hoạch và điều tiết giao thông hiệu quả.

2.1.1.a Đặc điểm giao thông tại TP.HCM

TP.HCM là đô thị có cơ cấu phương tiện đặc thù với tỷ lệ xe máy chiếm ưu thế tuyệt đối. Đặc điểm này hình thành một môi trường giao thông không đồng nhất, giàu biến động và khó chuẩn hóa. Một số đặc điểm quan trọng của giao thông TP.HCM bao gồm:

- **Xe máy chiếm hơn 90% tổng phương tiện:** tạo nên dòng di chuyển linh hoạt, không tuân thủ nghiêm ngặt làn đường.
- **Mật độ sở hữu xe máy cao (> 350 xe/1000 dân):** gây áp lực lớn lên hạ tầng đô thị.
- **Dòng xe hỗn hợp:** ô tô, xe tải, xe buýt, xe máy di chuyển xen kẽ, làm gia tăng độ bất định của lưu lượng.
- **Tác động mạnh từ môi trường xã hội:** thời tiết, sự kiện, ùn tắc cục bộ có thể làm biến đổi trạng thái giao thông toàn khu vực.

Những yếu tố này đòi hỏi mô hình phân tích phải đủ linh hoạt để phản ánh đúng tính động, tính phi tuyến và tính bất định của hệ thống giao thông đô thị.

2.1.1.b Hiện tượng lan truyền tắc nghẽn

Tắc nghẽn giao thông không chỉ xảy ra tại một điểm mà có khả năng lan truyền đến các đoạn đường lân cận. Một đoạn đường quá tải sẽ làm giảm tốc độ, tăng mật độ phương tiện, và gây ảnh hưởng lên khả năng lưu thông đầu vào – đầu ra của các nút láng giềng. Hiện tượng này có thể được mô hình hóa như một quá trình lan truyền:

$$\frac{dc(t)}{dt} = -\mu c(t) + \beta(k-1)c(t)[1-r(t)-c(t)] \quad (1)$$

Trong đó:

- $c(t)$: tỷ lệ đoạn đường đang tắc tại thời điểm t ,
- β : tốc độ lan truyền tắc nghẽn,
- μ : tốc độ hồi phục,
- k : bậc trung bình của nút trong mạng lưới,
- $r(t)$: tỷ lệ đoạn đường đã hồi phục.

Mô hình này cho phép mô tả cách tắc nghẽn di chuyển theo mạng lưới, từ đó hỗ trợ dự báo các hiện tượng "hiệu ứng domino" trong giao thông đô thị.

2.1.2 Phân tích tương quan không gian–thời gian

Hệ thống giao thông biểu hiện đồng thời hai loại phụ thuộc quan trọng:

1. **Phụ thuộc theo không gian**: trạng thái của một đoạn đường nằm dưới ảnh hưởng của các đoạn đường lân cận.
2. **Phụ thuộc theo thời gian**: trạng thái hiện tại liên quan mật thiết đến trạng thái của chính nó ở các thời điểm trước.

Sự kết hợp giữa hai chiều phụ thuộc này tạo nên tính chất "không gian–thời gian" (spatiotemporal), đóng vai trò then chốt trong các mô hình học sâu hiện đại.

2.1.2.a Tương quan không gian (Spatial Dependence)

Tương quan không gian phản ánh mức độ ảnh hưởng lẫn nhau giữa các nút giao hoặc đoạn đường. Khi một khu vực xảy ra ùn tắc, áp lực sẽ lan truyền theo các tuyến liên thông. Hệ số tương quan giữa hai đoạn đường i và j được xác định như sau:

$$r_{ij} = \frac{\sum_{d,t} (y_{itd} - \bar{y}_i) w_{ij} (y_{jtd} - \bar{y}_j)}{\sqrt{\sum_{d,t} (y_{itd} - \bar{y}_i)^2 \sum_{d,t} (y_{jtd} - \bar{y}_j)^2}} \quad (2)$$

Trong đó, trọng số w_{ij} biểu thị sự gần gũi không gian dựa trên:

- khoảng cách địa lý,
- mức độ kết nối trực tiếp giữa hai đoạn đường,
- thời gian di chuyển trung bình,
- hoặc cấu trúc của mạng lưới giao thông.

2.1.2.b Tương quan thời gian (Temporal Dependence)

Lưu lượng giao thông biến đổi theo các quy luật thời gian rõ rệt. Hai đặc điểm nổi bật bao gồm:

- **Tính chu kỳ:** Giao thông thường lặp lại theo các khung giờ cao điểm – thấp điểm hàng ngày (sáng, trưa, chiều) hoặc các mô hình theo ngày trong tuần.
- **Tính phụ thuộc thời điểm:** Lưu lượng tại thời điểm t chịu ảnh hưởng mạnh từ trạng thái của chính nó tại các thời điểm $t - 1, t - 2, \dots$

2.1.2.c Mô hình hóa tương quan động

Mối quan hệ giữa các đoạn đường không phải là bất biến. Chúng biến đổi theo thời gian do:

- thay đổi dòng xe,
- thời tiết, mưa, ngập,
- sự kiện đột xuất,
- tai nạn hoặc phong tỏa đường.

Do đó, ma trận quan hệ giữa các nút cần được mô hình hóa dưới dạng động:

$$A_t = f_{corr}(X_t, \theta) \quad (3)$$

Trong đó:

- A_t : ma trận kề đại diện cho cấu trúc không gian tại thời điểm t ,
- X_t : đặc trưng giao thông (tốc độ, lưu lượng, mật độ, độ trễ...),
- θ : tham số mô hình học được.

2.2 Lý thuyết đồ thị và Graph Neural Networks

2.2.1 Biểu diễn mạng giao thông dưới dạng đồ thị

2.2.1.a Định nghĩa đồ thị giao thông

Một mạng giao thông đô thị có thể được mô hình hoá dưới dạng đồ thị có hướng $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$, trong đó:

- $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ là tập N nút, mỗi nút đại diện cho một đoạn đường, giao lộ hoặc điểm đo cảm biến.
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ là tập các cạnh có hướng, phản ánh mối quan hệ kết nối hoặc luồng giao thông từ đoạn đường này sang đoạn đường khác.
- $\mathcal{A} \in \mathbb{R}^{N \times N}$ là ma trận kề, mã hoá mức độ liên kết giữa các nút.

Đối với bài toán dự báo giao thông, cách xây dựng ma trận kề $\mathcal{A}$ đóng vai trò đặc biệt quan trọng vì nó quyết định mô hình học các tương tác không gian như thế nào.

2.2.1.b Đặc trưng nút và cạnh

Ma trận đặc trưng của các nút tại thời điểm t được biểu diễn bởi $\mathbf{X}^{(t)} \in \mathbb{R}^{N \times F}$, trong đó:

- N là số lượng nút trong mạng.
- F là số chiều của không gian đặc trưng, ví dụ: tốc độ trung bình, mật độ xe, lưu lượng qua trạm, thời gian chờ,...
- $\mathbf{X}_{i,:}^{(t)}$ là vector đặc trưng của nút i tại thời điểm t .

Trong một số mô hình nâng cao, đặc trưng cạnh E_{ij} cũng có thể được bổ sung, ví dụ: chiều dài đoạn đường, số làn xe, tốc độ tối đa, hoặc dữ liệu cảm biến IoT đặt dọc tuyến.

2.2.2 Graph Convolutional Networks (GCN)

2.2.2.a Phép tích chập trên đồ thị

GCN được thiết kế để mở rộng phép tích chập từ không gian Euclidean (ảnh, âm thanh) sang cấu trúc đồ thị phi Euclidean. Công thức phổ biến của GCN (theo Kipf & Welling) là:

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right), \quad (4)$$

trong đó:

- $\tilde{A} = A + I_N$ bổ sung self-loop giúp mỗi nút sử dụng thêm thông tin của chính nó,
- $\tilde{D}$ là ma trận bậc tương ứng với $\tilde{A}$,
- $\mathbf{W}^{(l)}$ là trọng số của lớp thứ l ,
- σ là hàm kích hoạt phi tuyến.

GCN cho phép khuếch tán thông tin giữa các nút theo cấu trúc không gian của mạng giao thông, đóng vai trò nền tảng cho việc mô hình hóa tương quan không gian.

2.2.2.b Tính chất và ưu điểm của GCN

1. **Chia sẻ tham số (Parameter Sharing)** Bộ lọc đồ thị sử dụng chung cho tất cả các nút, giúp giảm số tham số và tăng khả năng khái quát.
2. **Localized Receptive Field** Mỗi lớp GCN chỉ tổng hợp thông tin từ láng giềng 1-hop, đảm bảo tính “địa phương” trong mô hình hoá luồng giao thông.
3. **Mở rộng vùng tiếp nhận thông tin (Expanded Receptive Field)** Khi xếp chồng nhiều lớp GCN, mô hình có thể thu thập thông tin từ các láng giềng 2-hop, 3-hop, ... phù hợp với việc mô phỏng lan truyền tắc đường.

2.2.3 Graph Attention Networks (GAT)

GAT được đề xuất nhằm khắc phục hạn chế của GCN trong việc gán trọng số cố định cho các láng giềng. Theo đó, GAT học trọng số attention một cách động:

$$\alpha_{ij} = \frac{\exp \left(\text{LeakyReLU} \left(\mathbf{a}^T [\mathbf{W} \mathbf{h}_i \| \mathbf{W} \mathbf{h}_j] \right) \right)}{\sum_{k \in \mathcal{N}_i} \exp \left(\text{LeakyReLU} \left(\mathbf{a}^T [\mathbf{W} \mathbf{h}_i \| \mathbf{W} \mathbf{h}_k] \right) \right)}.$$

Đầu ra của một nút:

$$\mathbf{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W} \mathbf{h}_j \right).$$

Nhờ cơ chế attention, mô hình có thể nhận biết nút nào đóng vai trò quan trọng hơn trong quá trình dự báo.

2.2.3.a Multi-head Attention

GAT sử dụng K attention heads độc lập nhằm tăng tính ổn định và khả năng biểu diễn:

$$\mathbf{h}'_i = \big\|_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \mathbf{h}_j \right).$$

Việc kết hợp nhiều head giúp mô hình học được nhiều dạng quan hệ không gian khác nhau trong mạng giao thông.

2.3 Mô hình học chuỗi thời gian

Việc dự báo giao thông là một trong những bài toán tiêu biểu của phân tích chuỗi thời gian, nơi dữ liệu có tính tuần hoàn, phụ thuộc dài hạn, và bị tác động bởi các yếu tố theo mùa (seasonality) như giờ cao điểm, ngày cuối tuần hay sự kiện đặc biệt. Để mô hình hóa các quan hệ theo thời gian, các kiến trúc mạng hồi quy sâu (Recurrent Neural Networks – RNN) đóng vai trò nền tảng, trước khi phát triển thành các biến thể mạnh mẽ hơn như LSTM và GRU.

2.3.1 Recurrent Neural Networks (RNN)

2.3.1.a Kiến trúc RNN cơ bản

RNN được thiết kế để xử lý dữ liệu có tính chuỗi bằng cách duy trì một *trạng thái ẩn* (hidden state) đóng vai trò như bộ nhớ tạm của mô hình. Tại mỗi thời điểm t , mô hình tiếp nhận đầu vào $\mathbf{x}_t$, cập nhật trạng thái ẩn dựa trên trạng thái trước đó và đưa ra đầu ra tương ứng.

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h),$$

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y.$$

Trong đó:

- $\mathbf{h}_t$ lưu trữ thông tin tích lũy đến thời điểm t ,
- $\mathbf{W}_{hh}$ mô hình hóa mức độ ảnh hưởng của trạng thái trước đó lên trạng thái hiện tại,

- $\mathbf{W}_{xh}$ biểu diễn ánh xạ từ đầu vào hiện tại lên trạng thái ẩn,
- σ là hàm kích hoạt phi tuyến như tanh hoặc ReLU.

Với đặc trưng chia sẻ trọng số theo thời gian, RNN rất phù hợp để mô hình hoá các mẫu chu kỳ trong giao thông như:

- tốc độ giảm mạnh vào giờ cao điểm,
- sự lặp lại theo ngày hoặc theo tuần,
- ảnh hưởng lan truyền theo thời gian từ các điểm ùn tắc.

2.3.1.b Vấn đề Vanishing/Exploding Gradients

Khi thực hiện lan truyền ngược theo thời gian (Backpropagation Through Time – BPTT), gradient của hàm mất mát đối với trọng số có dạng:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}} \propto \prod_{i=1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}.$$

Do $\frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}$ thường chứa đạo hàm của hàm tanh hay sigmoid (giá trị trong khoảng $(0, 1)$), tích số trên dễ dàng:

- **tiệm cận 0** khi t lớn $\rightarrow$ **vanishing gradient**: mô hình “quên” thông tin xa,
- **tăng nhanh đến ∞** $\rightarrow$ **exploding gradient**: mô hình học không ổn định, dễ gây sai lệch.

Trong bối cảnh giao thông, điều này khiến RNN truyền thống khó nắm bắt được các mối quan hệ dài hạn như:

- ảnh hưởng của ùn tắc từ 30–60 phút trước,
- các xu hướng thay đổi theo ngày hoặc theo tuần.

Vì vậy, các kiến trúc cải tiến LSTM và GRU ra đời nhằm giải quyết hạn chế cốt lõi này.

2.3.2 Long Short-Term Memory (LSTM)

2.3.2.a Kiến trúc LSTM

LSTM sử dụng một cơ chế gating để kiểm soát luồng thông tin ra vào. Điều này cho phép mô hình giữ lại thông tin quan trọng trong thời gian dài và loại bỏ các thông tin nhiễu.

LSTM giới thiệu hai trạng thái:

- **Cell state C_t** đóng vai trò như “bộ nhớ dài hạn”,
- **Hidden state h_t** là đại diện ngắn hạn được dùng để dự báo.

Các cổng điều khiển:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{forget gate})$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{input gate})$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad (\text{candidate memory})$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad (\text{updated memory})$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{output gate})$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

Cơ chế này cho phép LSTM thể hiện vượt trội trong các bài toán có sự phụ thuộc dài hạn — điều đặc biệt quan trọng đối với dữ liệu giao thông vốn có tính chu kỳ dài và thường xuyên chịu ảnh hưởng bởi nhiều yếu tố trước đó.

2.3.3 Gated Recurrent Unit (GRU)

2.3.3.a Kiến trúc GRU

GRU là biến thể tinh gọn hơn của LSTM, kết hợp cell state và hidden state thành một đại lượng duy nhất. Mặc dù đơn giản hơn, GRU vẫn giữ khả năng học phụ thuộc dài hạn

nhưng sử dụng ít tham số hơn.

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{reset gate})$$

$$\mathbf{u}_t = \sigma(\mathbf{W}_u[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{update gate})$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{candidate activation})$$

$$\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t$$

Reset gate giúp xác định bao nhiêu thông tin quá khứ cần loại bỏ, trong khi update gate kiểm soát mức độ giữ lại thông tin cũ so với thông tin mới.

2.3.4 So sánh LSTM và GRU

Bảng 1: So sánh LSTM và GRU

Đặc điểm	LSTM	GRU
Số cổng	3 (forget, input, output)	2 (update, reset)
Số tham số	Nhiều hơn	Ít hơn
Thời gian huấn luyện	Chậm hơn	Nhanh hơn
Khả năng lưu trữ dài hạn	Rất tốt	Tốt
Dữ liệu nhỏ / hạn chế	Kém phù hợp	Hiệu quả hơn
Khả năng khái quát hóa	Cao	Tùy bài toán

Nhìn chung:

- LSTM phù hợp với bài toán có cấu trúc phức tạp và chuỗi dài,
- GRU là lựa chọn nhẹ hơn, thời gian huấn luyện nhanh hơn, phù hợp trong môi trường sản xuất (production) hoặc dữ liệu hạn chế.

Trong dự báo giao thông, lựa chọn giữa LSTM và GRU phụ thuộc vào:

- mức độ nhiễu của dữ liệu cảm biến,
- chiều dài chuỗi đầu vào,
- tài nguyên tính toán,
- yêu cầu latency của ứng dụng thời gian thực.

2.4 Mô hình Transformer và BERT

2.4.1 Kiến trúc Transformer

2.4.1.a Cơ chế Self-Attention

Transformer là kiến trúc học sâu dựa trên cơ chế attention, được đề xuất bởi Vaswani và cộng sự năm 2017. Khác với RNN/LSTM xử lý tuần tự, Transformer có thể xử lý toàn bộ chuỗi đầu vào song song thông qua cơ chế self-attention.

Cho một chuỗi đầu vào $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T]$, self-attention tính toán như sau:

$$\mathbf{Q} = \mathbf{XW}^Q, \quad \mathbf{K} = \mathbf{XW}^K, \quad \mathbf{V} = \mathbf{XW}^V \quad (5)$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (6)$$

Trong đó:

- $\mathbf{Q}$ (Query), $\mathbf{K}$ (Key), $\mathbf{V}$ (Value) là các phép chiếu tuyến tính của đầu vào
- d_k là chiều của không gian key, dùng để chuẩn hóa
- Softmax tạo ra phân phối xác suất thể hiện mức độ quan trọng của mỗi vị trí

Cơ chế này cho phép mỗi vị trí trong chuỗi "chú ý" đến tất cả các vị trí khác, nắm bắt được mối quan hệ phụ thuộc dài hạn mà không gặp vấn đề vanishing gradient như RNN.

2.4.1.b Multi-Head Attention

Để tăng khả năng biểu diễn, Transformer sử dụng nhiều attention heads song song:

$$\text{head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V) \quad (7)$$

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (8)$$

Mỗi head học một khía cạnh khác nhau của mối quan hệ giữa các vị trí trong chuỗi.

2.4.1.c Positional Encoding

Vì Transformer xử lý song song, nó không có thông tin về thứ tự trong chuỗi. Do đó, positional encoding được thêm vào:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (9)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (10)$$

Điều này giúp mô hình phân biệt được vị trí tương đối và tuyệt đối của các phần tử trong chuỗi.

2.4.2 BERT: Bidirectional Encoder Representations from Transformers

2.4.2.a Kiến trúc BERT

BERT là mô hình pre-training dựa trên Transformer encoder, được giới thiệu bởi Devlin và cộng sự (Google AI) năm 2018. Khác với các mô hình ngôn ngữ truyền thống chỉ đọc văn bản theo một chiều (trái sang phải hoặc phải sang trái), BERT đọc theo cả hai chiều (bidirectional) để hiểu ngữ cảnh đầy đủ.

Cấu trúc cơ bản:

- BERT-Base: 12 lớp Transformer, 768 hidden units, 12 attention heads, 110M parameters
- BERT-Large: 24 lớp Transformer, 1024 hidden units, 16 attention heads, 340M parameters

Đầu vào của BERT là một chuỗi token được mã hóa thành:

$$\mathbf{E} = \mathbf{E}_{token} + \mathbf{E}_{segment} + \mathbf{E}_{position} \quad (11)$$

Trong đó:

- $\mathbf{E}_{token}$: embedding của token
- $\mathbf{E}_{segment}$: embedding phân biệt các câu khác nhau
- $\mathbf{E}_{position}$: positional encoding

2.4.2.b Chiến lược Pre-training của BERT

BERT sử dụng hai nhiệm vụ pre-training:

1. Masked Language Model (MLM):

Ngẫu nhiên che (mask) 15% tokens trong chuỗi đầu vào và huấn luyện mô hình dự đoán các token bị che:

$$\mathcal{L}_{MLM} = - \sum_{i \in \mathcal{M}} \log P(x_i | \mathbf{x}_{\setminus \mathcal{M}}) \quad (12)$$

Trong đó $\mathcal{M}$ là tập các vị trí bị mask.

2. Next Sentence Prediction (NSP):

Dự đoán xem câu B có phải là câu tiếp theo của câu A hay không:

$$\mathcal{L}_{NSP} = - \log P(IsNext|[CLS], \text{SentA}, \text{SentB}) \quad (13)$$

Token đặc biệt [CLS] được thêm vào đầu mỗi chuỗi và đại diện cho toàn bộ chuỗi trong các tác vụ phân loại.

2.4.2.c Fine-tuning BERT

Sau pre-training, BERT có thể được fine-tune cho các tác vụ downstream cụ thể bằng cách:

- Thêm một lớp output phù hợp (classification, regression, sequence labeling...)
- Huấn luyện end-to-end trên dữ liệu của tác vụ đích
- Sử dụng learning rate nhỏ để tránh "catastrophic forgetting"

2.4.3 Ưu điểm của BERT trong phân tích chuỗi thời gian

1. **Học biểu diễn ngữ cảnh hai chiều:** BERT xem xét cả thông tin quá khứ và tương lai, phù hợp với việc phân tích mối quan hệ phức tạp trong dữ liệu giao thông.
2. **Khả năng xử lý song song:** Khác với RNN/LSTM xử lý tuần tự, BERT có thể xử lý toàn bộ chuỗi đầu vào đồng thời, tăng tốc độ huấn luyện và inference.

3. **Nắm bắt phụ thuộc dài hạn:** Self-attention cho phép mô hình kết nối trực tiếp các vị trí xa nhau trong chuỗi, vượt trội hơn LSTM trong việc học các mẫu chu kỳ dài.
4. **Pre-training và transfer learning:** BERT có thể được pre-train trên dữ liệu lớn và fine-tune trên tác vụ cụ thể với dữ liệu hạn chế.
5. **Khả năng tích hợp với Graph:** BERT có thể kết hợp với GNN để tạo nên các mô hình hybrid mạnh mẽ cho dữ liệu đồ thị có tính thời gian.

2.4.4 Áp dụng BERT cho dữ liệu giao thông

2.4.4.a Tokenization cho chuỗi thời gian

Trong bài toán giao thông, mỗi "token" có thể là:

- Một bước thời gian (time step) với vector đặc trưng tương ứng
- Một cửa sổ thời gian được tổng hợp (sliding window)
- Một đoạn đường tại một thời điểm cụ thể

Vector đặc trưng cho mỗi token có thể bao gồm: tốc độ, mật độ, lưu lượng, thời tiết, ngày trong tuần, v.v.

2.4.4.b Spatial-Temporal BERT

Để áp dụng BERT vào dữ liệu giao thông có tính không gian-thời gian, ta cần:

1. Encoding không gian:

$$\mathbf{E}_{spatial}(v_i) = \text{Learnable_Embedding}(i) \quad \text{hoặc} \quad \text{GCN}(\mathbf{A}, \mathbf{X}_i) \quad (14)$$

2. Encoding thời gian:

$$\mathbf{E}_{temporal}(t) = \mathbf{E}_{position}(t) + \mathbf{E}_{hour}(t) + \mathbf{E}_{day}(t) + \mathbf{E}_{week}(t) \quad (15)$$

3. Kết hợp:

$$\mathbf{E}(v_i, t) = \mathbf{X}_{i,t} + \mathbf{E}_{spatial}(v_i) + \mathbf{E}_{temporal}(t) \quad (16)$$

2.4.4.c BERT-GNN Hybrid Model

Mô hình kết hợp có thể được thiết kế theo các cách:

Phương án 1: GNN trước, BERT sau

$$\mathbf{H}_t^{spatial} = \text{GCN}(\mathbf{A}, \mathbf{X}_t) \quad (17)$$

$$\mathbf{H}^{temporal} = \text{BERT}([\mathbf{H}_1^{spatial}, \dots, \mathbf{H}_T^{spatial}]) \quad (18)$$

Phương án 2: BERT trước, GNN sau

$$\mathbf{H}_i^{temporal} = \text{BERT}([\mathbf{X}_{i,1}, \dots, \mathbf{X}_{i,T}]) \quad (19)$$

$$\mathbf{H}^{output} = \text{GCN}(\mathbf{A}, [\mathbf{H}_1^{temporal}, \dots, \mathbf{H}_N^{temporal}]) \quad (20)$$

Phương án 3: Xen kẽ GNN và BERT

$$\mathbf{H}^{(l+1)} = \text{BERT}(\text{GCN}(\mathbf{H}^{(l)})) \quad (21)$$

2.4.5 So sánh BERT với các phương pháp khác

Bảng 2: So sánh BERT với RNN/LSTM/GRU

Tiêu chí	RNN/LSTM/GRU	BERT
Xử lý tuần tự	Có	Không (song song)
Tốc độ huấn luyện	Chậm	Nhanh
Phụ thuộc dài hạn	Khó (LSTM/GRU tốt hơn RNN)	Rất tốt
Ngữ cảnh hai chiều	Không (trừ Bi-LSTM)	Có
Số tham số	Ít đến trung bình	Nhiều
Yêu cầu dữ liệu	Thấp đến trung bình	Cao (pre-training)
Transfer learning	Khó	Dễ dàng
Khả năng diễn giải	Tương đối tốt	Khó hơn

2.5 Công nghệ nền tảng và framework

2.5.1 Python và hệ sinh thái Deep Learning

2.5.1.a Python trong phân tích dữ liệu và AI

Python đã trở thành ngôn ngữ lập trình tiêu chuẩn cho các ứng dụng Machine Learning và Deep Learning nhờ tính đơn giản, dễ học và hệ sinh thái thư viện phong phú. Đối với bài toán dự báo giao thông, Python cung cấp đầy đủ công cụ từ xử lý dữ liệu, huấn luyện mô hình đến triển khai sản phẩm.

Các ưu điểm chính bao gồm cú pháp rõ ràng giúp code dễ đọc và bảo trì, thư viện phong phú như NumPy, Pandas, Scikit-learn, PyTorch, TensorFlow, Transformers, cộng đồng lớn hỗ trợ tốt, và khả năng tích hợp với nhiều hệ thống và database khác nhau.

2.5.1.b PyTorch

PyTorch Core

PyTorch là framework deep learning được phát triển bởi Meta AI, nổi bật với cơ chế dynamic computational graph cho phép debug và thử nghiệm linh hoạt. PyTorch cung cấp các thành phần cốt lõi như tensor operations với hỗ trợ GPU, module system để xây dựng mạng neural network, các thuật toán tối ưu (AdamW, SGD, RMSprop), và cơ chế automatic differentiation cho backpropagation.

Kiến trúc của PyTorch được thiết kế theo nguyên tắc modular, cho phép research scientists dễ dàng thử nghiệm các ý tưởng mới trong khi vẫn đảm bảo hiệu năng cao khi triển khai production.

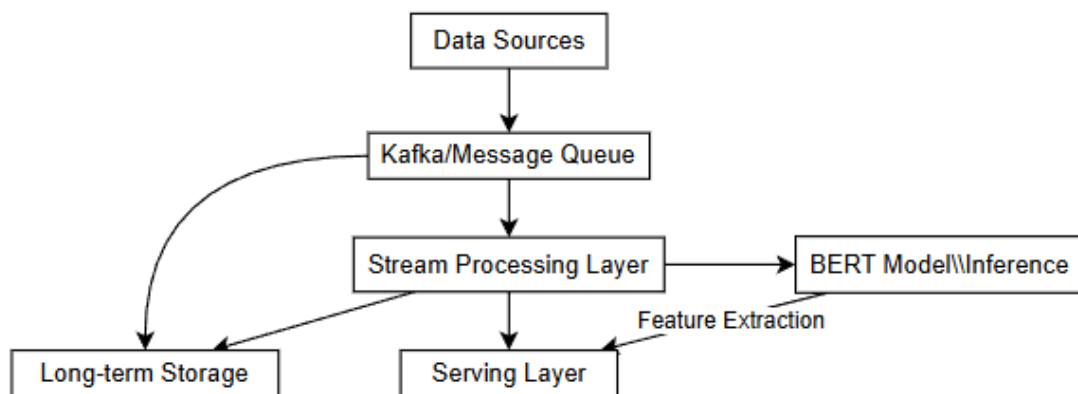
2.5.2 Kiến trúc Kappa cho xử lý dữ liệu streaming

2.5.2.a Tổng quan về Kappa Architecture

Kiến trúc Kappa là một mô hình xử lý dữ liệu hiện đại được đề xuất bởi Jay Kreps (LinkedIn) như một giải pháp đơn giản hóa so với Lambda Architecture truyền thống. Khác với Lambda Architecture có hai đường xử lý riêng biệt (batch và stream), Kappa Architecture chỉ sử dụng một đường xử lý streaming duy nhất, giảm độ phức tạp và chi phí bảo trì.

Các thành phần chính:

1. **Message Queue (Kafka)**: Nhận và lưu trữ tất cả dữ liệu đầu vào dưới dạng event



Hình 1: Kiến trúc Kappa với BERT Model Integration

log bất biến. Kafka đóng vai trò là single source of truth, cho phép replay dữ liệu khi cần thiết.

2. **Stream Processing Layer:** Xử lý dữ liệu theo thời gian thực, thực hiện transformations, aggregations, và feature engineering cho BERT model.
3. **ML Model Inference:** BERT model được deploy để thực hiện prediction trên streaming data.
4. **Serving Layer:** Lưu trữ kết quả prediction và phục vụ các truy vấn từ applications.
5. **Long-term Storage:** Lưu trữ dữ liệu lịch sử cho mục đích retraining model và analysis.

2.5.2.b Apache Kafka làm backbone

Apache Kafka là distributed streaming platform đóng vai trò trung tâm trong Kappa Architecture. Kafka được thiết kế như một commit log phân tán, nơi mọi message được lưu trữ theo thứ tự và có thể được đọc lại nhiều lần.

Các khái niệm cốt lõi:

- **Topics:** Danh mục logic để phân loại messages. Ví dụ: traffic_speed, traffic_volume, traffic_incidents.
- **Partitions:** Mỗi topic được chia thành nhiều partitions để tăng khả năng song song hóa và throughput.
- **Producers:** Các services ghi dữ liệu vào Kafka topics.

- **Consumers:** Các services đọc và xử lý dữ liệu từ topics.
- **Consumer Groups:** Nhóm consumers làm việc cùng nhau để xử lý song song dữ liệu từ một topic.

Ưu điểm của Kafka trong hệ thống giao thông:

- **High Throughput:** Xử lý hàng triệu messages/giây từ các cảm biến giao thông
- **Durability:** Messages được replicate và persist, đảm bảo không mất dữ liệu
- **Scalability:** Dễ dàng scale horizontal bằng cách thêm brokers và partitions
- **Replay Capability:** Có thể đọc lại dữ liệu lịch sử để recompute hoặc fix bugs
- **Low Latency:** Độ trễ chỉ vài milliseconds, phù hợp cho real-time applications

2.5.2.c So sánh Kappa với Lambda Architecture

Bảng 3: So sánh Kappa và Lambda Architecture

Tiêu chí	Lambda Architecture	Kappa Architecture
Batch Layer	Có (MapReduce/Spark)	Không có
Speed Layer	Có (Stream Processing)	Stream processing duy nhất
Độ phức tạp	Cao (2 codebases riêng)	Thấp (1 codebase)
Độ trễ	Cao hơn do batch	Thấp, real-time
Consistency	Eventual consistency	Easier to maintain
Reprocessing	Cần chạy lại batch job	Replay từ Kafka
Maintenance	Khó (sync 2 systems)	Dễ hơn
Use case phù hợp	Cần cả batch và stream	Streaming-first applications

2.5.2.d Ứng dụng Kappa trong hệ thống giao thông

Trong bài toán phân tích giao thông, Kappa Architecture phù hợp vì:

1. **Tính real-time:** Dữ liệu giao thông cần được xử lý ngay lập tức để cảnh báo kịp thời.
2. **Data replay:** Khi cần retrain model hoặc fix bugs, có thể replay dữ liệu từ Kafka mà không cần maintain batch pipeline riêng.
3. **Unified logic:** Cùng một đoạn code xử lý được sử dụng cho cả real-time và historical analysis.

4. **Event-driven:** Giao thông là event-driven system (xe đi qua cảm biến, tắc nghẽn xảy ra, tai nạn), phù hợp với stream processing.

Pipeline xử lý trong hệ thống:

1. Cảm biến giao thông (camera, loop detectors, GPS) publish data vào Kafka topics
2. Consumers đọc từ Kafka, thực hiện:
 - Data cleaning và validation
 - Feature extraction (tốc độ, volume, occupancy)
 - Windowed aggregations (5-min, 15-min averages)
 - Anomaly detection
 - Correlation analysis
3. Kết quả được ghi vào:
 - **Apache Parquet** (định dạng lưu trữ cột cho phân tích dữ liệu quy mô lớn)
4. ML models subscribe processed data để thực hiện predictions
5. Results được publish lại vào Kafka để downstream services consume

2.5.3 TomTom Historical Traffic Stats API

TomTom cung cấp dữ liệu giao thông lịch sử với độ bao phủ cao, bao gồm tốc độ trung bình, mức độ tắc nghẽn và chỉ số độ trễ theo từng đoạn đường. Dữ liệu được tổng hợp từ nhiều nguồn GPS và cập nhật theo khoảng thời gian từ 1–5 phút.

2.5.3.1 Ưu điểm

- Dữ liệu lịch sử dài hạn, hỗ trợ phân tích xu hướng theo giờ, ngày và tuần.
- Độ bao phủ rộng giúp bù đắp cho các khu vực thiếu cảm biến tại địa phương.
- Dễ dàng tích hợp vào các mô hình học chuỗi thời gian và mô hình mạng lưới.

2.5.3.2 Hạn chế

- Không phản ánh nguyên nhân của tắc nghẽn, chỉ mô tả hiện tượng.
- Chi phí sử dụng có thể cao nếu mở rộng cho toàn thành phố.

2.5.3.3 Vai trò trong đề tài Dữ liệu lịch sử từ TomTom giúp:

- Huấn luyện mô hình dự báo giao thông.
- Phân tích lan truyền tắc nghẽn giữa các đoạn đường.
- Xây dựng dữ liệu đầu vào cho mô hình học sâu theo dạng đồ thị.

2.5.4 Cơ sở dữ liệu và lưu trữ

2.5.4.a Lưu trữ dữ liệu dạng cột với Apache Parquet

Trong hệ thống phân tích giao thông đô thị, dữ liệu đầu vào có đặc trưng khối lượng lớn, cấu trúc rõ ràng và chủ yếu phục vụ cho các tác vụ phân tích, thống kê và huấn luyện mô hình học máy. Do đó, hệ thống lựa chọn **Apache Parquet** làm định dạng lưu trữ chính thay vì cơ sở dữ liệu giao dịch truyền thống.

Parquet là định dạng lưu trữ dạng cột (columnar storage), được thiết kế tối ưu cho các hệ thống phân tích dữ liệu lớn (Big Data Analytics) và các workload đọc nhiều, ghi ít.

Các đặc điểm kỹ thuật chính của Parquet:

- **Columnar Storage:** Dữ liệu được lưu theo cột thay vì theo hàng, giúp giảm lượng dữ liệu cần đọc khi chỉ truy vấn một số thuộc tính cụ thể (ví dụ: tốc độ, lưu lượng, thời gian).
- **Efficient Compression:** Parquet hỗ trợ nhiều thuật toán nén (Snappy, ZSTD, GZIP), tận dụng tính đồng nhất của dữ liệu trong cùng một cột để đạt tỷ lệ nén cao, giảm đáng kể dung lượng lưu trữ.
- **Predicate Pushdown:** Cho phép loại bỏ sớm các block dữ liệu không thỏa điều kiện truy vấn, giúp tăng tốc độ xử lý.
- **Schema Evolution:** Hỗ trợ thay đổi schema theo thời gian (thêm/bớt cột) mà không làm hỏng dữ liệu cũ, phù hợp với dữ liệu giao thông có thể mở rộng thuộc tính.
- **Cross-platform Compatibility:** Tương thích tốt với các hệ sinh thái phân tích như Apache Spark, Pandas, DuckDB và các thư viện học máy.

2.5.4.b Ứng dụng Parquet trong hệ thống giao thông

Trong hệ thống đề xuất, Parquet được sử dụng để lưu trữ các tập dữ liệu giao thông đã qua xử lý (processed data), bao gồm:

- Thông tin các đoạn đường (road segments)
- Đặc trưng giao thông theo thời gian (speed, volume, congestion level)
- Dữ liệu đặc trưng đầu vào cho mô hình học máy
- Kết quả dự đoán và thống kê theo từng khung thời gian

Dữ liệu được tổ chức theo cấu trúc thư mục phân tầng dựa trên thời gian (theo ngày, giờ hoặc batch xử lý), giúp thuận tiện cho việc truy xuất, huấn luyện mô hình và phân tích hậu kỳ.

2.5.5 API và Web Development Stack

2.5.5.a FastAPI

FastAPI là một web framework hiện đại dành cho Python, được thiết kế để xây dựng các dịch vụ API với hiệu năng cao, cú pháp rõ ràng và khả năng mở rộng tốt. Framework này đặc biệt phù hợp cho các hệ thống xử lý dữ liệu và mô hình học máy nhờ khả năng tích hợp chặt chẽ với hệ sinh thái Python.

Các đặc điểm kỹ thuật chính của FastAPI:

- **High Performance:** FastAPI được xây dựng trên nền Starlette và Pydantic, cho hiệu năng xử lý cao, đáp ứng tốt các yêu cầu API có tần suất truy cập lớn.
- **Type-based Validation:** Sử dụng Python type hints để tự động kiểm tra, xác thực và chuyển đổi dữ liệu đầu vào, giúp giảm lỗi và tăng độ tin cậy của hệ thống.
- **Asynchronous Processing:** Hỗ trợ native `async/await`, cho phép xử lý đồng thời nhiều request, phù hợp với các pipeline phân tích dữ liệu và inference mô hình.
- **Automatic API Documentation:** Tự động sinh tài liệu OpenAPI (Swagger) và ReDoc, giúp việc kiểm thử và tích hợp API trở nên thuận tiện.
- **Modular Architecture:** Dễ dàng tổ chức code theo các module (router, service, schema), phù hợp cho các hệ thống nghiên cứu và phát triển mở rộng.

Trong hệ thống giao thông được đề xuất, FastAPI đóng vai trò là lớp trung gian cung cấp các API phục vụ:

- Truy xuất dữ liệu giao thông đã được xử lý và lưu trữ dưới dạng Parquet
- Cung cấp đầu vào và đầu ra cho mô hình học máy
- Kết nối giữa pipeline xử lý dữ liệu và giao diện trực quan

2.5.5.b Streamlit

Streamlit là framework Python nhẹ, được thiết kế để xây dựng nhanh các ứng dụng web phục vụ trực quan hóa dữ liệu và trình diễn kết quả phân tích mà không cần phát triển frontend phức tạp.

Ưu điểm của Streamlit trong bối cảnh nghiên cứu và đồ án:

- **Rapid Prototyping:** Cho phép xây dựng giao diện trực quan chỉ với một lượng nhỏ mã Python, rút ngắn thời gian phát triển.
- **Tight Integration with Data Stack:** Tích hợp trực tiếp với Pandas, NumPy, PyArrow và các thư viện học máy, phù hợp cho việc hiển thị dữ liệu Parquet và kết quả mô hình.
- **Interactive Visualization:** Hỗ trợ tương tác người dùng theo thời gian thực (filter, slider, selection) cho các phân tích giao thông.
- **Lightweight Deployment:** Không yêu cầu quản lý state phức tạp hay frontend build system, phù hợp cho môi trường nghiên cứu và demo hệ thống.

Trong hệ thống này, Streamlit được sử dụng để:

- Trực quan hóa dữ liệu giao thông theo không gian và thời gian
- Hiển thị kết quả dự đoán và phân tích của mô hình
- Phục vụ mục đích demo, đánh giá và trình bày hệ thống

Cách tiếp cận kết hợp FastAPI và Streamlit giúp tách biệt rõ ràng giữa lớp xử lý nghiệp vụ và lớp trình diễn, đảm bảo tính đơn giản, dễ bảo trì và phù hợp với mục tiêu nghiên cứu của đề tài.

2.5.6 Containerization và Orchestration

2.5.6.a Docker

Docker là platform containerization cho phép đóng gói applications và dependencies thành containers độc lập, nhất quán trên mọi môi trường từ development đến production.

Lợi ích của Docker:

- **Consistency:** "Works on my machine" problem được giải quyết

- **Isolation:** Mỗi service chạy trong môi trường riêng
- **Portability:** Deploy trên bất kỳ platform nào hỗ trợ Docker
- **Resource Efficiency:** Lightweight hơn VMs, chia sẻ kernel
- **Versioning:** Image versioning cho rollback dễ dàng

Docker Compose quản lý multi-container applications, định nghĩa toàn bộ stack (API, databases, cache, message queue) trong một file YAML, cho phép start/stop toàn bộ hệ thống với một command.

2.5.6.b Kubernetes

Kubernetes (K8s) là container orchestration platform mã nguồn mở, tự động hóa việc deploy, scale, và manage containerized applications trong môi trường production.

Khái niệm cốt lõi:

- **Pods:** Đơn vị nhỏ nhất, chứa một hoặc nhiều containers
- **ReplicaSets:** Đảm bảo số lượng pod replicas mong muốn
- **Deployments:** Quản lý rolling updates và rollbacks
- **Services:** Expose pods qua stable network endpoint
- **Ingress:** HTTP routing và load balancing
- **ConfigMaps và Secrets:** Quản lý configuration và sensitive data
- **Persistent Volumes:** Storage abstraction cho stateful apps
- **Namespaces:** Logical isolation giữa các môi trường

Tính năng quan trọng cho production:

1. **Auto-scaling:** Horizontal Pod Autoscaler (HPA) tự động scale dựa trên CPU, memory, hoặc custom metrics
2. **Self-healing:** Tự động restart failed containers, replace và reschedule pods khi nodes die

3. **Service Discovery và Load Balancing:** Tự động distribute traffic đến healthy pods
4. **Rolling Updates:** Zero-downtime deployments với progressive rollout
5. **Resource Management:** CPU và memory requests/limits để optimize cluster utilization

Với hệ thống giao thông cần high availability và khả năng scale dynamic theo traffic load, Kubernetes cung cấp infrastructure layer mạnh mẽ để đảm bảo reliability và performance.

2.6 Tóm tắt stack công nghệ

Bảng dưới đây tóm tắt các công nghệ chính được sử dụng trong đề tài:

Bảng 4: *Stack công nghệ hệ thống*

Layer	Technology	Purpose
Programming	Python 3.11+	Main development language
ML Framework	PyTorch 2.0+	Deep learning models
Graph Library	PyTorch Geometric	GNN implementation
Message Queue	Apache Kafka	Event streaming backbone
Stream Processing	Python + Kafka	Real-time data processing
Storage	Apache Parquet	Long-term historical data
Model Serving	FastAPI + PyTorch	Inference API

Kiến trúc này được thiết kế để:

- Xử lý dữ liệu streaming real-time hiệu quả
- Scale horizontal khi traffic tăng
- Maintain low latency cho inference
- Support reprocessing và retraining
- Provide observability và monitoring đầy đủ

2.7 Kết chương

Chương 2 đã trình bày một cách toàn diện nền tảng lý thuyết và công nghệ cần thiết để xây dựng hệ thống phân tích mối liên hệ giao thông đô thị. Những kiến thức này tạo thành bộ công cụ quan trọng phục vụ cho các chương tiếp theo của đồ án.

Về mặt lý thuyết, chúng ta đã nắm được:

- Đặc điểm phức tạp của giao thông đô thị TP.HCM với cơ cấu phương tiện đặc thù, tỷ lệ xe máy chiếm ưu thế tuyệt đối ($>90\%$), và hiện tượng lan truyền tắc nghẽn theo mạng lưới đường. Việc hiểu rõ bản chất này giúp thiết kế mô hình phù hợp với đặc thù địa phương.
- Cơ sở toán học vững chắc về biểu diễn mạng giao thông dưới dạng đồ thị $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$, các phép biến đổi trên cấu trúc đồ thị, và cách mô hình hóa tương quan không gian-thời gian.
- Các kiến trúc Graph Neural Networks tiên tiến như GCN và GAT, cho phép học các mối quan hệ phức tạp giữa các nút giao thông mà không cần định nghĩa thủ công. Cơ chế message passing và attention mechanism là nền tảng để mô hình tự động phát hiện các mối tương quan ẩn.
- Các mô hình chuỗi thời gian từ RNN cơ bản đến LSTM và GRU, giúp nắm bắt phụ thuộc temporal với khả năng học long-term dependencies. Việc hiểu rõ ưu nhược điểm của từng loại giúp lựa chọn kiến trúc phù hợp cho bài toán dự báo đa bước (multi-step prediction).

Về mặt công nghệ, chương này đã xác định được bộ khung kỹ thuật (Technology Stack) hiện đại và mạnh mẽ để hiện thực hóa hệ thống:

- **Hệ sinh thái Python & PyTorch**: Cung cấp môi trường phát triển linh hoạt, đặc biệt là thư viện PyTorch Geometric chuyên dụng cho các mô hình GNN, giúp đơn giản hóa việc cài đặt các thuật toán phức tạp trên dữ liệu đồ thị.
- **Kiến trúc Kappa & Apache Kafka**: Đảm bảo khả năng xử lý dữ liệu streaming thời gian thực với độ trễ thấp và độ tin cậy cao, phù hợp với đặc thù dữ liệu liên tục từ các cảm biến giao thông.
- **Lưu trữ dữ liệu không gian – thời gian**: Hệ thống sử dụng định dạng Apache Parquet để lưu trữ dữ liệu giao thông có đặc trưng vừa mang thông tin không gian (tọa độ, đoạn đường), vừa mang tính chuỗi thời gian. Cách tiếp cận này cho phép

tổ chức dữ liệu theo các phân vùng thời gian và thuộc tính không gian, đồng thời tối ưu hiệu năng cho các tác vụ phân tích, thống kê và huấn luyện mô hình học máy.

- **Hạ tầng triển khai:** Việc sử dụng Docker và Kubernetes đảm bảo hệ thống có khả năng mở rộng (scalability), dễ dàng triển khai và vận hành ổn định trong môi trường thực tế.

Sự kết hợp chặt chẽ giữa các nền tảng lý thuyết vững chắc và các công nghệ tiên tiến này là tiền đề quan trọng để bước sang Chương 3 – nơi chúng ta sẽ khảo sát các công trình liên quan để định vị rõ hơn đóng góp của đề tài, và Chương 4 – nơi mô hình hệ thống cụ thể sẽ được đề xuất.

3 Công trình liên quan

Chương này tổng hợp và phân tích các công trình nghiên cứu tiêu biểu cùng các hệ thống thực tế liên quan đến bài toán phân tích và dự báo giao thông đô thị. Mục tiêu của chương không chỉ dừng lại ở việc liệt kê các hướng tiếp cận đã có, mà còn làm rõ tiến trình phát triển tư duy mô hình hóa giao thông, những giới hạn tồn tại, và cơ sở khoa học cho hướng tiếp cận mà đề tài lựa chọn.

Nội dung được tổ chức theo ba trục chính: (i) các mô hình truyền thống và thống kê mô tả cơ chế lan truyền tắc nghẽn; (ii) các mô hình học sâu trên đồ thị, đặc biệt là sự chuyển dịch từ đồ thị tĩnh sang đồ thị động và cơ chế học quan hệ; và (iii) các hệ thống giao thông thực tế, từ đó xác định khoảng trống nghiên cứu mà đề tài hướng đến giải quyết.

3.1 Các nghiên cứu liên quan

3.1.1 Mô hình hóa cơ chế lan truyền tắc nghẽn giao thông

Trước khi học sâu trở thành xu hướng chủ đạo, nhiều nghiên cứu trong lĩnh vực giao thông tập trung vào việc mô hình hóa bản chất vật lý và thống kê của dòng chảy giao thông. Các công trình này đặt nền móng quan trọng cho việc hiểu tắc nghẽn không chỉ là hiện tượng cục bộ, mà là một quá trình lan truyền có cấu trúc trong không gian và thời gian.

3.1.1.a Mô hình lan truyền dạng sóng

Jiang và cộng sự, thông qua phân tích dữ liệu cảm biến giao thông tại các đô thị lớn như Bắc Kinh và Thâm Quyển, đã xác định sự tồn tại của các *jam centers* – những tâm điểm khởi phát tắc nghẽn. Nghiên cứu chỉ ra rằng tắc nghẽn lan truyền theo dạng sóng với vận tốc tương đối ổn định (khoảng 30 ± 10 km/h) và cường độ suy giảm theo khoảng cách.

Đóng góp cốt lõi của hướng tiếp cận này là cung cấp bằng chứng thực nghiệm cho tính *không gian* của tắc nghẽn, tuy nhiên các mô hình dạng sóng thường mang tính mô tả vĩ mô và khó mở rộng để dự báo chi tiết trên mạng lưới giao thông phức tạp.

3.1.1.b Mô hình lây lan và quá trình lan truyền

Saberi và cộng sự tiếp cận bài toán theo hướng xem tắc nghẽn như một quá trình lây lan, tương tự dịch bệnh, sử dụng các mô hình SIR để mô tả trạng thái của từng đoạn đường theo thời gian. Cách tiếp cận này cho phép phân tích động lực lan truyền trên toàn mạng lưới và làm rõ vai trò của tương tác liên vùng.

Tuy nhiên, các mô hình lây lan cổ điển thường giả định quan hệ giữa các nút là cố định hoặc đơn giản, chưa đủ linh hoạt để phản ánh sự biến đổi nhanh và phi tuyến của giao thông đô thị hiện đại.

3.1.1.c Xác định tuyến và nút ảnh hưởng

Một hướng nghiên cứu quan trọng khác tập trung vào việc xác định các tuyến đường hoặc nút giao thông có mức độ ảnh hưởng lớn đến toàn mạng lưới. Công trình về *Most Influential Routes* đề xuất khái niệm “độ ảnh hưởng” của một tuyến, không chỉ dựa trên mức độ tắc nghẽn tại chính nó, mà còn dựa trên khả năng gây ảnh hưởng lan truyền sang các tuyến khác.

Tư tưởng cốt lõi của hướng tiếp cận này là chuyển trọng tâm từ dự báo giá trị (ví dụ: tốc độ, lưu lượng) sang phân tích *tác động và lan truyền*, đây là tiền đề quan trọng cho các mô hình đồ thị học quan hệ trong đề tài.

3.2 Các mô hình học sâu trên đồ thị giao thông

3.2.1 Đồ thị tĩnh và các mô hình GCN truyền thống

Sự xuất hiện của Graph Convolutional Networks (GCN) đánh dấu bước tiến lớn trong việc khai thác cấu trúc topo của mạng lưới giao thông. Các mô hình như T-GCN, ST-GCN hay DCRNN kết hợp GCN với các mô hình chuỗi thời gian (GRU, LSTM) để học đồng thời thông tin không gian và thời gian.

Các mô hình này cho thấy hiệu quả vượt trội so với các phương pháp thống kê truyền thống. Tuy nhiên, điểm hạn chế mang tính cấu trúc là ma trận kề thường được xây dựng tĩnh, dựa trên khoảng cách địa lý hoặc kết nối vật lý, và không thay đổi theo thời gian. Giả định này không còn phù hợp trong bối cảnh giao thông đô thị phức tạp, nơi mối quan hệ giữa các nút có thể biến đổi mạnh theo giờ cao điểm, sự cố, hoặc hành vi người tham gia giao thông.

3.2.2 Học quan hệ động và Attention trên đồ thị

Nhằm khắc phục hạn chế của đồ thị tĩnh, nhiều nghiên cứu gần đây tập trung vào học ma trận kề động dựa trên dữ liệu quan sát. Các mô hình như DTC-STGCN cập nhật quan hệ giữa các nút theo thời gian thông qua tương quan hoặc thống kê động, cho thấy cải thiện đáng kể về độ chính xác dự báo.

Song song đó, cơ chế Attention và Graph Attention Networks (GAT) cho phép mô hình tự động học trọng số ảnh hưởng giữa các nút mà không cần định nghĩa ma trận kề cố định. Điều này đặc biệt phù hợp với các đô thị có cấu trúc giao thông phức tạp, nơi tồn

tại nhiều *kết nối ngữ nghĩa* không thể hiện trực tiếp qua topo vật lý.

3.2.3 Transformer cho biểu diễn đồ thị

Công trình “Do Transformers Really Perform Badly for Graph Representation?” (NeurIPS 2021) đặt lại câu hỏi về vai trò của Transformer trong học biểu diễn đồ thị. Nghiên cứu chỉ ra rằng, khi được thiết kế phù hợp với inductive bias của đồ thị, Transformer không những không kém hơn GCN mà còn có khả năng học quan hệ dài hạn và linh hoạt hơn.

Kết luận này có ý nghĩa quan trọng đối với bài toán giao thông: lan truyền tắc nghẽn không chỉ phụ thuộc vào các nút lân cận trực tiếp, mà còn chịu ảnh hưởng của các tuyến xa hơn thông qua các luồng di chuyển chủ đạo. Điều này mở ra hướng tiếp cận kết hợp tư tưởng Attention/Transformer với đồ thị giao thông động, thay vì chỉ dựa vào convolution cục bộ.

3.3 Các hệ thống và nền tảng giao thông thực tế

Các hệ thống như Google Maps, Waze, TomTom hay HERE đã chứng minh khả năng dự báo giao thông thời gian thực ở quy mô lớn. Tuy nhiên, phần lớn các hệ thống này hoạt động như “hộp đen”, tập trung vào cung cấp kết quả điều hướng hoặc ETA, thay vì hỗ trợ phân tích cơ chế lan truyền và mức độ ảnh hưởng giữa các nút giao thông.

Đối với các nền tảng giám sát nội địa như UTraffic, hệ thống hiện tại chủ yếu dừng ở mức thu thập và hiển thị dữ liệu, chưa khai thác sâu tiềm năng của các mô hình học sâu để dự báo dài hạn và giải thích hiện tượng tắc nghẽn.

3.4 Khoảng trống nghiên cứu

Từ tổng quan trên, có thể xác định ba khoảng trống nghiên cứu chính:

1. **Giới hạn của đồ thị tĩnh:** Phần lớn các mô hình hiện tại chưa phản ánh được tính động và phi tuyến của quan hệ giao thông.
2. **Thiếu phân tích lan truyền và ảnh hưởng:** Nhiều mô hình tập trung vào dự báo giá trị cục bộ, nhưng chưa làm rõ vai trò ảnh hưởng liên vùng của các tuyến và nút giao thông.
3. **Khoảng cách giữa dự báo và giải thích:** Các hệ thống thực tế mạnh về dự báo nhưng yếu về khả năng giải thích và hỗ trợ ra quyết định chiến lược.

Tóm tắt

Chương này đã đặt nền tảng lý thuyết và thực tiễn cho đề tài bằng cách phân tích tiến trình phát triển từ các mô hình lan truyền truyền thống đến các mô hình học sâu trên đồ thị và Attention. Dựa trên các khoảng trống được xác định, đề tài lựa chọn hướng tiếp cận kết hợp tư tưởng **lan truyền ảnh hưởng trên mạng giao thông** và **học quan hệ động bằng cơ chế Attention/Transformer**, nhằm nâng cao đồng thời khả năng dự báo và giải thích tắc nghẽn giao thông đô thị.

4 Phương pháp đề xuất

4.1 Tổng quan phương pháp và động cơ lựa chọn mô hình

4.1.1 Định hướng giải pháp

Trong nghiên cứu này, nhóm đề xuất một phương pháp dự báo ùn tắc giao thông dựa trên **mô hình Transformer encoder-only theo phong cách BERT + Spatial Encoding(Laplacian) + Temporal Encoding**, được thiết kế để học biểu diễn không gian-thời gian trên các *vùng giao thông động* (dynamic traffic zones). Mỗi vùng giao thông được biểu diễn như một đồ thị con (subgraph) bao gồm các tuyến đường có mối liên hệ về không gian, cấu trúc mạng lưới và hành vi giao thông, được khởi tạo từ một sự kiện ùn tắc tại thời điểm nhất định.

Việc lựa chọn kiến trúc Transformer encoder-only + Spatial Encoding(Laplacian) + Temporal Encoding được thúc đẩy bởi những hạn chế cố hữu của các phương pháp học biểu diễn đồ thị dựa trên cơ chế truyền thông tin cục bộ (message passing), chẳng hạn như Graph Convolutional Networks (GCN) hay các biến thể GNN phổ biến khác. Các mô hình này cập nhật biểu diễn nút thông qua việc tổng hợp thông tin từ lân cận k-hop, do đó gặp khó khăn trong việc nắm bắt cấu trúc toàn cục của đồ thị và phân biệt các nút có vai trò cấu trúc khác nhau nhưng có lân cận cục bộ tương tự. Hơn nữa, khi tăng số lớp, các mô hình GNN thường gặp hiện tượng *over-smoothing*, khiến biểu diễn của các nút trở nên khó phân biệt. Ngoài ra, trong bối cảnh dữ liệu chuỗi thời gian dài hạn (long-term temporal dependency) như bài toán giao thông đô thị, các mô hình học tuần tự truyền thống dựa trên Recurrent Neural Networks (RNN) hoặc các biến thể như LSTM/GRU cũng bộc lộ những hạn chế đáng kể. Mặc dù được thiết kế nhằm giảm thiểu hiện tượng suy biến gradient, các mô hình này vẫn gặp khó khăn trong việc học các phụ thuộc thời gian dài khi độ sâu theo thời gian tăng lên, dẫn đến hiện tượng vanishing gradient, làm suy giảm khả năng lan truyền thông tin từ các thời điểm xa trong quá khứ đến đầu ra dự đoán. Tương tự, các phương pháp dựa trên Temporal Convolution với receptive field hữu hạn thường chỉ nắm bắt được các mẫu cục bộ theo trục thời gian, và việc mở rộng phạm vi phụ thuộc đòi hỏi phải tăng độ sâu mạng, từ đó làm gia tăng độ phức tạp và nguy cơ mất ổn định trong huấn luyện.

Những hạn chế trên đã được phân tích và làm rõ trong công trình của Ying et al. [ying2021graphormer], trong đó tác giả chỉ ra rằng Transformer có thể đạt hiệu quả cao trong học biểu diễn đồ thị nếu được bổ sung các cơ chế mã hóa cấu trúc phù hợp. Cụ thể, Graphormer cho thấy rằng việc đưa thông tin cấu trúc của đồ thị trực tiếp vào cơ chế self-attention cho phép mô hình vượt qua giới hạn biểu diễn của các kiến trúc GNN truyền thống, đồng thời cung cấp trường nhìn toàn cục (global receptive field) cho

mỗi nút trong đồ thị.

4.1.2 Quy trình xử lý chính

Pipeline xử lý dữ liệu và phân tích được thực hiện theo trình tự sau:

1. **Thu thập dữ liệu:** Lấy dữ liệu lịch sử từ TomTom API cho các nút giao thông được chọn
2. **Tiền xử lý:** Làm sạch, chuẩn hóa, và xử lý missing values
3. **Trích xuất đặc trưng:** Tính toán các đặc trưng giao thông và thời gian
4. **Phân tích tương quan:** Xác định mối tương quan không gian-thời gian
5. **Kiểm định nhân quả:** Áp dụng Granger Causality Test
6. **Xây dựng đồ thị động:** Tạo ma trận kề thay đổi theo thời gian
7. **Huấn luyện mô hình:** Phát triển mô hình dự báo dựa trên GNN
8. **Đánh giá và triển khai:** Kiểm thử và tích hợp vào hệ thống

4.2 Thu thập và chuẩn bị dữ liệu

4.2.1 Nguồn dữ liệu: TomTom Historical Traffic Stats

4.2.1.a Lý do lựa chọn TomTom API

Dữ liệu giao thông lịch sử từ TomTom được lựa chọn làm nguồn dữ liệu chính cho đề tài vì các lý do sau:

- **Độ phủ toàn diện:** TomTom cung cấp dữ liệu cho hầu hết các tuyến đường chính tại TP.HCM, bao gồm cả các khu vực trung tâm và vùng ven.
- **Chất lượng dữ liệu:** Dữ liệu được tổng hợp từ nhiều nguồn (GPS probe data, cảm biến, crowdsourcing) và đã qua xử lý, lọc nhiễu bởi TomTom.
- **Dữ liệu lịch sử dài hạn:** API cung cấp dữ liệu lịch sử từ nhiều tháng đến nhiều năm, đủ để huấn luyện mô hình học sâu.
- **Độ phân giải thời gian phù hợp:** Dữ liệu có độ phân giải 5 phút, cân bằng giữa chi tiết và khả năng xử lý.
- **Các chỉ số phong phú:** Bao gồm tốc độ thực tế, tốc độ tự do, thời gian di chuyển, độ tin cậy, giúp phân tích đa chiều.

4.2.1.b Cấu trúc dữ liệu TomTom

TomTom Historical Traffic Stats API cung cấp dữ liệu theo cấu trúc phân cấp như sau:

Thông tin Segment (Đoạn đường):

- **segmentId, newSegmentId:** Định danh duy nhất của đoạn đường
- **streetName:** Tên đường
- **distance:** Chiều dài đoạn đường (mét)
- **speedLimit:** Giới hạn tốc độ (km/h)
- **frc (Functional Road Class):** Phân loại chức năng đường (0-7, với 0 là cao tốc)
- **shape:** Tọa độ địa lý các điểm trên đoạn đường (latitude, longitude)

Kết quả theo thời gian (segmentTimeResults):

- `timeSet`: Khung thời gian trong ngày (độ phân giải 5 phút)
- `dateRange`: Khoảng ngày thu thập dữ liệu

Chỉ số tốc độ:

- `harmonicAverageSpeed`: Tốc độ trung bình điều hòa (km/h)
- `medianSpeed`: Tốc độ trung vị (km/h)
- `averageSpeed`: Tốc độ trung bình (km/h)
- `standardDeviationSpeed`: Độ lệch chuẩn tốc độ (km/h)

Chỉ số thời gian di chuyển:

- `averageTravelTime`: Thời gian di chuyển trung bình (giây)
- `medianTravelTime`: Thời gian di chuyển trung vị (giây)
- `travelTimeStandardDeviation`: Độ lệch chuẩn thời gian (giây)
- `travelTimeRatio`: Tỷ lệ thời gian thực tế/lý tưởng

Độ tin cậy:

- `sampleSize`: Số lượng mẫu dữ liệu thu thập được

4.2.2 Lựa chọn khu vực nghiên cứu

4.2.2.a Tiêu chí lựa chọn

Việc lựa chọn khu vực giao thông để nghiên cứu dựa trên các tiêu chí sau:

1. Tầm quan trọng chiến lược:

- Các tuyến huyết mạch nối các quận trung tâm
- Cổng vào/ra khu vực trung tâm TP.HCM
- Các trục đường có lưu lượng cao và thường xuyên ùn tắc

2. Độ phủ dữ liệu:

- Đảm bảo có dữ liệu TomTom liên tục cho khu vực

- Tỷ lệ missing data dưới 10%
- Sample size trung bình đủ lớn để đảm bảo độ tin cậy

3. Tính đại diện:

- Bao gồm nhiều loại đường: đại lộ, đường chính, đường nhánh
- Đa dạng về mật độ giao thông và đặc điểm vận hành
- Phân bố đều về không gian

4. Khả năng kết nối:

- Các đoạn đường có mối liên hệ trực tiếp hoặc gián tiếp
- Nằm trong cùng một mạng lưới liên thông

4.2.2.b Khu vực được chọn

Dựa trên các tiêu chí trên, đề tài lựa chọn khu vực trung tâm TP.HCM được giới hạn bởi tọa độ:

- Vĩ độ: 10.7535894° - 10.7969493° Bắc
- Kinh độ: 106.6816889° - 106.7151954° Đông

Khu vực này là khu vực trung tâm như Quận 1, với mật độ giao thông cao và hệ thống đường sá phức tạp.

4.2.3 Quy trình thu thập dữ liệu

4.2.3.a Kiến trúc Data Pipeline

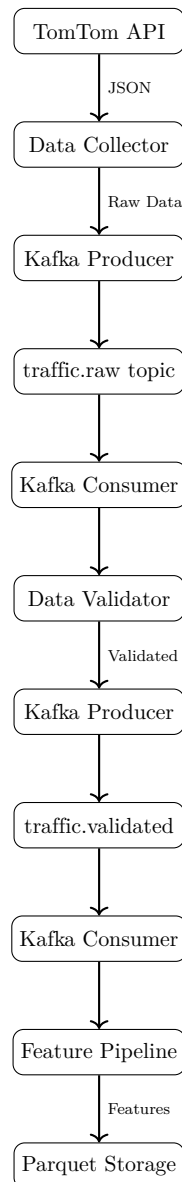
Pipeline thu thập dữ liệu được thiết kế theo kiến trúc Kappa Architecture, kết hợp batch processing và stream processing để đảm bảo tính ổn định, hiệu quả và khả năng mở rộng. Kiến trúc pipeline được mô tả trong Hình 2.

4.2.3.b Các bước thu thập

Bước 1: Tạo Analysis Job

Sử dụng TomTomTrafficDataCollector để tạo job phân tích khu vực:

Bước 2: Monitoring và Download



Hình 2: Kiến trúc Data Pipeline thu thập và xử lý dữ liệu

- Poll job status định kỳ để theo dõi tiến độ xử lý
- Download kết quả khi job hoàn thành (định dạng JSON hoặc GeoJSON)
- Implement exponential backoff khi gặp rate limit
- Xử lý lỗi và retry logic

Bước 3: Streaming Ingestion

Đưa dữ liệu vào Kafka stream để xử lý theo thời gian thực

Bước 4: Validation Processing

Consume từ `traffic.raw`, validate và gửi vào `traffic.validated`:

- Kiểm tra tính hợp lệ của segment data
- Validate các time results (speed, travel time trong giới hạn hợp lý)
- Lọc bỏ dữ liệu không đạt tiêu chuẩn
- Commit offset chỉ khi xử lý thành công

Bước 5: Feature Extraction và Storage

Consume từ `traffic.validated`, trích xuất features và lưu trữ:

- Convert nested JSON structure thành flat records
- Batch processing (10,000 records/batch) để tối ưu hiệu suất
- Pipeline xử lý: Clean → Extract → Encode → Spatial → Normalize
- Lưu trữ vào Parquet với partitioning theo date/time

4.2.3.c Error Handling và Retry Strategy

API Request Handling:

- Exponential backoff: $\text{delay} = \text{base_delay} \times 2^{\text{attempt}}$
- Maximum 3 retry attempts cho mỗi request
- Timeout: 30s cho mỗi request
- Rate limit: 100 requests/hour

Stream Processing Handling:

- Log tất cả errors với context đầy đủ (topic, partition, offset)
- Không commit offset khi có lỗi xử lý
- Alert khi error rate vượt 5%
- Dead Letter Queue cho messages không xử lý được

4.2.3.d Tối ưu hóa hiệu suất

Batch Processing:

- Aggregate multiple records trước khi lưu Parquet
- Sử dụng compression (snappy) để giảm storage
- Partitioning theo date và time_set để query hiệu quả

Parallel Processing:

- Kafka consumer groups cho horizontal scaling
- Multiple partitions trên Kafka topics
- Async I/O cho API requests

Caching Strategy:

- Cache API responses để tránh duplicate requests
- In-memory cache cho lookup tables (FRC mapping, etc.)

4.2.4 Tiền xử lý dữ liệu

4.2.4.a Xử lý missing values

Dữ liệu giao thông thực tế thường có missing values do nhiều nguyên nhân: lỗi cảm biến, mất kết nối, hoặc không đủ probe data. Chiến lược xử lý được implement trong DataCleaner:

1. Phân loại missing patterns:

- *MCAR (Missing Completely At Random)*: Xảy ra ngẫu nhiên
- *MAR (Missing At Random)*: Phụ thuộc vào biến quan sát được
- *MNAR (Missing Not At Random)*: Phụ thuộc vào giá trị bị missing

2. Phương pháp imputation:

Linear Interpolation - Cho gaps nhỏ (< 3 time steps):

$$v_t = v_{t-1} + \frac{v_{t+k} - v_{t-1}}{k + 1}$$

Segment-based median imputation - Sử dụng median theo từng segment

Global median - Fallback cho missing values còn lại

3. Quality threshold:

- Loại bỏ segment có $> 20\%$ missing data
- Đánh dấu imputed values để đánh giá độ tin cậy

4.2.4.b Outlier detection và xử lý

Outliers trong dữ liệu giao thông có thể do tai nạn, sự kiện đặc biệt, hoặc lỗi đo. DataCleaner sử dụng kết hợp nhiều phương pháp:

1. Statistical Methods:

Z-score method:

$$z_i = \frac{x_i - \mu}{\sigma}, \quad |z_i| > 3 \Rightarrow \text{outlier}$$

IQR (Interquartile Range):

$$\text{outlier if } x < Q_1 - 1.5 \cdot \text{IQR} \text{ or } x > Q_3 + 1.5 \cdot \text{IQR}$$

2. Combined Detection:

Chỉ đánh dấu outlier khi cả Z-score VÀ IQR đều phát hiện (giảm false positive)

3. Handling Strategy:

- **Capping:** Giới hạn giá trị trong khoảng [lower_bound, upper_bound]
- Giữ lại outliers có thể là sự kiện thực (không remove)
- Log tất cả outliers để phân tích sau

4.2.4.c Data validation

DataValidator thực hiện kiểm tra tính hợp lệ trên nhiều cấp độ:

1. Structure Validation:

- Kiểm tra required fields (segmentId, distance, shape, segmentTimeResults)

- Validate data types
- Kiểm tra nested structure

2. Value Range Validation:

- Speed: min_speed (0 km/h) đến max_speed (120 km/h)
- Distance: > 0 và $< 10,000$ mét
- Travel time: ≥ 0
- Sample size: ≥ 0
- Coordinates: latitude $\in [-90, 90]$, longitude $\in [-180, 180]$

3. Data Quality Checks:

- Missing data ratio $< 20\%$ per column
- Non-null values cho critical columns (segment_id, average_speed)
- Consistency checks giữa các metrics liên quan

4. Invalid Row Removal:

- Remove rows với speed = 0 (likely errors)
- Remove rows với sample_size < 10 (unreliable)

4.2.4.d Conversion và chuẩn hóa đơn vị

Distance Conversion:

```
1 # Convert from meters to kilometers
2 if df['distance'].max() > 10: # Likely in meters
3     df['distance'] = df['distance'] / 1000.0
```

Listing 1: Chuyển đổi đơn vị khoảng cách

Derived Features:

- free_flow_speed: 95th percentile của average_speed theo segment

$$v_{\text{free_flow}} = \text{percentile}_{95}(v_{\text{avg}})$$

- `free_flow_travel_time`: $t_{\text{free_flow}} = \frac{d}{v_{\text{free_flow}}}$
- Các chỉ số này được tính tự động nếu không có trong raw data

4.2.4.e Lưu trữ trung gian

Sau mỗi giai đoạn xử lý, dữ liệu được lưu vào Parquet với partitioning

Bảng 5: Cấu trúc lưu trữ dữ liệu Parquet

Loại dữ liệu	Đường dẫn partition
Raw Data	<code>parquet_dir/raw_traffic_data/date_range=YYYYMMDD/</code>
Validated Data	<code>parquet_dir/validated_traffic_data/date_range=.../time_set=.../</code>
Features	<code>parquet_dir/traffic_features/date=YYYY-MM-DD/</code>

Việc partitioning mang lại các lợi ích:

- Query hiệu quả theo time range
- Incremental processing
- Easy data management và cleanup
- Tối ưu storage với compression

4.2.5 Pipeline tích hợp

`TrafficDataPipeline` là class chính điều phối toàn bộ quy trình:

```
1 pipeline = TrafficDataPipeline()  
2  
3 # Run full pipeline  
4 pipeline.run_full_pipeline(  
5     geometry=hcmc_area ,  
6     date_from="2024-08-01" ,  
7     date_to="2024-08-31" ,  
8     job_name="HCMC Traffic Analysis"  
9 )
```

Listing 2: Pipeline tích hợp

Pipeline thực hiện tuần tự các stages:

1. **Stage 1:** Batch Collection từ TomTom API
2. **Stage 2:** Streaming Ingestion vào Kafka

3. **Stage 3:** Validation Processing

4. **Stage 4:** Feature Extraction và Storage

Mỗi stage có thể chạy độc lập hoặc kết hợp, cho phép flexibility trong việc xử lý và debug.

4.3 Mỗi tương quan giữa các tuyến trong giao thông đô thị

Dựa trên những phân tích này, nhóm xây dựng mô hình Transformer encoder-only để học biểu diễn không gian-thời gian cho từng vùng giao thông. Tuy nhiên, khác với các bài toán đồ thị tĩnh, bài toán giao thông đô thị đòi hỏi mô hình phải nắm bắt được **tương tác động và có độ trễ theo thời gian giữa các tuyến đường**, phản ánh cơ chế lan truyền của ùn tắc trong mạng lưới giao thông.

Để giải quyết yêu cầu này, nhóm sẽ khai thác các mối tương quan chéo có xét đến độ trễ thời gian giữa các tuyến đường, dựa trên khuôn khổ mạng tương quan giao thông được đề xuất bởi Guo et al. [guo2019identifying]. Cách tiếp cận này cho phép định lượng mức độ ảnh hưởng giữa các tuyến đường và xác định các tuyến có vai trò quan trọng trong quá trình lan truyền ùn tắc.

Cụ thể, với mỗi cặp tuyến đường i và j , từ chuỗi thời gian vận tốc đã được chuẩn hóa $S_i(t)$ và $S_j(t)$, hàm tương quan chéo có độ trễ được định nghĩa như sau:

$$X_{i,j}(\tau) = \frac{\sum_{t=1}^{L-\tau} (S_i(t) - \bar{S}_i) (S_j(t+\tau) - \bar{S}_j)}{\sqrt{\sum_{t=1}^{L-\tau} (S_i(t) - \bar{S}_i)^2 \sum_{t=1}^{L-\tau} (S_j(t+\tau) - \bar{S}_j)^2}}, \quad (22)$$

trong đó $\tau \in [-\tau_{\max}, \tau_{\max}]$ là độ trễ thời gian, L là độ dài chuỗi thời gian, và $\bar{S}_i, \bar{S}_j$ lần lượt là giá trị trung bình của các chuỗi tương ứng. Với $\tau < 0$, ta có $X_{i,j}(\tau) = X_{j,i}(-\tau)$.

Từ hàm tương quan chéo này, Guo et al. [guo2019identifying] đề xuất trọng số liên kết dương (positive link weight) giữa hai tuyến đường i và j được xác định bởi:

$$W_{i,j}^{\text{pos}} = \frac{\max_{\tau} X_{i,j}(\tau) - \text{mean}_{\tau} (X_{i,j}(\tau))}{\text{std}_{\tau} (X_{i,j}(\tau))}, \quad (23)$$

trong đó $\max_{\tau}$, mean_{τ} và std_{τ} lần lượt là giá trị lớn nhất, trung bình và độ lệch chuẩn của hàm tương quan chéo trên miền τ . Đồng thời, độ trễ tương ứng với ảnh hưởng mạnh nhất được xác định là:

$$\tau_{i,j}^{\text{pos}} = \arg \max_{\tau} X_{i,j}(\tau). \quad (24)$$

Trọng số $W_{i,j}^{\text{pos}}$ phản ánh mức độ nổi trội của đỉnh tương quan so với nền nhiễu, giúp lọc bỏ các tương quan yếu hoặc ngẫu nhiên, trong khi $\tau_{i,j}^{\text{pos}}$ cung cấp thông tin về hướng và độ trễ của ảnh hưởng giữa các tuyến đường. Việc sử dụng các đại lượng này cho phép nhóm xác định các liên kết giao thông mang ý nghĩa chức năng, phù hợp để mô hình hóa quá trình lan truyền ùn tắc trong không gian và thời gian.

Trên cơ sở các liên kết tương quan có ý nghĩa thống kê và ràng buộc không gian, nhóm

xây dựng các vùng giao thông chức năng và xác định tập các tuyến mục tiêu cần dự đoán. Các vùng này sau đó được đưa vào mô hình Transformer encoder-only với các cơ chế mã hóa không gian–thời gian, cho phép học biểu diễn giàu ngữ cảnh và nâng cao khả năng dự báo ùn tắc giao thông trong môi trường đô thị phức tạp.

4.4 Xây dựng vùng giao thông động

Như đã trình bày ở Mục 4.1, vùng giao thông trong nghiên cứu này không được xem là một khu vực không gian cố định, mà là một *vùng chức năng động* (functional dynamic zone) được khởi tạo từ các sự kiện ùn tắc và mở rộng dựa trên mối liên hệ không gian–thời gian giữa các tuyến đường. Quá trình xây dựng vùng giao thông động bao gồm hai giai đoạn chính: (i) phát hiện cụm ùn tắc ban đầu tại từng thời điểm, và (ii) mở rộng vùng dựa trên cấu trúc mạng lưới và các tiêu chí không gian.

4.4.1 Phát hiện cụm ùn tắc theo snapshot

Tại mỗi thời điểm t , nhóm xác định tập các tuyến đường đang ở trạng thái ùn tắc dựa trên nhãn nhị phân $y_i(t) \in \{0, 1\}$. Đối với mỗi tuyến đường, tọa độ không gian được biểu diễn bằng điểm trung tâm của tuyến, được tính từ trung điểm giữa hai nút đầu và cuối của tuyến trong đồ thị giao thông.

Trên tập các tuyến đang ùn tắc tại thời điểm t , nhóm áp dụng thuật toán phân cụm DBSCAN để phát hiện các *cụm ùn tắc theo snapshot*. DBSCAN được lựa chọn do không yêu cầu số cụm cố định trước và có khả năng phát hiện các cụm với hình dạng bất kỳ, phù hợp với đặc điểm phân bố không gian của ùn tắc giao thông. Mỗi cụm thu được từ DBSCAN tại thời điểm t được xem như một *seed congestion cluster*, đóng vai trò khởi tạo cho quá trình xây dựng vùng giao thông động.

4.4.2 Chiến lược chọn seed vùng

Để đảm bảo tính cân bằng giữa các trạng thái giao thông khác nhau trong quá trình huấn luyện, nhóm áp dụng chiến lược chọn seed vùng kết hợp giữa các cụm ùn tắc và các vùng không ùn tắc. Cụ thể, trong quá trình sinh mẫu huấn luyện, 60% seed được lấy từ các cụm ùn tắc được phát hiện bởi DBSCAN tại thời điểm t , trong khi 40% seed còn lại được chọn từ các vùng không ùn tắc.

Các seed không ùn tắc có thể được tạo bằng cách áp dụng DBSCAN trên tập các tuyến không ùn tắc. Sau khi đã chọn ra các cụm kẹt hoặc không kẹt từ DBSCAN, mô hình sẽ lựa chọn ngẫu nhiên một vùng cụ thể với điều kiện tương ứng tại thời điểm t . Chiến lược này giúp mô hình học được cả các mẫu lan truyền ùn tắc lẫn các trạng thái giao thông ổn định, đồng thời giảm thiểu hiện tượng lệch lớp trong dữ liệu.

4.4.3 Mở rộng vùng theo cấu trúc mạng lưới

Từ mỗi seed congestion cluster, nhóm tiến hành mở rộng vùng giao thông để thu thập tập các tuyến ứng viên có khả năng liên quan đến động lực giao thông của vùng. Trước hết, tất cả các tuyến thuộc seed cluster được đưa vào tập vùng ban đầu. Sau đó, dựa trên cấu trúc đồ thị giao thông toàn cục, vùng được mở rộng bằng cách bổ sung các tuyến lân cận theo quan hệ topology, với số bước mở rộng (hop) được nhóm chọn là 1.

Quá trình mở rộng này nhằm đảm bảo rằng vùng giao thông không chỉ bao gồm các tuyến đang ùn tắc tại thời điểm hiện tại, mà còn bao phủ các tuyến có khả năng đóng vai trò trung gian hoặc chịu ảnh hưởng từ quá trình lan truyền ùn tắc trong tương lai.

4.4.4 Lọc vùng theo bán kính không gian

Để giới hạn phạm vi không gian của vùng giao thông và đảm bảo tính hợp lý về mặt vật lý, nhóm áp dụng một bước lọc cứng dựa trên bán kính không gian. Cụ thể, đối với mỗi seed cluster, một điểm trung tâm của vùng được xác định từ trung tâm hình học của các tuyến trong seed.

Với mỗi tuyến ứng viên thu được từ bước mở rộng theo topology, khoảng cách Haversine từ tuyến đó đến tâm seed được tính toán. Chỉ những tuyến có khoảng cách nhỏ hơn hoặc bằng ngưỡng $R_{\max}$ mới được giữ lại trong vùng giao thông cuối cùng. Ở mô hình này nhóm chọn bán kính là 1500m cho mỗi vùng. Bước lọc này giúp loại bỏ các tuyến ở quá xa seed cluster, vốn khó có mối liên hệ trực tiếp với sự kiện ùn tắc ban đầu.

4.4.5 Vùng giao thông chức năng

Sau các bước trên, vùng giao thông thu được không còn là một cụm không gian thuần túy, mà là một *vùng giao thông chức năng*, bao gồm các tuyến đường có mối liên hệ về không gian, cấu trúc mạng lưới và tiềm năng ảnh hưởng giao thông. Đáng chú ý, các tuyến nằm trong vùng giao thông này không nhất thiết phải thuộc seed congestion cluster ban đầu, mà có thể nằm ngoài cụm nhưng vẫn có vai trò quan trọng trong động lực lan truyền ùn tắc.

Vùng giao thông chức năng này đóng vai trò là đơn vị đầu vào cho các bước tiếp theo, trong đó các mối tương quan giao thông có độ trễ sẽ được khai thác để xác định các liên kết chức năng và tập các tuyến mục tiêu cần dự đoán.

4.5 Khai thác tương quan giao thông và xác định tuyến mục tiêu

Sau khi xây dựng vùng giao thông chức năng như trình bày ở Mục 4.4, bước tiếp theo nhằm làm rõ các mối quan hệ ảnh hưởng giữa các tuyến đường trong vùng và xác định tập các tuyến mục tiêu cần dự đoán. Trong bối cảnh giao thông đô thị, ùn tắc không lan truyền một cách ngẫu nhiên mà thường tuân theo các mối quan hệ có hướng và có độ trễ theo thời gian, phản ánh động lực di chuyển của phương tiện trong mạng lưới giao thông. Do đó, việc khai thác các mối tương quan giao thông có xét đến độ trễ đóng vai trò then chốt trong việc xác định các tuyến có ảnh hưởng quan trọng.

4.5.1 Tương quan chéo có độ trễ trong vùng giao thông

Đối với mỗi vùng giao thông chức năng, nhóm chỉ xét các tuyến đường thuộc vùng này khi tính toán tương quan, nhằm đảm bảo tính cục bộ và giảm độ phức tạp tính toán. Với mỗi cặp tuyến đường (i, j) trong vùng, từ chuỗi thời gian vận tốc đã được chuẩn hóa $S_i(t)$ và $S_j(t)$, hàm tương quan chéo có độ trễ $X_{i,j}(\tau)$ được tính trên một cửa sổ thời gian lịch sử, với $\tau \in [-\tau_{\max}, \tau_{\max}]$.

Từ hàm tương quan chéo này, nhóm xác định hai đại lượng đặc trưng cho mỗi quan hệ giữa hai tuyến đường: (i) trọng số liên kết dương $W_{i,j}^{\text{pos}}$, phản ánh mức độ nổi trội của đỉnh tương quan so với nền nhiễu, và (ii) độ trễ tương ứng $\tau_{i,j}^{\text{pos}}$, xác định hướng và thời điểm ảnh hưởng mạnh nhất. Các đại lượng này được tính theo khuôn khổ mạng tương quan giao thông được đề xuất bởi Guo et al. [guo2019identify].

Việc sử dụng $W_{i,j}^{\text{pos}}$ thay vì giá trị tương quan cực đại thuận tiện cho phép loại bỏ các mối tương quan yếu hoặc ngẫu nhiên, đồng thời làm nổi bật các liên kết có ý nghĩa thống kê trong động lực giao thông. Trong khi đó, $\tau_{i,j}^{\text{pos}}$ cung cấp thông tin về quan hệ lead-lag giữa các tuyến, phản ánh hướng lan truyền tiềm năng của ùn tắc.

4.5.2 Loại liên kết tương quan theo tiêu chí thống kê và không gian

Không phải mọi cặp tuyến có tương quan dương đều mang ý nghĩa chức năng trong bối cảnh lan truyền ùn tắc. Do đó, nhóm áp dụng một tập các tiêu chí lọc để chỉ giữ lại các liên kết tương quan mạnh và hợp lý về mặt không gian-thời gian. Cụ thể, một liên kết (i, j) được giữ lại nếu thỏa mãn đồng thời các điều kiện sau:

$$W_{i,j}^{\text{pos}} \geq W_{\min}, \quad D_{\min} \leq D_{i,j} \leq D_{\max}, \quad |\tau_{i,j}^{\text{pos}}| \leq \tau_{\text{cut}}, \quad (25)$$

trong đó $D_{i,j}$ là khoảng cách Haversine giữa hai tuyến đường i và j . Các ngưỡng $W_{\min}$, $D_{\min}$, $D_{\max}$ và τ_{cut} được lựa chọn nhằm cân bằng giữa việc giữ lại các liên kết có ý nghĩa và loại bỏ nhiễu.

Bước lọc này đảm bảo rằng các liên kết tương quan được sử dụng trong mô hình không chỉ mạnh về mặt thống kê, mà còn phù hợp với ràng buộc vật lý và động học của giao thông đô thị. Các liên kết có khoảng cách quá xa hoặc độ trễ quá lớn thường khó phản ánh mối quan hệ ảnh hưởng trực tiếp và do đó bị loại bỏ.

4.5.3 Xác định tập tuyến mục tiêu

Từ tập các liên kết tương quan thỏa mãn các tiêu chí lọc, nhóm xác định *tập các tuyến mục tiêu* trong mỗi vùng giao thông. Cụ thể, một tuyến đường j được xem là tuyến mục tiêu nếu nó xuất hiện trong ít nhất một liên kết tương quan được giữ lại với bất kỳ tuyến nào khác trong vùng. Trong trường hợp số lượng tuyến mục tiêu quá lớn, nhóm chỉ giữ lại một tập con gồm các tuyến có trọng số liên kết W^{pos} lớn nhất.

4.6 Biểu diễn không gian bằng Laplacian Positional Encoding

Trong các mô hình Transformer tiêu chuẩn, cơ chế self-attention không tự thân nắm bắt được thông tin vị trí hay cấu trúc của các phần tử đầu vào, mà cần được bổ sung thông qua các cơ chế mã hóa vị trí (positional encoding). Đối với dữ liệu chuỗi, vị trí có thể được xác định một cách tự nhiên theo trật tự thời gian. Tuy nhiên, trong bài toán giao thông trên đồ thị, các tuyến đường không được sắp xếp theo một trật tự tuyến tính cố định, mà tồn tại trong một không gian phi Euclid được xác định bởi cấu trúc mạng lưới giao thông.

Do đó, để cung cấp thông tin không gian cho mô hình Transformer, nhóm sử dụng *Laplacian Positional Encoding*, một phương pháp mã hóa vị trí dựa trên phổ của đồ thị, đã được chứng minh là hiệu quả trong các mô hình Transformer cho dữ liệu đồ thị.

4.6.1 Xây dựng đồ thị không gian trong vùng giao thông

Đối với mỗi vùng giao thông chức năng được xây dựng ở các bước trước, nhóm xem vùng này như một đồ thị con $G_z = (V_z, E_z)$ của đồ thị giao thông toàn cục, trong đó mỗi đỉnh $v_i \in V_z$ tương ứng với một tuyến đường trong vùng, và các cạnh E_z được xác định dựa trên quan hệ kết nối topology giữa các tuyến trong mạng lưới giao thông.

Từ đồ thị con này, một ma trận kề $A_z \in \mathbb{R}^{|V_z| \times |V_z|}$ được xây dựng, trong đó $A_z(i, j) = 1$ nếu tồn tại cạnh nối giữa hai tuyến i và j trong đồ thị giao thông, và $A_z(i, j) = 0$ trong trường hợp ngược lại. Trên cơ sở đó, ma trận bậc D_z được xác định với $D_z(i, i) = \sum_j A_z(i, j)$.

4.6.2 Laplacian và phân rã phổ

Ma trận Laplacian của đồ thị vùng được định nghĩa dưới dạng Laplacian tổ hợp:

$$L_z = D_z - A_z. \quad (26)$$

Ma trận Laplacian này mã hóa đầy đủ cấu trúc kết nối của vùng giao thông và đóng vai trò như một toán tử khuếch tán trên đồ thị. Thực hiện phân rã phổ của L_z cho phép ta thu được hệ eigenvalue và eigenvector:

$$L_z = U_z \Lambda_z U_z^T, \quad (27)$$

trong đó $U_z = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_{|V_z|}]$ là các eigenvector tương ứng với các eigenvalue được sắp xếp theo thứ tự không giảm $0 = \lambda_1 \leq \lambda_2 \leq \dots$.

Trong nghiên cứu này, nhóm chỉ sử dụng d_{spa} eigenvector ứng với các eigenvalue nhỏ nhất (bỏ qua eigenvector hằng ứng với $\lambda_1 = 0$ nếu cần), tạo thành ma trận mã hóa vị trí không gian:

$$\mathbf{P}_z = [\mathbf{u}_2, \mathbf{u}_3, \dots, \mathbf{u}_{d_{\text{spa}}+1}] \in \mathbb{R}^{|V_z| \times d_{\text{spa}}}. \quad (28)$$

Việc lựa chọn các eigenvector bậc thấp tương ứng với các thành phần tần số thấp của đồ thị cho phép mã hóa cấu trúc không gian một cách trơn (smooth) và mang tính toàn cục, phản ánh vai trò cấu trúc của mỗi tuyến đường trong vùng giao thông.

4.6.3 Ý nghĩa của eigenvector bậc thấp trong bối cảnh giao thông

Các eigenvector bậc thấp của Laplacian đồ thị có thể được xem như các “tọa độ cấu trúc” (structural coordinates) của các tuyến đường trong vùng. Các tuyến có vị trí cấu trúc gần nhau trong đồ thị (ví dụ cùng nằm ở trung tâm vùng hoặc cùng thuộc rìa vùng) sẽ có biểu diễn gần nhau trong không gian phổ, ngay cả khi chúng không kề nhau trực tiếp về mặt topology.

Trong bối cảnh giao thông đô thị, điều này đặc biệt quan trọng vì hiện tượng ùn tắc thường lan truyền một cách liên tục và có cấu trúc theo vùng, thay vì thay đổi đột ngột trên từng tuyến riêng lẻ. Việc sử dụng các eigenvector bậc thấp giúp mô hình nắm bắt được cấu trúc không gian tổng thể của vùng giao thông, đồng thời giảm độ nhạy với nhiễu cục bộ hoặc các liên kết không ổn định.

4.6.4 Tích hợp Laplacian Positional Encoding vào Transformer

Ma trận mã hóa vị trí không gian $\mathbf{P}_z$ được sử dụng như một *spatial positional encoding* cho các tuyến đường trong vùng giao thông. Đối với mỗi mẫu huấn luyện, mã hóa này được ghép với biểu diễn theo thời gian của dữ liệu đầu vào và được cộng vào embedding đầu vào của mô hình Transformer.

Bằng cách này, cơ chế self-attention trong Transformer không chỉ dựa trên sự tương

đồng ngữ nghĩa của các đặc trưng giao thông theo thời gian, mà còn được điều kiện hóa bởi thông tin vị trí cấu trúc của các tuyến trong vùng. Điều này cho phép mô hình học được các tương tác không gian–thời gian phức tạp và vượt qua các giới hạn của các mô hình dựa trên cơ chế truyền thông tin cục bộ truyền thống.

Laplacian Positional Encoding do đó đóng vai trò then chốt trong việc cung cấp thông tin không gian cho mô hình Transformer encoder-only, tạo nền tảng cho việc học biểu diễn vùng giao thông một cách hiệu quả và có ý nghĩa về mặt cấu trúc.

4.7 Biểu diễn thời gian bằng Temporal Encoding

Bên cạnh cấu trúc không gian của mạng lưới giao thông, tín hiệu giao thông còn thể hiện tính chu kỳ mạnh theo thời gian, đặc biệt là theo *thời điểm trong ngày* (giờ cao điểm sáng/chiều) và *ngày trong tuần*. Do đó, để mô hình Transformer có thể phân biệt được các mẫu hình giao thông theo ngữ cảnh thời gian và khai thác được tính tuần hoàn, nhóm thiết kế một cơ chế *Temporal Encoding* để mã hóa thông tin thời gian và bổ sung vào embedding đầu vào.

4.7.1 Mã hóa vị trí tương đối theo bước thời gian

Đối với mỗi mẫu huấn luyện, dữ liệu đầu vào được cắt theo một cửa sổ thời gian dài L bước: $\{t - L + 1, \dots, t\}$. Vì Transformer không có khái niệm thứ tự nội tại giữa các token, nhóm sử dụng *relative position embedding* để mã hóa vị trí của mỗi bước thời gian trong cửa sổ này. Cụ thể, với chỉ số vị trí tương đối $p \in \{0, 1, \dots, L - 1\}$, mỗi vị trí được gán một vector embedding học được:

$$\mathbf{e}_p \in \mathbb{R}^{d_{\text{model}}}. \quad (29)$$

Cơ chế này giúp mô hình phân biệt rõ các quan sát gần hiện tại và các quan sát xa hơn trong quá khứ, từ đó học được động lực thay đổi của giao thông theo thời gian.

4.7.2 Mã hóa chu kỳ theo thời điểm trong ngày và ngày trong tuần

Để khai thác tính chu kỳ của giao thông, nhóm sử dụng các đặc trưng thời gian gồm: (i) thời điểm trong ngày đã chuẩn hóa $\text{tod}(t) \in [0, 1]$, và (ii) ngày trong tuần $\text{dow}(t) \in \{0, \dots, 6\}$. Các đặc trưng này được đưa về dạng biểu diễn lượng giác nhằm phản ánh tính tuần hoàn:

$$\mathbf{c}_{\text{tod}}(t) = [\sin(2\pi \text{tod}(t)), \cos(2\pi \text{tod}(t))], \quad (30)$$

$$\mathbf{c}_{\text{dow}}(t) = [\sin(2\pi \text{dow}(t)/7), \cos(2\pi \text{dow}(t)/7)]. \quad (31)$$

Sau đó, nhóm ghép các thành phần trên thành một vector 4 chiều và chiếu tuyến tính sang không gian d_{model} :

$$\mathbf{g}(t) = \mathbf{W}_t [\mathbf{c}_{\text{tod}}(t) \parallel \mathbf{c}_{\text{dow}}(t)] + \mathbf{b}_t, \quad \mathbf{W}_t \in \mathbb{R}^{d_{\text{model}} \times 4}. \quad (32)$$

4.7.3 Kết hợp Temporal Encoding vào embedding đầu vào

Đối với mỗi bước thời gian trong cửa sổ đầu vào, Temporal Encoding được tạo bằng cách cộng hai thành phần: (1) embedding vị trí tương đối theo chỉ số p và (2) embedding chu kỳ theo ngữ cảnh thời gian thực:

$$\mathbf{t}(p) = \mathbf{e}_p + \mathbf{g}(t - L + 1 + p), \quad p = 0, \dots, L - 1. \quad (33)$$

Vector $\mathbf{t}(p)$ sau đó được cộng vào embedding của tất cả các tuyến đường tại bước thời gian p . Nhờ đó, cơ chế self-attention có thể điều kiện hóa trên ngữ cảnh thời gian, giúp mô hình học được các mẫu hình giao thông mang tính lặp lại (ví dụ cao điểm sáng/chiều) và phân biệt các trạng thái giao thông giữa các ngày trong tuần.

Tóm lại, Temporal Encoding cung cấp cho mô hình Transformer một tín hiệu thời gian giàu ngữ cảnh, bổ sung cho Spatial Encoding dựa trên Laplacian, tạo nền tảng cho việc học biểu diễn không gian-thời gian trong phần kiến trúc mô hình trình bày ở mục tiếp theo.

5 Kiến trúc mô hình

5.1 Đầu vào của mô hình

Mô hình được thiết kế để xử lý dữ liệu giao thông theo vùng, trong đó mỗi mẫu đầu vào bao gồm chuỗi quan sát giao thông trong một cửa sổ thời gian, cùng với thông tin ngữ cảnh không gian và thời gian tương ứng. Cụ thể, đầu vào của mô hình bao gồm ba thành phần chính:

- **Chuỗi giao thông theo vùng:** lịch sử tốc độ giao thông của các tuyến đường trong cùng một vùng giao thông chức năng.
- **Thông tin thời gian:** bao gồm thời điểm trong ngày và ngày trong tuần, được mã hóa thông qua Temporal Encoding nhằm phản ánh tính chu kỳ của giao thông đô thị.
- **Thông tin không gian:** biểu diễn vị trí cấu trúc của các tuyến đường trong vùng

thông qua Laplacian Positional Encoding dựa trên phổ đồ thị.

Ba thành phần này được kết hợp tại tầng embedding đầu vào và cung cấp cho Transformer encoder một biểu diễn thống nhất, trong đó mỗi token đồng thời mang thông tin giao thông, vị trí không gian và ngữ cảnh thời gian.

5.2 Attention score bias trong Transformer

Khác với Transformer tiêu chuẩn, trong đó trọng số attention chỉ phụ thuộc vào độ tương đồng giữa truy vấn (query) và khóa (key), mô hình của nhóm cho phép tích hợp thêm một *attention score bias* bằng cách cộng một thành phần điều chỉnh vào ma trận điểm attention trước khi áp dụng hàm softmax. Cụ thể, với một head attention, điểm attention giữa hai token i và j được tính như sau:

$$\alpha_{i,j} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} + \gamma B_{i,j}, \quad (34)$$

trong đó $\mathbf{q}_i$ và $\mathbf{k}_j$ lần lượt là vector truy vấn và khóa tương ứng, d_k là kích thước không gian khóa, $B_{i,j}$ là attention bias phản ánh mối quan hệ chức năng giữa hai token, và γ là một hệ số vô hướng được học trong quá trình huấn luyện.

Sau đó, trọng số attention được chuẩn hóa theo softmax:

$$\text{Attn}_{i,j} = \frac{\exp(\alpha_{i,j})}{\sum_{j'} \exp(\alpha_{i,j'})}. \quad (35)$$

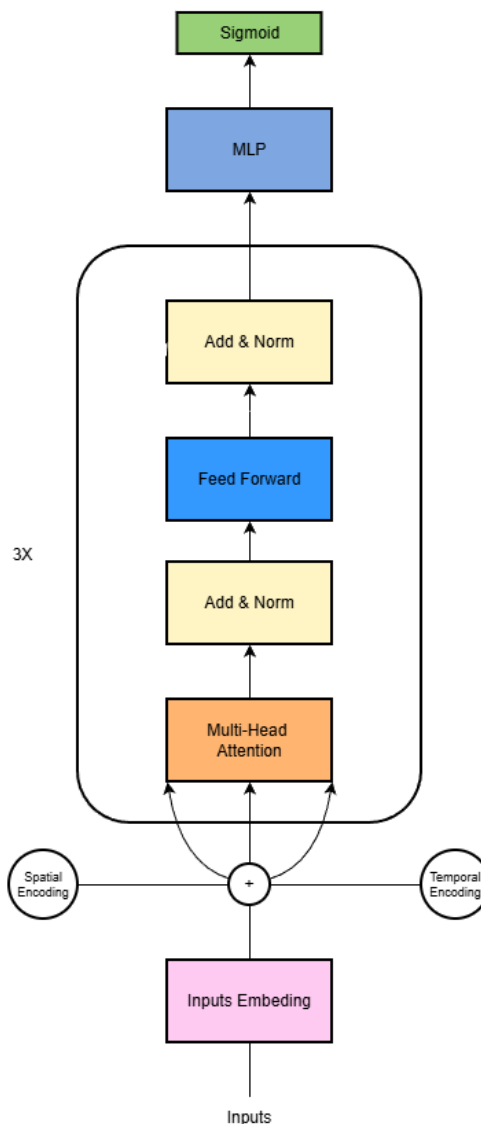
Attention bias $B_{i,j}$ được xây dựng bên ngoài mô hình Transformer và đóng vai trò mã hóa thông tin bổ sung về quan hệ giữa các tuyến đường trong cùng một vùng giao thông, chẳng hạn như mối tương quan giao thông có độ trễ hoặc mối liên hệ không gian-cấu trúc trong mạng lưới. Việc đưa bias vào dưới dạng một thành phần cộng cho phép mô hình điều chỉnh điểm attention mà không làm thay đổi cấu trúc cơ bản của cơ chế self-attention.

Khác với các Transformer đồ thị như Graphormer, trong đó bias cấu trúc thường được thiết kế cố định và phụ thuộc vào khoảng cách đồ thị toàn cục, attention bias trong mô hình của chúng tôi mang tính linh hoạt theo từng vùng giao thông và từng thời điểm. Hơn nữa, hệ số γ được học cho phép mô hình tự động cân bằng giữa thông tin cấu trúc được mã hóa trong bias và độ tương đồng ngữ nghĩa thuần túy giữa các token.

Trong quá trình thực nghiệm, nhóm đã từng làm mô hình không sử dụng attention score bias trong cùng một thiết lập huấn luyện. Kết quả cho thấy, mô hình không có attention bias kém hiệu quả hơn. Mô hình tích hợp attention bias giúp cải thiện độ ổn định của

quá trình huấn luyện và mang lại hiệu năng dự báo tốt hơn so với Transformer tiêu chuẩn không có bias. Điều này cho thấy rằng việc đưa thông tin quan hệ chức năng trực tiếp vào điểm attention là một cơ chế hiệu quả để tăng cường khả năng mô hình hóa các tương tác không gian–thời gian trong bài toán giao thông đô thị.

5.3 Transformer encoder-only theo phong cách BERT



Hình 3: Kiến trúc mô hình Transformer encoder-only với Spatial và Temporal Encoding

Sau khi kết hợp các embedding đầu vào, dữ liệu được đưa vào một Transformer encoder-only theo phong cách BERT. Kiến trúc này bao gồm nhiều lớp encoder xếp chồng, mỗi lớp có cùng cấu trúc chuẩn gồm self-attention đa đầu và mạng feed-forward theo vị trí.

Mỗi lớp Transformer encoder trong mô hình bao gồm hai khối chính. Khối thứ nhất là

multi-head self-attention, cho phép mỗi token trong vùng giao thông tương tác trực tiếp với tất cả các token khác trong cùng cửa sổ thời gian, nhờ đó mô hình có thể học được các quan hệ không gian–thời gian mang tính toàn cục.

Khối thứ hai là *feed-forward network* (FFN), bao gồm hai lớp tuyến tính với hàm kích hoạt phi tuyến ở giữa. FFN được áp dụng độc lập cho từng token và đóng vai trò tăng khả năng biểu diễn phi tuyến của mô hình sau bước attention.

Trong mỗi lớp encoder, chuẩn hóa lớp (Layer Normalization) được áp dụng trước mỗi khối self-attention và FFN, theo thiết kế *pre-norm*. Thiết kế này giúp ổn định quá trình huấn luyện, đặc biệt khi sử dụng nhiều lớp encoder. Các kết nối dư (residual connections) được sử dụng xuyên suốt để bảo toàn thông tin và hỗ trợ lan truyền gradient.

Để giảm hiện tượng overfitting, dropout được áp dụng tại các đầu ra của self-attention và bên trong mạng feed-forward. Việc sử dụng dropout ở nhiều vị trí giúp mô hình tổng quát tốt hơn khi huấn luyện trên dữ liệu giao thông có tính nhiễu và biến động cao.

5.4 Lớp dự đoán MLP

Sau khi đi qua các lớp Transformer encoder, mô hình sử dụng biểu diễn tại thời điểm cuối của cửa sổ thời gian để thực hiện dự đoán. Đối với mỗi tuyến đường trong vùng, embedding tương ứng được đưa qua một lớp MLP dùng chung tham số giữa các tuyến.

Lớp MLP này bao gồm một tầng ẩn với hàm kích hoạt phi tuyến, tiếp theo là dropout để giảm overfitting, và một tầng tuyến tính cuối cùng sinh ra một logit cho mỗi tuyến. Thiết kế MLP nông giúp giữ cho tầng dự đoán đơn giản, trong khi phần lớn năng lực mô hình được tập trung vào việc học biểu diễn không gian–thời gian thông qua Transformer encoder.

Kiến trúc dự đoán này phù hợp với giả định rằng các tuyến đường trong cùng một vùng giao thông chia sẻ các quy luật động học tương tự, đồng thời cho phép mô hình tổng quát hóa tốt sang các vùng giao thông khác nhau.

6 Kết quả thực nghiệm và đánh giá

6.1 Thiết lập bài toán và tiêu chí đánh giá

Trong phần này, nhóm trình bày kết quả thực nghiệm của mô hình đề xuất trên bài toán dự đoán xác suất ùn tắc sau 15 phút kể từ thời điểm quan sát cuối cùng của cửa sổ lịch sử. Các kết quả được báo cáo trên tập kiểm tra (test set), với mô hình được lựa chọn dựa trên giá trị F1-score cao nhất trên tập validation.

Do dữ liệu giao thông có tính chất chuỗi thời gian mạnh và tồn tại sự phụ thuộc theo

thời gian, việc chia tập dữ liệu một cách ngẫu nhiên (random split) có thể dẫn đến hiện tượng *data leakage*, làm sai lệch kết quả đánh giá. Vì vậy, trong nghiên cứu này, nhóm áp dụng chiến lược chia tập theo thứ tự thời gian

Cụ thể, toàn bộ dữ liệu được sắp xếp theo trục thời gian và được chia thành ba tập không giao nhau như sau:

- **Tập huấn luyện (train set):** chiếm 70% khoảng thời gian đầu tiên của dữ liệu, được sử dụng để học các tham số của mô hình.
- **Tập xác thực (validation set):** chiếm 15% khoảng thời gian tiếp theo, dùng để lựa chọn mô hình tốt nhất và điều chỉnh các quyết định huấn luyện (ví dụ lựa chọn epoch tốt nhất).
- **Tập kiểm tra (test set):** chiếm 15% khoảng thời gian cuối cùng, chỉ được sử dụng để đánh giá hiệu năng cuối cùng của mô hình.

Việc chia tập được thực hiện ở **cấp độ ngày (day-level split)**, tức là toàn bộ các mẫu thuộc cùng một ngày sẽ chỉ xuất hiện trong một tập duy nhất. Cách làm này giúp tránh việc các mẫu có mối liên hệ thời gian gần nhau bị phân tán giữa các tập khác nhau.

Ngoài ra, dữ liệu trong mỗi ngày được chia thành hai phiên giao thông độc lập:

- phiên buổi sáng (07:00–10:00),
- phiên buổi chiều (15:00–18:00).

Các tuyến đường mục tiêu dự đoán của mô hình: Dự đoán các tuyến có trong cluster được chọn sau khi DBSCAN và các tuyến liên quan mạnh tới các tuyến ở trong cluster đó

Khi xây dựng các mẫu huấn luyện theo cửa sổ thời gian, nhóm không cho phép cửa sổ vượt qua ranh giới giữa hai phiên nhằm đảm bảo tính nhất quán về ngữ cảnh giao thông.

Đối với mỗi mẫu, nhãn dự đoán tại thời điểm $t + \Delta$ (với $\Delta = 15$ phút trong thí nghiệm này) được đảm bảo thuộc cùng tập dữ liệu với cửa sổ đầu vào. Điều này giúp loại bỏ hoàn toàn khả năng mô hình sử dụng thông tin tương lai trong quá trình huấn luyện hoặc đánh giá.

Ở mỗi batch (ít trong ảnh), với tỉ lệ 60:40 cho kẹt và không kẹt, mô hình sẽ quyết định nên dùng dbscan cho kẹt hay không kẹt. Từ đó lấy ra các cluster tương ứng với trạng thái ùn tắc hiện tại, sau đó thì chọn ngẫu nhiên một cluster để train cho mô hình. Mỗi batch mô hình sẽ chọn ngẫu nhiên 8 clusters để train. Bên cạnh đó, tập valid, test cũng sẽ đổi

clusters chứ không cố định sau mỗi epoch. Điều này nhằm kiểm tra và đảm bảo sự tổng quát hóa của mô hình và khả năng dự đoán của mô hình khi phải yêu cầu dự đoán một tuyến đường từ trạng thái một vùng bất kỳ

6.2 Quá trình huấn luyện và hội tụ

Mô hình được huấn luyện khoảng 150 epoch và ưu tiên chọn state có recall tốt nhất

```
===== Epoch 149/150 | threshold=0.3 =====
[E149][train] it=0000 loss=0.213012 eval_nodes=139 valid_nodes=163
[E149][train] it=0001 loss=0.276967 eval_nodes=537 valid_nodes=571
[E149][train] it=0002 loss=0.328699 eval_nodes=521 valid_nodes=557
[E149][train] it=0003 loss=0.213563 eval_nodes=93 valid_nodes=114
[E149][train] it=0004 loss=0.046399 eval_nodes=191 valid_nodes=232
[E149][train] it=0005 loss=0.074337 eval_nodes=148 valid_nodes=184
[E149][train] it=0006 loss=0.137197 eval_nodes=94 valid_nodes=103
[E149][train] it=0007 loss=0.221346 eval_nodes=111 valid_nodes=122
[E149][train] it=0008 loss=0.012122 eval_nodes=107 valid_nodes=199
[E149][train] it=0009 loss=0.287712 eval_nodes=62 valid_nodes=90
[E149][train] it=0010 loss=0.349959 eval_nodes=424 valid_nodes=492
[E149][train] it=0011 loss=0.088939 eval_nodes=102 valid_nodes=145
[E149][train] it=0012 loss=0.185938 eval_nodes=91 valid_nodes=165
[E149][train] it=0013 loss=0.177004 eval_nodes=156 valid_nodes=177
[E149][train] it=0014 loss=0.239131 eval_nodes=98 valid_nodes=113
[E149][train] it=0015 loss=0.267758 eval_nodes=513 valid_nodes=553
[E149][train] it=0016 loss=0.034158 eval_nodes=111 valid_nodes=174
[E149][train] it=0017 loss=0.355588 eval_nodes=536 valid_nodes=587
[E149][train] it=0018 loss=0.046529 eval_nodes=204 valid_nodes=239
[E149][train] it=0019 loss=0.107219 eval_nodes=118 valid_nodes=129
[E149][train] it=0020 loss=0.076917 eval_nodes=118 valid_nodes=160
[E149][train] it=0021 loss=0.066520 eval_nodes=122 valid_nodes=156
[E149][train] it=0022 loss=0.351513 eval_nodes=552 valid_nodes=572
[E149][train] it=0023 loss=0.094907 eval_nodes=134 valid_nodes=182
[E149][train] it=0024 loss=0.352265 eval_nodes=511 valid_nodes=534
[E149][train] it=0025 loss=0.195134 eval_nodes=134 valid_nodes=175
[E149][train] it=0026 loss=0.175848 eval_nodes=112 valid_nodes=133
[E149][train] it=0027 loss=0.102821 eval_nodes=155 valid_nodes=213
[E149][train] it=0028 loss=0.011300 eval_nodes=193 valid_nodes=315
[E149][train] it=0029 loss=0.362808 eval_nodes=63 valid_nodes=157
[E149][valid] it=0000 loss=0.092058 eval_nodes=146 valid_nodes=155
[E149][valid] it=0001 loss=0.133200 eval_nodes=125 valid_nodes=136
[E149][valid] it=0002 loss=0.073069 eval_nodes=176 valid_nodes=215
[E149][valid] it=0003 loss=0.118997 eval_nodes=134 valid_nodes=151
[E149][valid] it=0004 loss=0.077243 eval_nodes=97 valid_nodes=120
[E149][valid] it=0005 loss=0.328054 eval_nodes=531 valid_nodes=564
[E149][test] it=0000 loss=0.068080 eval_nodes=126 valid_nodes=128
[E149][test] it=0001 loss=0.147320 eval_nodes=167 valid_nodes=219
[E149][test] it=0002 loss=0.025482 eval_nodes=164 valid_nodes=179
[E149][test] it=0003 loss=0.086080 eval_nodes=181 valid_nodes=232
[E149][test] it=0004 loss=0.263022 eval_nodes=114 valid_nodes=127
[E149][test] it=0005 loss=0.387432 eval_nodes=466 valid_nodes=489
[E149][test] it=0006 loss=0.004080 eval_nodes=224 valid_nodes=257
[E149][test] it=0007 loss=0.095621 eval_nodes=135 valid_nodes=167
[E149] train: loss=0.181787 acc=0.9099 prec=0.8630 rec=0.9763 f1=0.9162 eval_nodes=6450 valid_nodes=7706
[E149] valid: loss=0.137103 acc=0.9247 prec=0.8683 rec=0.9709 f1=0.9167 eval_nodes=1209 valid_nodes=1341
[E149] test : loss=0.134640 acc=0.9296 prec=0.8586 rec=0.9730 f1=0.9123 eval_nodes=1577 valid_nodes=1798
```

Hình 4: Kết quả của Epoch 149

```

===== Epoch 150/150 | threshold=0.3 =====
[E150][train] it=0000 loss=0.084951 eval_nodes=141 valid_nodes=610
[E150][train] it=0001 loss=0.130708 eval_nodes=62 valid_nodes=138
[E150][train] it=0002 loss=0.156479 eval_nodes=68 valid_nodes=101
[E150][train] it=0003 loss=0.199562 eval_nodes=122 valid_nodes=181
[E150][train] it=0004 loss=0.038189 eval_nodes=175 valid_nodes=211
[E150][train] it=0005 loss=0.090633 eval_nodes=188 valid_nodes=270
[E150][train] it=0006 loss=0.218719 eval_nodes=118 valid_nodes=257
[E150][train] it=0007 loss=0.040474 eval_nodes=108 valid_nodes=165
[E150][train] it=0008 loss=0.169092 eval_nodes=146 valid_nodes=184
[E150][train] it=0009 loss=0.019628 eval_nodes=110 valid_nodes=154
[E150][train] it=0010 loss=0.285019 eval_nodes=87 valid_nodes=110
[E150][train] it=0011 loss=0.113105 eval_nodes=103 valid_nodes=115
[E150][train] it=0012 loss=0.310958 eval_nodes=471 valid_nodes=506
[E150][train] it=0013 loss=0.140859 eval_nodes=94 valid_nodes=143
[E150][train] it=0014 loss=0.265341 eval_nodes=113 valid_nodes=174
[E150][train] it=0015 loss=0.053681 eval_nodes=129 valid_nodes=150
[E150][train] it=0016 loss=0.100647 eval_nodes=122 valid_nodes=146
[E150][train] it=0017 loss=0.069963 eval_nodes=163 valid_nodes=211
[E150][train] it=0018 loss=0.047808 eval_nodes=127 valid_nodes=154
[E150][train] it=0019 loss=0.122624 eval_nodes=166 valid_nodes=200
[E150][train] it=0020 loss=0.123824 eval_nodes=189 valid_nodes=232
[E150][train] it=0021 loss=0.072426 eval_nodes=115 valid_nodes=160
[E150][train] it=0022 loss=0.280661 eval_nodes=489 valid_nodes=513
[E150][train] it=0023 loss=0.130642 eval_nodes=104 valid_nodes=167
[E150][train] it=0024 loss=0.397872 eval_nodes=505 valid_nodes=527
[E150][train] it=0025 loss=0.301261 eval_nodes=91 valid_nodes=121
[E150][train] it=0026 loss=0.046173 eval_nodes=81 valid_nodes=143
[E150][train] it=0027 loss=0.106107 eval_nodes=97 valid_nodes=184
[E150][train] it=0028 loss=0.077796 eval_nodes=121 valid_nodes=213
[E150][train] it=0029 loss=0.104851 eval_nodes=83 valid_nodes=195
[E150][valid] it=0000 loss=0.047367 eval_nodes=141 valid_nodes=159
[E150][valid] it=0001 loss=0.122780 eval_nodes=143 valid_nodes=173

[E150][valid] it=0001 loss=0.122780 eval_nodes=143 valid_nodes=173
[E150][valid] it=0002 loss=0.072419 eval_nodes=116 valid_nodes=121
[E150][valid] it=0003 loss=0.304067 eval_nodes=479 valid_nodes=506
[E150][valid] it=0004 loss=0.054957 eval_nodes=146 valid_nodes=162
[E150][valid] it=0005 loss=0.357570 eval_nodes=531 valid_nodes=566
[E150][test] it=0000 loss=0.062022 eval_nodes=123 valid_nodes=148
[E150][test] it=0001 loss=0.055352 eval_nodes=190 valid_nodes=223
[E150][test] it=0002 loss=0.093429 eval_nodes=122 valid_nodes=140
[E150][test] it=0003 loss=0.286319 eval_nodes=489 valid_nodes=503
[E150][test] it=0004 loss=0.030501 eval_nodes=162 valid_nodes=185
[E150][test] it=0005 loss=0.359523 eval_nodes=522 valid_nodes=537
[E150][test] it=0006 loss=0.141382 eval_nodes=134 valid_nodes=154
[E150][test] it=0007 loss=0.063432 eval_nodes=169 valid_nodes=233
[E150] train: loss=0.143335 acc=0.9298 prec=0.8738 rec=0.9637 f1=0.9166 eval_nodes=4688 valid_nodes=6635
[E150] valid: loss=0.159860 acc=0.9132 prec=0.8757 rec=0.9888 f1=0.9288 eval_nodes=1556 valid_nodes=1687
[E150] test : loss=0.136495 acc=0.9267 prec=0.8876 rec=0.9781 f1=0.9307 eval_nodes=1911 valid_nodes=2123

===== BEST checkpoint: epoch=108 valid_f1=0.9387 =====
[E108][test(best)] it=0000 loss=0.091626 eval_nodes=124 valid_nodes=142
[E108][test(best)] it=0001 loss=0.389924 eval_nodes=506 valid_nodes=548
[E108][test(best)] it=0002 loss=0.165267 eval_nodes=69 valid_nodes=84
[E108][test(best)] it=0003 loss=0.182842 eval_nodes=156 valid_nodes=182
[E108][test(best)] it=0004 loss=0.061043 eval_nodes=146 valid_nodes=159
[E108][test(best)] it=0005 loss=0.174307 eval_nodes=108 valid_nodes=134
[E108][test(best)] it=0006 loss=0.688295 eval_nodes=65 valid_nodes=75
[E108][test(best)] it=0007 loss=0.282460 eval_nodes=537 valid_nodes=583
[BEST] Test ROC-AUC = 0.962996
{'best_epoch': 108, 'best_valid_score': 0.9386934673366834, 'best_metric': 'f1', 'threshold': 0.3, 'checkpoint': './outputs2/best_model.pt', 'out_dir': './outputs2', 'test_best': {'loss': 0.2544705946929753, 'metrics': {'acc': 0.8901227352425483, 'prec': 0.8497695852534562, 'rec': 0.9736008447729673, 'f1': 0.90748031496063, 'counts': {'tp': 922, 'tn': 601, 'fp': 163, 'fn': 25}, 'eval_nodes': 1711, 'valid_nodes': 1907}, 'test_auc': 0.9629963346418433, 'plots': ['loss_curves.png', 'metrics_train_valid_test.png', 'test_confusion_matrix.png', 'test_pr_curve.png', 'test_roc_auc_curve.png']}

```

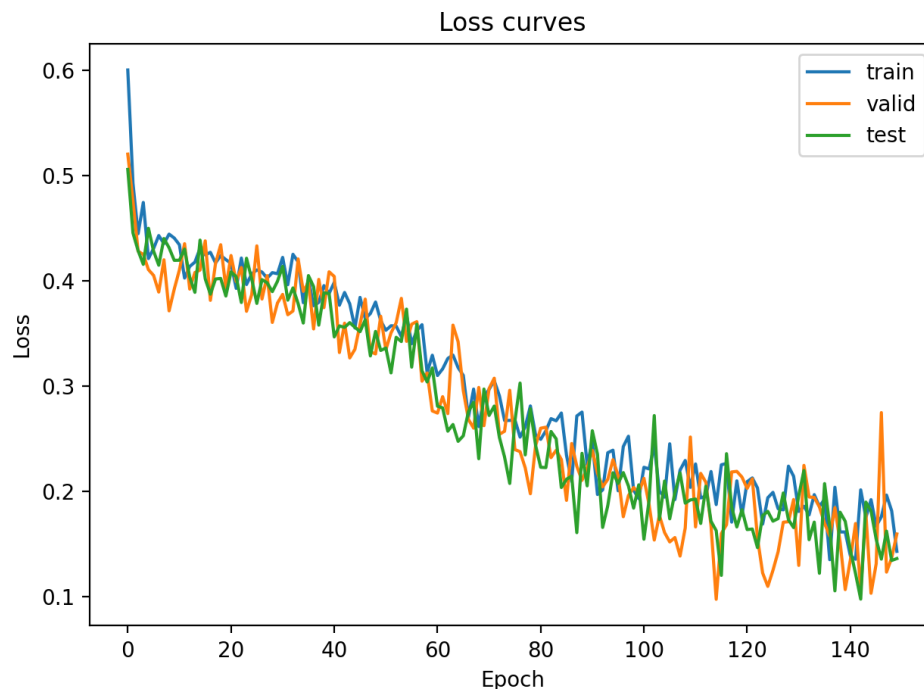
Hình 5: Kết quả của Epoch 150

Nhìn chung, có thể quan sát thấy hàm loss giảm dần từ epoch 149 đến epoch 150. Precision duy trì ở mức trên 85% và Recall ổn định trên 95% xuyên suốt các epoch trên cả ba tập dữ liệu train, validation và test.

Eval nodes là tổng số node được mô hình dự đoán sau khi áp dụng bước chọn lọc dựa trên mức độ tương quan mạnh nhất. Valid nodes là tập hợp bao gồm toàn bộ node trong

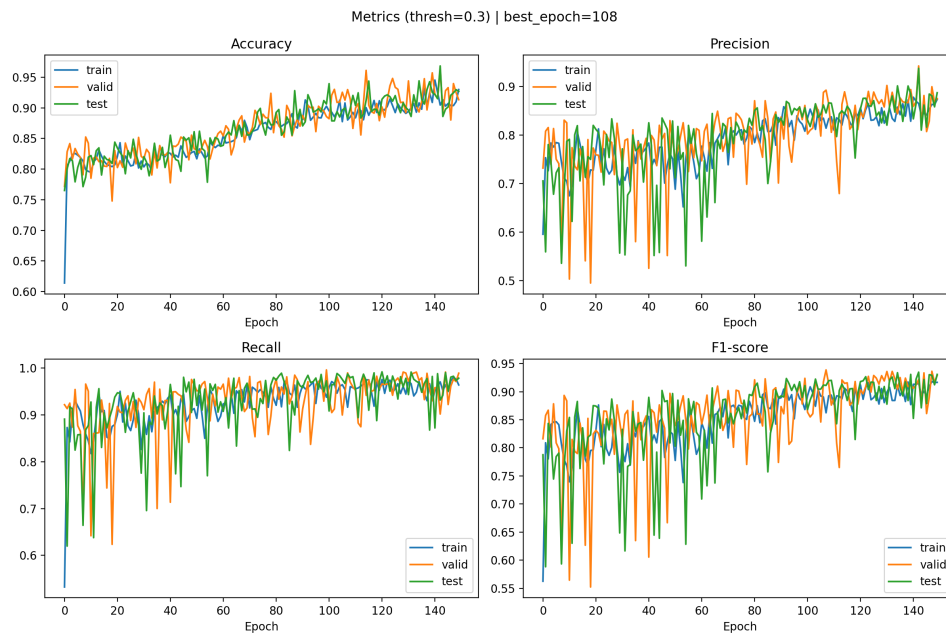
cluster cùng với các eval nodes.

Các chỉ số eval_nodes, valid_nodes, TP, FP, TN và FN tại mỗi epoch được tính bằng cách cộng dồn kết quả từ tất cả các batch trong epoch đó.



Hình 6: Hàm loss qua các epoch

Hình trên minh họa diễn biến hàm mất mát trên tập train, validation và test trong suốt quá trình huấn luyện. Có thể quan sát thấy rằng loss giảm dần và ổn định theo số epoch, cho thấy mô hình hội tụ tốt và không xuất hiện hiện tượng overfitting nghiêm trọng. Đường loss của ba tập có xu hướng bám sát nhau, phản ánh khả năng tổng quát hóa tốt của mô hình đối với dữ liệu chưa từng thấy.



Hình 7: Accuracy, Precision, Recall, F1-score trên train, valid, test qua các epoch

Bên cạnh đó, có thể thấy rằng hình trên trình bày sự thay đổi của các chỉ số accuracy, precision, recall và F1-score theo epoch. Các chỉ số này tăng dần và đạt trạng thái ổn định sau một số epoch đầu, cho thấy mô hình dần học được các quy luật không gian–thời gian quan trọng của dữ liệu giao thông.

Dựa trên F1-score của tập validation, checkpoint tốt nhất được lựa chọn tại epoch 108, và mô hình tại epoch này được sử dụng để đánh giá cuối cùng trên tập test.

6.3 Đánh giá định lượng trên tập kiểm tra

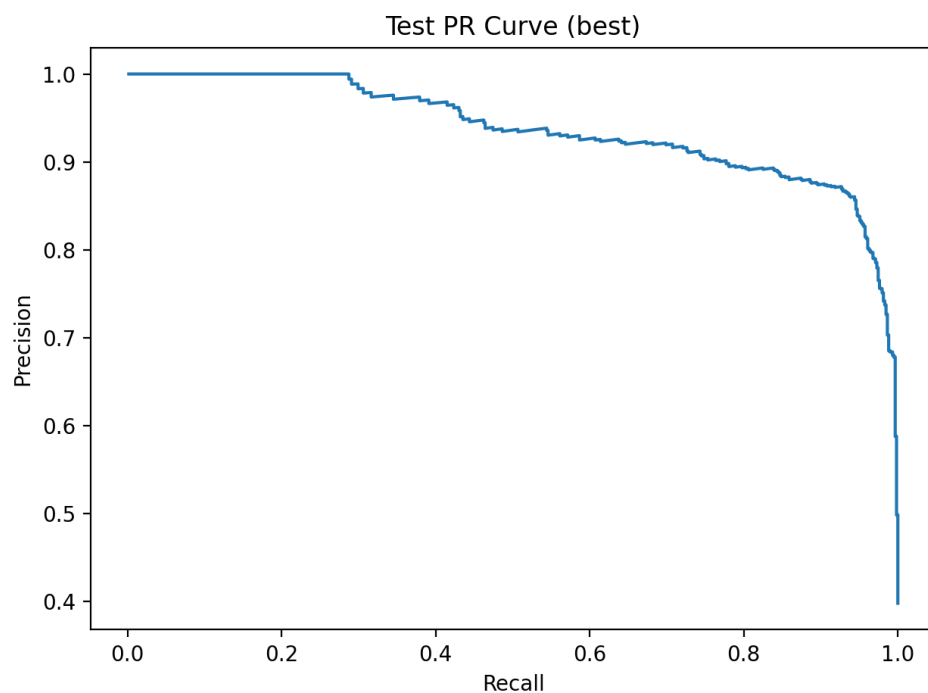
Bảng dưới đây tổng hợp các chỉ số đánh giá chính trên tập test tại checkpoint tốt nhất. Mô hình đạt F1-score cao, cùng với recall lớn, cho thấy khả năng phát hiện các trạng thái ùn tắc trong tương lai một cách hiệu quả, trong khi vẫn duy trì mức precision ổn định.

Bảng 6: Kết quả định lượng trên tập test (dự đoán 15 phút sau).

Metric	Accuracy	Precision	Recall	F1-score
Test set	0.89	0.85	0.97	0.91

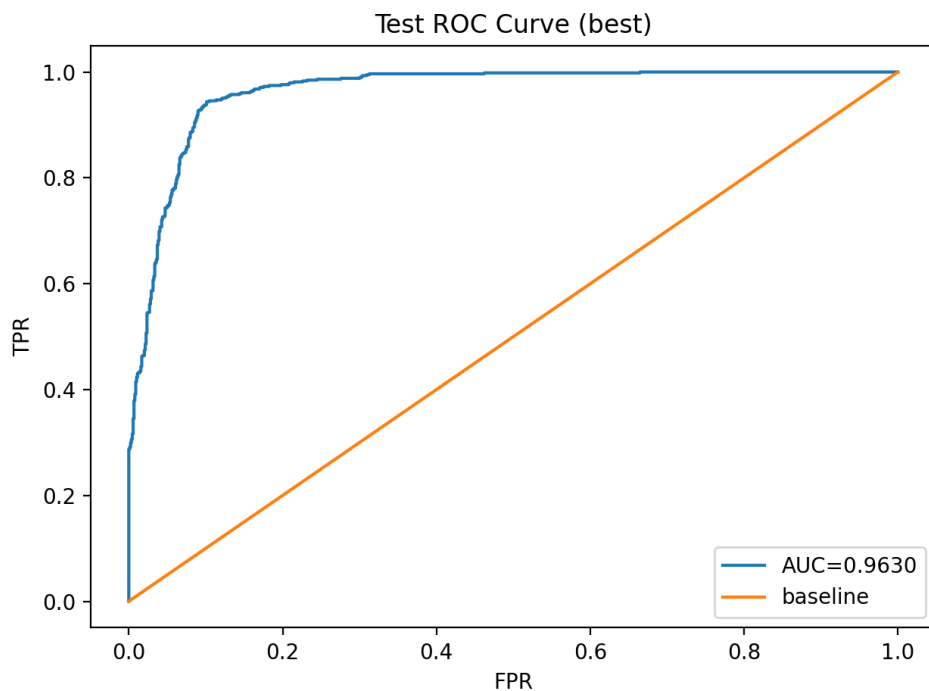
Giá trị recall cao cho thấy mô hình ít bỏ sót các trường hợp ùn tắc, điều này đặc biệt quan trọng trong các ứng dụng hỗ trợ điều hành giao thông, nơi việc phát hiện sớm ùn tắc có ý nghĩa thực tiễn lớn.

6.4 Phân tích đường cong PR và ROC



Hình 8: *Precision-Recall curve*

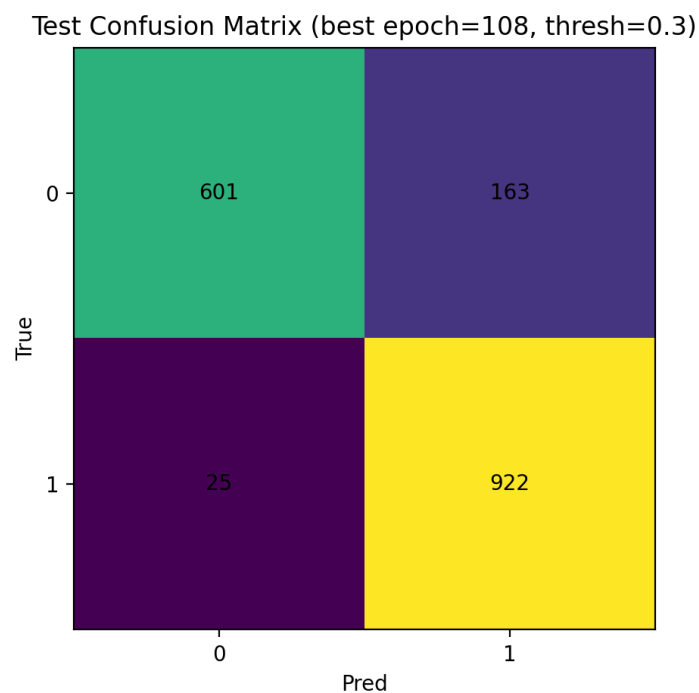
Hình trên trình bày đường cong Precision–Recall trên tập test tại checkpoint tốt nhất. Đường cong cho thấy mô hình duy trì precision cao trong một dải recall rộng, phản ánh khả năng cân bằng tốt giữa hai tiêu chí này ngay cả khi thay đổi ngưỡng phân loại.



Hình 9: *ROC, AUC*

Hình trên minh họa đường cong ROC tương ứng, với diện tích dưới đường cong (AUC) đạt giá trị cao. Điều này cho thấy mô hình có khả năng phân biệt rõ ràng giữa hai lớp ỳn tắc và không ỳn tắc, độc lập với lựa chọn ngưỡng phân loại.

6.5 Phân tích ma trận nhầm lẫn



Hình 10: Ma trận nhầm lẫn

Để phân tích chi tiết hơn các dạng sai lệch của mô hình, nhóm sẽ xem xét ma trận nhầm lẫn trên tập test tại checkpoint tốt nhất. Có thể thấy số lượng false negative là tương đối nhỏ so với true positive, cho thấy mô hình ưu tiên phát hiện đúng các trạng thái ứn tắc trong tương lai. Ngoài ra, TP và TN khá cao cho thấy khả năng dự đoán tốt của mô hình. Bên cạnh đó, FP ở mức trung bình điều này làm giảm precision của mô hình.

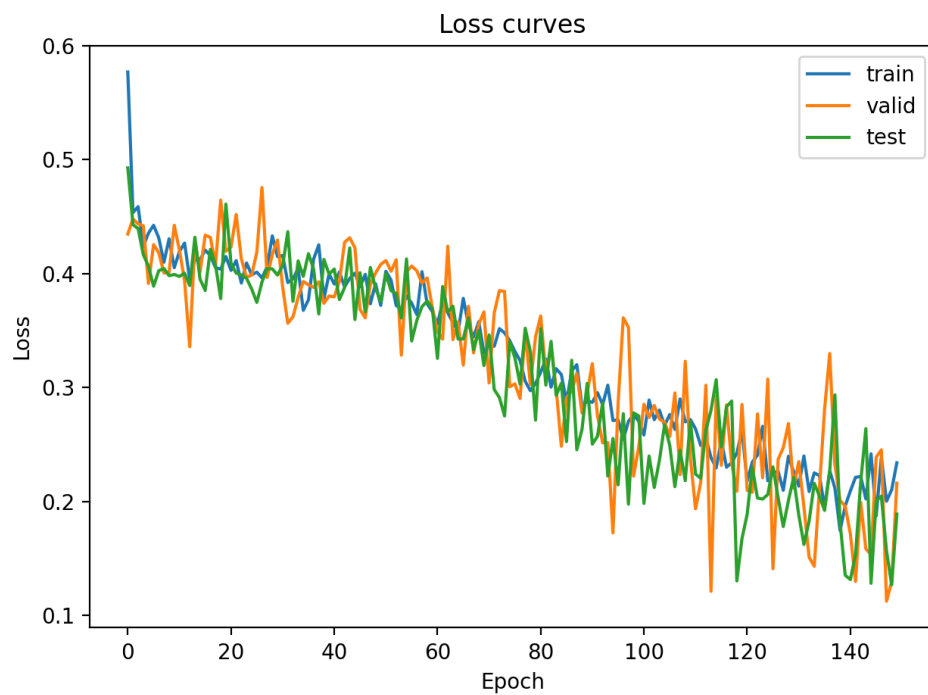
6.6 Một số số liệu phản ánh khả năng học và dự đoán của mô hình sau 30 phút

6.7 Thảo luận

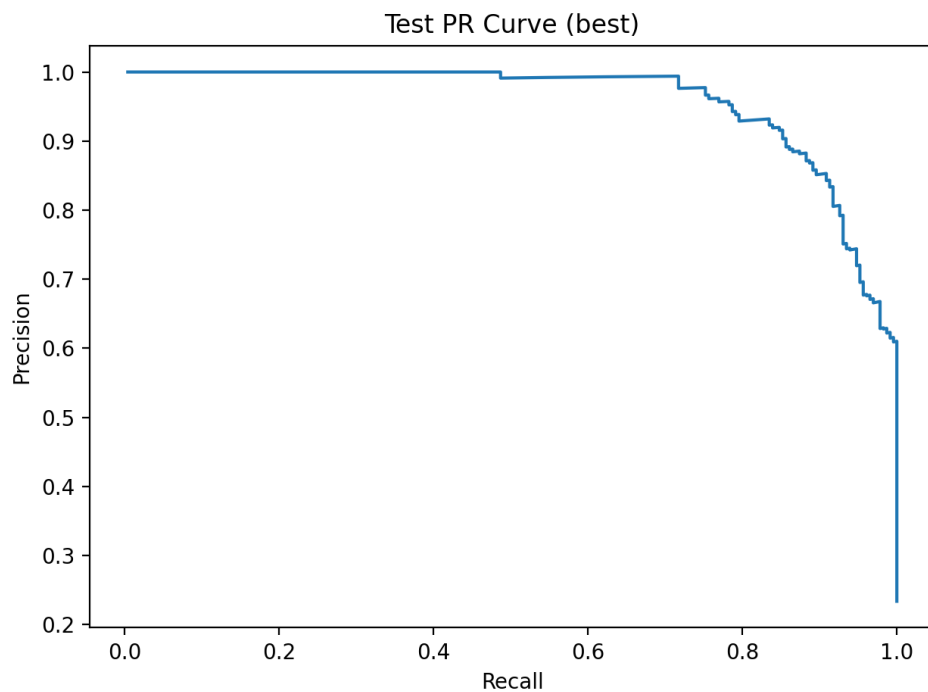
```
===== Epoch 149/150 | threshold=0.3 =====
[E149][train] it=0000 loss=0.314583 eval_nodes=534 valid_nodes=564
[E149][train] it=0001 loss=0.113628 eval_nodes=213 valid_nodes=250
[E149][train] it=0002 loss=0.264009 eval_nodes=538 valid_nodes=567
[E149][train] it=0003 loss=0.180254 eval_nodes=128 valid_nodes=161
[E149][train] it=0004 loss=0.099553 eval_nodes=145 valid_nodes=173
[E149][train] it=0005 loss=0.445242 eval_nodes=299 valid_nodes=368
[E149][train] it=0006 loss=0.234157 eval_nodes=99 valid_nodes=105
[E149][train] it=0007 loss=0.169155 eval_nodes=46 valid_nodes=68
[E149][train] it=0008 loss=0.053821 eval_nodes=153 valid_nodes=197
[E149][train] it=0009 loss=0.467809 eval_nodes=451 valid_nodes=522
[E149][train] it=0010 loss=0.267466 eval_nodes=501 valid_nodes=527
[E149][train] it=0011 loss=0.131540 eval_nodes=80 valid_nodes=162
[E149][train] it=0012 loss=0.199863 eval_nodes=113 valid_nodes=132
[E149][train] it=0013 loss=0.152353 eval_nodes=138 valid_nodes=202
[E149][train] it=0014 loss=0.283087 eval_nodes=72 valid_nodes=77
[E149][train] it=0015 loss=0.140877 eval_nodes=73 valid_nodes=90
[E149][train] it=0016 loss=0.429613 eval_nodes=526 valid_nodes=556
[E149][train] it=0017 loss=0.114446 eval_nodes=135 valid_nodes=158
[E149][train] it=0018 loss=0.111998 eval_nodes=101 valid_nodes=174
[E149][train] it=0019 loss=0.241471 eval_nodes=117 valid_nodes=222
[E149][train] it=0020 loss=0.053607 eval_nodes=217 valid_nodes=292
[E149][train] it=0021 loss=0.118487 eval_nodes=139 valid_nodes=200
[E149][train] it=0022 loss=0.203868 eval_nodes=86 valid_nodes=132
[E149][train] it=0023 loss=0.092999 eval_nodes=116 valid_nodes=171
[E149][train] it=0024 loss=0.146683 eval_nodes=134 valid_nodes=189
[E149][train] it=0025 loss=0.436714 eval_nodes=312 valid_nodes=356
[E149][valid] it=0000 loss=0.096423 eval_nodes=136 valid_nodes=157
[E149][valid] it=0001 loss=0.292676 eval_nodes=579 valid_nodes=623
[E149][valid] it=0002 loss=0.110351 eval_nodes=163 valid_nodes=181
[E149][valid] it=0003 loss=0.096632 eval_nodes=213 valid_nodes=258
[E149][valid] it=0004 loss=0.106866 eval_nodes=118 valid_nodes=154
[E149][valid] it=0005 loss=0.075934 eval_nodes=49 valid_nodes=68

[E149][test] it=0000 loss=0.049328 eval_nodes=183 valid_nodes=213
[E149][test] it=0001 loss=0.081759 eval_nodes=202 valid_nodes=239
[E149][test] it=0002 loss=0.063241 eval_nodes=108 valid_nodes=125
[E149][test] it=0003 loss=0.089119 eval_nodes=158 valid_nodes=173
[E149][test] it=0004 loss=0.388287 eval_nodes=375 valid_nodes=430
[E149][test] it=0005 loss=0.121637 eval_nodes=96 valid_nodes=136
[E149][test] it=0006 loss=0.094594 eval_nodes=132 valid_nodes=155
[E149] train: loss=0.210280 acc=0.8882 prec=0.8323 rec=0.9531 f1=0.8886 eval_nodes=5466 valid_nodes=6615
[E149] valid: loss=0.129814 acc=0.9205 prec=0.8532 rec=0.9812 f1=0.9127 eval_nodes=1258 valid_nodes=1441
[E149] test : loss=0.126852 acc=0.9115 prec=0.7943 rec=0.9088 f1=0.8477 eval_nodes=1254 valid_nodes=1471
```

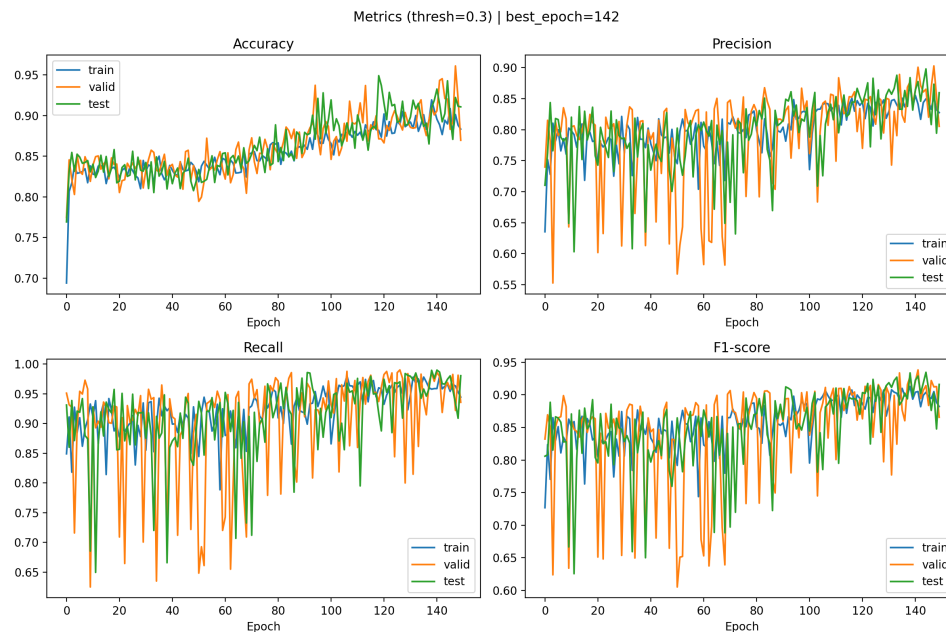
Hình 11: *Epoch 149*



Hình 12: *Hàm loss qua các epoch*



Hình 13: *Precision-Recall curve*



Hình 14: *Precision-Recall curve*

Ở đây mô hình được nạp và học dữ liệu lại và điều chỉnh để cho đúng với nhãn kẹt hoặc không kẹt 30 phút sau. Nhìn chung mô hình vẫn học được tốt, các hàm loss vẫn giảm đều và số liệu vẫn cho thấy mô hình có khả năng học được tốt

Kết quả thực nghiệm cho thấy mô hình Transformer đề xuất, với việc tích hợp bias vào attention score và các mã hóa không gian-thời gian, có khả năng dự báo hiệu quả trạng thái ùn tắc trong tương lai ngắn hạn (15 phút, 30 phút, ...). Sự ổn định của quá trình huấn luyện và các chỉ số đánh giá cao trên tập test cho thấy mô hình không chỉ học tốt trên dữ liệu huấn luyện mà còn tổng quát hóa tốt sang dữ liệu chưa từng thấy.

7 Hệ thống đề xuất

7.1 Mô tả hệ thống

Hệ thống được thiết kế nhằm hỗ trợ phân tích và đánh giá tình trạng giao thông (TTGT) trong môi trường đô thị thông qua việc xử lý, tổ chức và phân tích dữ liệu giao thông được cung cấp. Hệ thống cho phép mô hình hóa mạng lưới giao thông, nhận diện các mối quan hệ giữa các đoạn đường và khai thác dữ liệu theo cả hai khía cạnh không gian và thời gian để hỗ trợ đánh giá và hiểu rõ hơn về tình trạng lưu thông.

7.1.1 Mục đích của hệ thống

Mục tiêu chính của hệ thống bao gồm:

- Quản lý dữ liệu giao thông từ nhiều nguồn (dữ liệu bản đồ số, dữ liệu cảm biến, dữ liệu GPS, dữ liệu từ các nền tảng giao thông của bên thứ ba, v.v.).
- Tiền xử lý, chuyển đổi và chuẩn hóa dữ liệu để phục vụ phân tích.
- Mô hình hóa mối liên hệ giữa các đoạn đường trong mạng lưới giao thông nhằm hiểu rõ cấu trúc và dòng chảy giao thông.
- Phân tích dữ liệu để hỗ trợ nhận diện tình trạng ùn tắc, đánh giá xu hướng và hỗ trợ ra quyết định.
- Trực quan hóa kết quả phân tích bằng biểu đồ, bản đồ hoặc báo cáo nhằm giúp người dùng theo dõi và hiểu kết quả.
- Cung cấp nền tảng linh hoạt để thử nghiệm, so sánh hoặc tích hợp các phương pháp phân tích trong tương lai.

7.1.2 Nhóm người dùng mục tiêu

Hệ thống hướng đến ba nhóm người dùng chính:

- **Nhà nghiên cứu và sinh viên:** sử dụng hệ thống để phân tích dữ liệu giao thông, thử nghiệm các phương pháp mô hình hóa khác nhau và đánh giá các yếu tố ảnh hưởng đến TTGT.
- **Cơ quan quản lý giao thông hoặc nhà quy hoạch đô thị:** khai thác hệ thống để theo dõi tình trạng giao thông, phát hiện điểm nóng ùn tắc và đánh giá tác động của các giải pháp tổ chức giao thông.

- **Kỹ sư dữ liệu và nhà phát triển hệ thống:** sử dụng hệ thống như nền tảng để xử lý dữ liệu giao thông, mở rộng mô hình, tích hợp thêm tính năng hoặc triển khai vào môi trường thực tế.

7.1.3 Tổng quan

Tổng thể, hệ thống được thiết kế theo hướng mở, linh hoạt. Điều này cho phép hệ thống dễ dàng mở rộng, thay đổi hoặc kết hợp các phương pháp phân tích khi cần thiết, đồng thời đảm bảo phù hợp cho cả mục tiêu nghiên cứu lẫn ứng dụng thực tiễn trong phân tích tình trạng giao thông đô thị.

7.2 Yêu cầu

7.2.1 Yêu cầu chức năng

7.2.1.1 Tiền xử lý dữ liệu

- **FR1.1:** Loại bỏ dữ liệu thiếu, sai định dạng hoặc bất thường.
- **FR1.2:** Chuẩn hoá các thuộc tính như tốc độ, thời gian di chuyển, timestamp.
- **FR1.3:** Ghép nối dữ liệu với mạng lưới đường (map-matching) khi cần.

7.2.1.2 Phân tích mạng lưới giao thông

- **FR2.1:** Xây dựng mô hình đồ thị gồm các nút (node) và cạnh (segment).
- **FR2.2:** Tính toán các thông số của mạng đường: độ dài, phân cấp đường (FRC), các chỉ số trung tâm.
- **FR2.3:** Xác định “điểm nóng” (hotspot) tắc nghẽn theo thời gian.

7.2.1.3 Phân tích mối liên hệ giữa các đoạn đường

- **FR3.1:** Tính toán mức độ tương quan giữa các đoạn đường theo thời gian.
- **FR3.2:** Đo độ tương đồng dạng thời gian bằng kỹ thuật Dynamic Time Warping (DTW).
- **FR3.3:** Phát hiện hiện tượng lan truyền tắc nghẽn trong mạng lưới.

7.2.1.4 Dự báo lan truyền ùn tắc giao thông

- **FR4.1:** Xây dựng tập dữ liệu theo chuỗi thời gian kết hợp với cấu trúc mạng lưới giao thông nhằm mô hình hóa sự lan truyền của ùn tắc giữa các nút và đoạn đường.
- **FR4.2:** Áp dụng và huấn luyện các mô hình dự báo lan truyền ùn tắc giao thông trên mạng lưới giao thông.
- **FR4.3:** Đánh giá hiệu quả và độ chính xác của mô hình dự báo lan truyền thông qua các thước đo như MAE, RMSE, đồng thời phân tích khả năng phản ánh xu hướng lan truyền theo không gian và thời gian.

7.2.1.5 Trực quan hoá dữ liệu

- **FR5.1:** Hiển thị bản đồ mạng giao thông bằng dữ liệu OSM hoặc TomTom.
- **FR5.2:** Trực quan tốc độ trung bình theo thời gian của từng segment.
- **FR5.3:** Hiển thị biểu đồ chuỗi thời gian.
- **FR5.4:** Hiển thị ma trận tương quan giữa các đoạn đường.
- **FR5.5:** Cho phép thao tác xem dữ liệu trên dashboard.

7.2.1.6 Xuất báo cáo

- **FR6.1:** Xuất kết quả phân tích ra các định dạng CSV, JSON hoặc PDF.
- **FR6.2:** Cho phép tải xuống kết quả.

7.2.2 Yêu cầu phi chức năng

7.2.2.1 Khả năng mở rộng

- Hệ thống phải có khả năng xử lý dữ liệu lớn lên đến hàng triệu bản ghi, đồng thời có thể mở rộng mà không suy giảm hiệu năng.

7.2.2.2 Hiệu năng

- Thời gian xử lý dữ liệu phải đảm bảo hiệu quả.
- Thời gian tải và hiển thị biểu đồ phải dưới 5 giây.

7.2.2.3 Độ tin cậy

- Hệ thống phải hoạt động ổn định và cho kết quả nhất quán khi sử dụng cùng một bộ dữ liệu.

7.2.2.4 Tính bảo trì

- Mã nguồn phải được tổ chức thành các module độc lập gồm: tiền xử lý, phân tích, mô hình hoá và trực quan hoá.

7.2.2.5 Tính tương thích

- Hệ thống phải hoạt động tốt với Python 3.10+, các thư viện phân tích (Pandas, GeoPandas, NetworkX), TomTom MOVE và dữ liệu OSM.

7.2.2.6 Tính dễ sử dụng

- Giao diện và quy trình thao tác phải đơn giản, giúp người dùng dễ dàng tải dữ liệu, xem biểu đồ và tạo báo cáo.

7.2.2.7 An toàn dữ liệu

- Không thu thập hoặc xử lý dữ liệu định danh cá nhân. Dữ liệu giao thông chỉ được xử lý ở mức tổng hợp.

7.2.3 Yêu cầu dữ liệu

7.2.3.a Loại dữ liệu cần thu thập

Hệ thống cần dữ liệu mô tả tình trạng giao thông đô thị có thể từ một hoặc nhiều nguồn khác nhau (API giao thông, cảm biến, dữ liệu mở, dữ liệu bản đồ số, v.v.). Các nhóm dữ liệu chính bao gồm:

- **Dữ liệu cấu trúc mạng lưới giao thông:**
 - Thông tin các đoạn đường (segment).
 - Mã định danh đoạn đường.
 - Tên đường, khu vực, vùng hoặc zone.

- Thông tin bản đồ nền và phiên bản bản đồ.
- **Dữ liệu thời gian:**
 - Thời điểm thu thập dữ liệu.
 - Khoảng ngày quan sát.
 - Khoảng thời gian trong ngày.
- **Dữ liệu tình trạng giao thông:**
 - Tốc độ trung bình, tốc độ tự do.
 - Mức độ chậm trễ, chỉ số ùn tắc.
 - Điều kiện lưu thông trên từng đoạn đường.
 - Nguồn thu thập (GPS, cảm biến, v.v.) nếu có.
- **Thông tin cấu hình và siêu dữ liệu:**
 - Tên phiên thu thập hoặc mã job.
 - Đơn vị đo vận tốc.
 - Các lựa chọn cấu hình do người dùng đặt.
 - Các tham số được sử dụng để tạo bộ dữ liệu.

7.2.3.b Yêu cầu về định dạng và lưu trữ dữ liệu

- Dữ liệu đầu vào có thể ở dạng JSON, CSV hoặc các định dạng bán cấu trúc khác, tùy theo nguồn thu thập.
- Hệ thống cần hỗ trợ chuyển đổi sang dạng bảng (tabular) để phân tích.
- Dữ liệu được lưu trữ ở các định dạng thuận tiện cho xử lý như CSV, Parquet...
- Hỗ trợ nhập dữ liệu từ nhiều nguồn: file tải lên, API hoặc kho lưu trữ trên nền tảng đám mây.

7.2.3.c Yêu cầu về chất lượng dữ liệu

- **Đầy đủ:** có đủ thông tin tối thiểu để phân tích tình trạng giao thông.
- **Nhất quán:** đồng nhất về đơn vị đo, định dạng thời gian và mã định danh.
- **Đúng cấu trúc:** các trường phải đúng kiểu dữ liệu.

- **Hạn chế dữ liệu lỗi hoặc thiếu:** hạn chế phần tử bị trống hoặc sai lệch.
- **Có thể truy vết:** biết rõ nguồn và thời điểm thu thập.

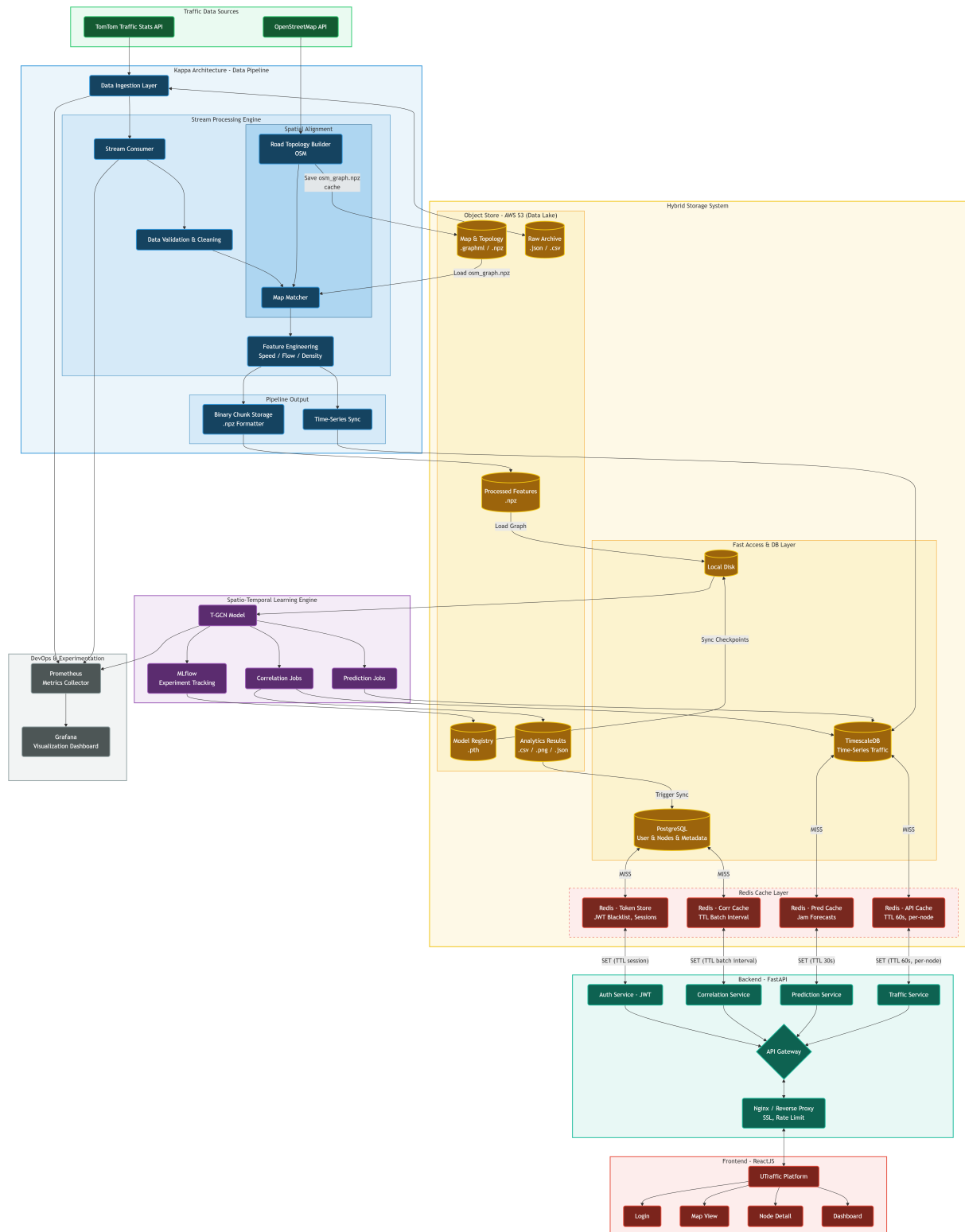
7.2.3.d Yêu cầu về cập nhật dữ liệu

- Hệ thống cần hỗ trợ cập nhật dữ liệu từ nhiều nguồn khác nhau trong tương lai.
- Hệ thống cho phép cập nhật định kỳ khi có dữ liệu mới.

8 Phân tích & Thiết kế hệ thống

8.1 Kiến trúc tổng thể hệ thống

Hệ thống UTraffic được thiết kế theo mô hình **microservice** kết hợp với **kiến trúc Kappa** (Kappa Architecture) nhằm đảm bảo khả năng xử lý dữ liệu giao thông thời gian thực, đồng thời hỗ trợ các tác vụ phân tích và dự báo phức tạp dựa trên mô hình học sâu trên đồ thị (T-GCN). Toàn bộ hệ thống được tổ chức thành 6 tầng chức năng độc lập nhưng liên kết chặt chẽ với nhau, bao gồm: tầng thu thập dữ liệu, tầng xử lý luồng, tầng lưu trữ lai ghép, tầng học máy không gian - thời gian, tầng phục vụ và tầng giám sát.



Hình 15: Kiến trúc tổng thể hệ thống UTraffic

8.2 Tầng thu thập dữ liệu (Traffic Data Sources)

Hệ thống tiếp nhận dữ liệu giao thông từ hai nguồn chính:

- **TomTom Traffic Stats API:** Cung cấp dữ liệu tốc độ thực tế theo từng phân đoạn đường (road segment) với độ phân giải thời gian 15 phút. Dữ liệu bao gồm các thuộc tính: tốc độ trung bình (`average_speed`), tốc độ điều hòa (`harmonic_average_speed`), tốc độ trung vị (`median_speed`), thời gian di chuyển (`average_travel_time`), tỉ lệ thời gian di chuyển so với điều kiện lý tưởng (`travel_time_ratio`), và kích thước mẫu (`sample_size`). Dữ liệu được thu thập theo mô hình batch job từ TomTom Movement Stats API, bao gồm 31 ngày $\times$ 24 khung giờ 15 phút/ngày cho các phân đoạn đường thuộc khu vực nghiên cứu tại TP.HCM (bounding box: lat 10.760–10.805, lon 106.675–106.715).
- **OpenStreetMap API (OSMnx):** Cung cấp dữ liệu topo học đường bộ dưới dạng đồ thị định hướng, bao gồm các node (giao lộ) với tọa độ địa lý và các edge (đoạn đường) với các thuộc tính như chiều dài, tốc độ giới hạn, số làn xe, và loại đường (`highway type`). Dữ liệu OSM được lưu cache dưới định dạng `.graphml` để tránh tải lại mỗi lần chạy pipeline.

8.3 Tầng xử lý luồng (Kappa Architecture – Stream Processing Engine)

Kiến trúc Kappa được lựa chọn thay cho kiến trúc Lambda nhằm đơn giản hóa hệ thống bằng cách xử lý tất cả dữ liệu thông qua một luồng duy nhất, loại bỏ sự phức tạp của việc duy trì song song hai lớp batch và streaming. Tầng xử lý bao gồm các thành phần sau:

8.3.1 Data Ingestion Layer

Là điểm tiếp nhận dữ liệu từ các nguồn bên ngoài vào hệ thống. Thành phần này chịu trách nhiệm xác thực định dạng đầu vào và phân phối dữ liệu thô vào hàng đợi xử lý. Trong giai đoạn triển khai hiện tại, Data Ingestion Layer hoạt động theo mô hình kéo dữ liệu định kỳ (scheduled pull) từ TomTom API thay vì streaming thời gian thực do giới hạn của API phía nhà cung cấp.

8.3.2 Stream Consumer & Data Validation

Stream Consumer tiêu thụ dữ liệu từ hàng đợi và chuyển tiếp sang module Data Validation & Cleaning. Tại đây, dữ liệu được kiểm tra tính toàn vẹn (null check, range check cho các giá trị tốc độ), loại bỏ các bản ghi có `sample_size = 0` (không có xe đi qua), và chuẩn hóa định dạng timestamp theo múi giờ Asia/Ho_Chi_Minh.

8.3.3 Road Topology Builder (OSM) & Map Matcher

Module Road Topology Builder sử dụng thư viện **OSMnx** để xây dựng đồ thị đường bộ từ OpenStreetMap. Đồ thị này sau đó được lưu cache dưới dạng **.npz** để phục vụ các bước xử lý tiếp theo.

Map Matcher là module trọng tâm của pipeline, thực hiện gán (matching) các phân đoạn TomTom vào các cạnh OSM tương ứng thông qua thuật toán **shortest-path map matching**. Với mỗi phân đoạn TomTom, hệ thống xác định node OSM gần nhất với điểm đầu và điểm cuối của phân đoạn, sau đó tính đường đi ngắn nhất giữa hai node đó. Tất cả các cạnh OSM trên đường đi này đều nhận thuộc tính giao thông của phân đoạn TomTom tương ứng. Thiết kế này giải quyết vấn đề không khớp tỉ lệ giữa phân đoạn TomTom (thường dài 200–500m) và cạnh OSM (thường dài 30–100m). Kết quả là một **subgraph** chỉ bao gồm các cạnh OSM đã được xác nhận có dữ liệu TomTom tương ứng, đảm bảo 100% dữ liệu là thực tế (không có giá trị mặc định giả tạo).

8.3.4 Feature Engineering

Module này tính toán các đặc trưng bổ sung từ dữ liệu thô:

- **Congestion Index:** $CI = 1 - \frac{v_{observed}}{v_{limit}}$, với giá trị từ 0 (thông thoáng) đến 1 (tắc hoàn toàn).
- **Speed Limit Ratio:** $SLR = \frac{v_{observed}}{v_{limit}}$, biểu diễn mức độ sử dụng năng lực đường.
- **Std Speed:** Độ lệch chuẩn tốc độ trong khung giờ, phản ánh mức độ ổn định của dòng chảy giao thông.
- **Node-level aggregation:** Đặc trưng topo học của mỗi node (bậc đỉnh, betweenness centrality, tọa độ chuẩn hóa) được tính từ subgraph sau map matching.

8.3.5 Binary .npz Formatter & Time-Series Sync

Dữ liệu sau xử lý được lưu dưới hai định dạng song song:

- **Binary .npz:** Lưu trữ tensor đặc trưng dạng (T, N, F) cho T-GCN, trong đó T là số bước thời gian, $N = 385$ là số node trong subgraph, và F là số đặc trưng (9 đặc trưng TomTom + 4 đặc trưng topo học OSM). Định dạng này được tối ưu cho việc nạp dữ liệu vào mô hình học máy.
- **Time-Series Sync:** Ghi dữ liệu quan sát theo thời gian vào TimescaleDB thông qua batch insert, phục vụ truy vấn lịch sử và tính toán aggregate.

8.4 Tầng lưu trữ lai ghép (Hybrid Storage System)

Hệ thống lưu trữ được thiết kế theo mô hình lai ghép nhằm cân bằng giữa hiệu năng truy xuất, chi phí lưu trữ, và khả năng mở rộng:

8.4.1 Object Store – AWS S3 (Data Lake)

Lưu trữ toàn bộ dữ liệu thô và đã xử lý dưới dạng file nhị phân:

- **Raw Archive (.json/.csv):** Dữ liệu gốc từ TomTom API, lưu nguyên bản để phục vụ tái xử lý khi cần.
- **Map & Topology (.graphml/.npz):** Đồ thị OSM và kết quả map matching.
- **Processed Features (.npz):** Tensor đặc trưng đã xử lý, sẵn sàng cho huấn luyện mô hình.
- **Model Registry (.pth):** Checkpoint của các mô hình T-GCN và DTC-STGCN đã huấn luyện.
- **Analytics Results (.csv/.png/.json):** Kết quả phân tích tương quan, heatmap, và báo cáo so sánh mô hình.

8.4.2 Fast Access & DB Layer

- **Local Disk Cache:** Lưu cache các file .npz thường xuyên truy cập (graph structure, model weights, correlation matrix) trên ổ SSD cục bộ để giảm độ trễ khi inference.
- **TimescaleDB – Time-Series Traffic:** Mở rộng của PostgreSQL được tối ưu cho dữ liệu chuỗi thời gian. Bảng `traffic_observations` được cấu hình là hypertable với chunk theo tuần, cho phép truy vấn dữ liệu lịch sử hiệu quả thông qua continuous aggregates.
- **PostgreSQL – User & Nodes & Metadata:** Lưu trữ dữ liệu có cấu trúc bền vững: thông tin người dùng, metadata node/edge của đồ thị đường bộ, kết quả tương quan, và phiên bản mô hình. Tích hợp extension **PostGIS** để hỗ trợ truy vấn không gian địa lý.

8.4.3 Redis Cache Layer

Bốn instance Redis chuyên biệt phục vụ các mục đích khác nhau:

- **Redis-Auth (Token Store):** Lưu JWT blacklist và session metadata. TTL được đặt bằng thời gian hết hạn của token.
- **Redis-Corr (Correlation Cache):** Cache kết quả tương quan top-20 láng giềng của từng node dưới dạng JSON blob. Kích thước cache ước tính ~ 0.63 MB cho toàn bộ 385 node. TTL được cập nhật theo batch interval khi có correlation job mới.
- **Redis-Pred (Prediction Cache):** Cache dự báo tắc nghẽn cho 4 horizon tiếp theo của từng node. TTL 30 giây để đảm bảo dữ liệu luôn gần với thời gian thực.
- **Redis-API (API Cache):** Cache response của các API endpoint thường xuyên được gọi (danh sách node, thông tin đồ thị). TTL 60 giây.

8.5 Tầng học máy không gian – thời gian (Spatio-Temporal Learning Engine)

8.5.1 Mô hình T-GCN

Mô hình **Temporal Graph Convolutional Network (T-GCN)** là thành phần cốt lõi của tầng ML. Kiến trúc T-GCN kết hợp **Graph Convolutional Network (GCN)** để học mối phụ thuộc không gian giữa các node với **Gated Recurrent Unit (GRU)** để mô hình hóa sự phụ thuộc thời gian trong chuỗi tốc độ. Cụ thể:

- **Encoder:** Xử lý chuỗi lịch sử $seq_len = 12$ bước thời gian (tương ứng 3 giờ với bước 15 phút). Tại mỗi bước, TGCNCell thực hiện: (1) GCN tổng hợp thông tin từ các node láng giềng theo ma trận kề chuẩn hóa $\hat{A} = D^{-1/2}AD^{-1/2}$; (2) GRUCell cập nhật trạng thái ẩn; (3) LayerNorm ổn định gradient.
- **Decoder:** Sinh dự báo tự hồi quy (auto-regressive) cho $pred_len = 4$ bước tiếp theo (tương ứng 60 phút), sử dụng giá trị dự báo của bước trước làm đầu vào cho bước tiếp theo.
- **Output:** Tensor dự báo tốc độ có shape $(batch, 4, 385, 1)$, trong đó 4 horizon tương ứng với các mốc dự báo T+15, T+30, T+45, T+60 phút.

Sau quá trình thực nghiệm và so sánh với các baseline (Historical Average, GRU, SVR, DTC-STGCN), mô hình T-GCN đạt kết quả tốt nhất với $RMSE = 0.443$ và $R^2 = 0.797$ tại horizon 1 bước (15 phút).

8.5.2 Correlation Jobs

Module phân tích tương quan sử dụng class **TGCNCorrelationExtractor** để trích xuất ma trận tương quan nút–nút từ mô hình T-GCN đã huấn luyện. Phương pháp **detrended Pearson correlation** được áp dụng trên các chuỗi dự báo tốc độ (speed predictions) thay vì trên hidden states, nhằm tránh hiện tượng tương quan giả (spurious correlation) do LayerNorm gây ra. Kết quả là ma trận 385×385 với giá trị trong khoảng $[-1, 1]$, được lưu vào PostgreSQL (các cặp có $|corr| \geq 0.7$) và file **.npz** (ma trận đầy đủ).

8.5.3 Prediction Jobs

Chạy định kỳ mỗi 15 phút để sinh dự báo cho 4 horizon tiếp theo. Kết quả dự báo được ghi đồng thời vào PostgreSQL (bảng **predictions**) và Redis-Pred cache để phục vụ API.

8.5.4 MLflow Experiment Tracking

Toàn bộ quá trình huấn luyện và đánh giá mô hình được theo dõi bằng MLflow, bao gồm: hyperparameter, metrics theo từng epoch, checkpoint path, và kết quả so sánh giữa các mô hình.

8.6 Tầng phục vụ (Serving Layer)

8.6.1 Backend – FastAPI

Backend được xây dựng bằng **FastAPI** theo mô hình microservice với 4 service độc lập:

- **Auth Service – JWT:** Quản lý đăng ký, đăng nhập, làm mới token (refresh token rotation), và thu hồi token. Sử dụng Redis-Auth để blacklist JWT khi người dùng đăng xuất.
- **Correlation Service:** Cung cấp API trả về top-K node tương quan với một node cho trước. Ưu tiên đọc từ Redis-Corr cache; trong trường hợp cache miss mới truy vấn PostgreSQL.
- **Prediction Service:** Cung cấp API trả về dự báo tắc nghẽn theo horizon cho một hoặc nhiều node. Kết hợp dữ liệu từ Redis-Pred và PostgreSQL.
- **Traffic Service:** Cung cấp API phục vụ bản đồ Folium, bao gồm: truy vấn node/edge theo bounding box (sử dụng PostGIS), thống kê tốc độ theo giờ, và dữ liệu topo học đồ thị.

Tất cả các service đều đi qua **API Gateway** để xử lý xác thực, giới hạn tốc độ (rate limiting), và định tuyến request.

8.6.2 Nginx / Reverse Proxy

Nginx đóng vai trò reverse proxy và load balancer, xử lý SSL termination và rate limiting ở tầng mạng. Cấu hình gzip compression cho response JSON nhằm giảm băng thông khi trả về dữ liệu bản đồ.

8.6.3 Frontend – ReactJS (UTraffic Platform)

Giao diện người dùng được xây dựng bằng **ReactJS** với 4 màn hình chính:

- **Login:** Xác thực người dùng với JWT, hỗ trợ remember me thông qua refresh token.
- **Map View:** Hiển thị bản đồ giao thông tương tác sử dụng **Folium** (Python) để sinh tile GeoJSON, kết hợp với Leaflet.js trên frontend. Áp dụng kỹ thuật **lazy loading theo bounding box**: chỉ tải dữ liệu node/edge trong viewport hiện tại, tự động cập nhật khi người dùng pan hoặc zoom bản đồ.
- **Node Detail:** Khi người dùng click vào một node giao thông, hệ thống hiển thị overlay màu sắc biểu diễn mức độ tương quan của node đó với tất cả các node còn lại (đỏ = tương quan cao, xanh = thấp), cùng với biểu đồ dự báo tốc độ 4 horizon tiếp theo.
- **Dashboard:** Tổng quan hệ thống với các thống kê tổng hợp: phân bố tốc độ theo giờ, heat map tắc nghẽn theo thời gian, và danh sách top node có mức độ tương quan cao nhất.

8.7 Tầng giám sát – DevOps (Monitoring & Experimentation)

- **Prometheus Metrics Collector:** Thu thập các chỉ số hệ thống (CPU, memory, request latency, cache hit rate) từ tất cả các service.
- **Grafana Visualization Dashboard:** Trực quan hóa các chỉ số Prometheus theo thời gian thực, hỗ trợ cài đặt cảnh báo (alert) khi các chỉ số vượt ngưỡng.

8.8 Luồng dữ liệu tổng thể

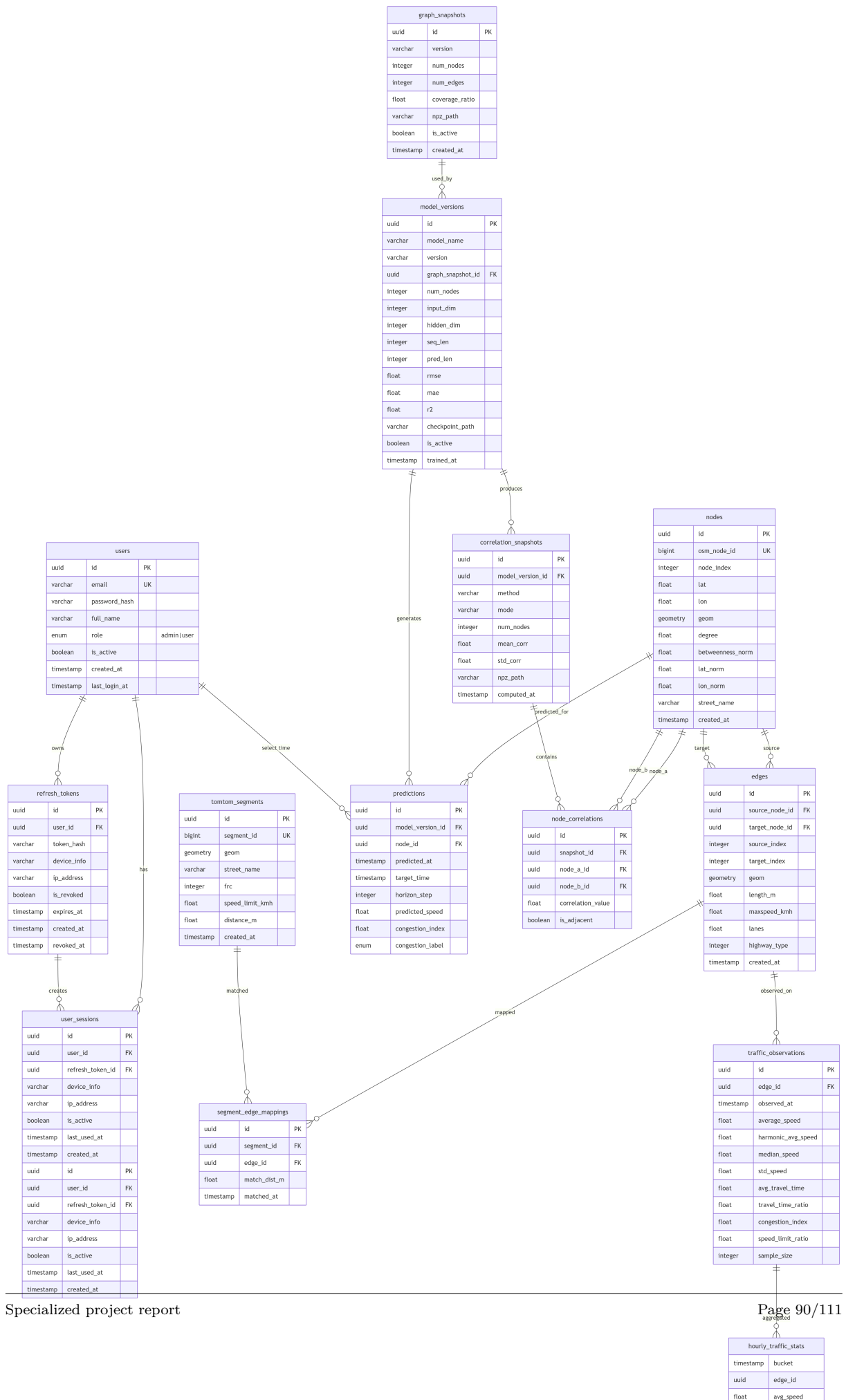
Luồng dữ liệu chính của hệ thống có thể tóm tắt qua các bước sau:

1. TomTom API và OSM API cung cấp dữ liệu thô cho Data Ingestion Layer.
2. Pipeline Kappa xử lý, làm sạch, map-match và trích xuất đặc trưng, lưu kết quả vào S3 (.npz) và TimescaleDB.

3. T-GCN đọc dữ liệu từ Local Disk Cache (.npz), thực hiện huấn luyện và inference, ghi kết quả vào PostgreSQL và Redis.
4. FastAPI Backend đọc từ Redis (cache hit) hoặc PostgreSQL/TimescaleDB (cache miss), trả về response cho Frontend.
5. ReactJS Frontend hiển thị bản đồ Folium với lazy loading và overlay tương quan theo yêu cầu người dùng.

8.9 Thiết kế cơ sở dữ liệu

Hệ thống UTraffic sử dụng **PostgreSQL** (kết hợp extension **PostGIS** và **TimescaleDB**) làm cơ sở dữ liệu chính. Thiết kế gồm 14 bảng được tổ chức thành 5 nhóm chức năng: xác thực người dùng, topo học đồ thị đường bộ, dữ liệu giao thông thời gian thực, học máy và phiên bản mô hình, phân tích tương quan. Sơ đồ ERD tổng thể được trình bày trong Hình 16.



8.9.1 Nhóm bảng xác thực người dùng (Authentication)

8.9.1.1 Bảng users Lưu trữ thông tin tài khoản người dùng hệ thống. Hệ thống hỗ trợ hai vai trò: **admin** (quản trị viên, có quyền xem toàn bộ dữ liệu phân tích và quản lý mô hình) và **user** (người dùng thông thường, có quyền xem bản đồ và kết quả tương quan).

Bảng 7: Cấu trúc bảng *users*

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	Định danh duy nhất
email	VARCHAR	UK, NOT NULL	Email đăng nhập
password_hash	VARCHAR	NOT NULL	Mật khẩu băm (bcrypt)
full_name	VARCHAR		Tên hiển thị
role	VARCHAR	NOT NULL	Vai trò: admin hoặc user
is_active	BOOLEAN	DEFAULT TRUE	Trạng thái tài khoản
created_at	TIMESTAMP	NOT NULL	Thời điểm tạo tài khoản
last_login_at	TIMESTAMP		Lần đăng nhập cuối

8.9.1.2 Bảng refresh_tokens Quản lý refresh token theo cơ chế **Refresh Token Rotation**: mỗi lần làm mới access token, refresh token cũ bị thu hồi (**is_revoked = true**) và một refresh token mới được cấp phát. Cơ chế này phát hiện tấn công replay attack khi một token cũ bị sử dụng lại.

Bảng 8: Cấu trúc bảng *refresh_tokens*

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	Định danh duy nhất
user_id	UUID	FK → users	Liên kết người dùng
token_hash	VARCHAR	NOT NULL	Hash của refresh token
device_info	VARCHAR		Thông tin thiết bị
ip_address	VARCHAR		Địa chỉ IP đăng nhập
is_revoked	BOOLEAN	DEFAULT FALSE	Đã thu hồi hay chưa
expires_at	TIMESTAMP	NOT NULL	Thời điểm hết hạn
created_at	TIMESTAMP	NOT NULL	Thời điểm cấp phát
revoked_at	TIMESTAMP		Thời điểm thu hồi

8.9.1.3 Bảng user_sessions Theo dõi phiên hoạt động của người dùng trên từng thiết bị. Cho phép admin xem danh sách thiết bị đang đăng nhập và thu hồi phiên từ xa khi cần.

8.9.2 Nhóm bảng topo học đồ thị đường bộ (Road Graph)

Nhóm này lưu trữ cấu trúc đồ thị đường bộ sau quá trình map matching giữa OSM và TomTom. Đây là nhóm bảng trung tâm, được tham chiếu bởi hầu hết các nhóm còn lại.

8.9.2.1 Bảng nodes Mỗi node tương ứng với một giao lộ (ngã ba, ngã tư, v.v.) trong subgraph đã được map match. Cột `node_index` (0–384) là chỉ số số nguyên tương ứng với vị trí của node trong tensor T-GCN, đảm bảo căn chỉnh (alignment) chính xác giữa cơ sở dữ liệu và mô hình học máy.

Bảng 9: Cấu trúc bảng *nodes*

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>id</code>	UUID	PK	Định danh nội bộ
<code>osm_node_id</code>	BIGINT	UK, NOT NULL	ID node trong OSM
<code>node_index</code>	INTEGER	NOT NULL	Chỉ số tensor (0–384)
<code>lat</code>	FLOAT	NOT NULL	Vĩ độ
<code>lon</code>	FLOAT	NOT NULL	Kinh độ
<code>geom</code>	GEOMETRY(POINT)	NOT NULL	Điểm địa lý (PostGIS)
<code>degree</code>	FLOAT		Bậc đỉnh trong subgraph
<code>betweenness_norm</code>	FLOAT		Betweenness centrality
<code>lat_norm</code>	FLOAT		Vĩ độ chuẩn hóa [0,1]
<code>lon_norm</code>	FLOAT		Kinh độ chuẩn hóa [0,1]
<code>street_name</code>	VARCHAR		Tên đường gần nhất
<code>created_at</code>	TIMESTAMP	NOT NULL	Thời điểm tạo bản ghi

Index: Bắt buộc tạo GIST index trên cột `geom` để hỗ trợ truy vấn bounding box từ Folium lazy loading: `CREATE INDEX ON nodes USING GIST (geom)`.

8.9.2.2 Bảng edges Mỗi edge là một cạnh có hướng trong subgraph, tương ứng với một đoạn đường một chiều trong OSM. Một tuyến đường hai chiều được biểu diễn bằng hai edge ngược chiều nhau.

Bảng 10: Cấu trúc bảng *edges*

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>id</code>	UUID	PK	Định danh duy nhất
<code>source_node_id</code>	UUID	FK → nodes	Node đầu cạnh
<code>target_node_id</code>	UUID	FK → nodes	Node cuối cạnh
<code>source_index</code>	INTEGER	NOT NULL	Chỉ số tensor node đầu
<code>target_index</code>	INTEGER	NOT NULL	Chỉ số tensor node cuối
<code>geom</code>	GEOMETRY(LINESTRING)	NOT NULL	Hình học cạnh (PostGIS)
<code>length_m</code>	FLOAT		Chiều dài (mét)
<code>maxspeed_kmh</code>	FLOAT		Tốc độ giới hạn (km/h)
<code>lanes</code>	FLOAT		Số làn xe
<code>highway_type</code>	INTEGER		Phân loại đường OSM
<code>created_at</code>	TIMESTAMP	NOT NULL	Thời điểm tạo bản ghi

8.9.2.3 Bảng `tomtom_segments` và `segment_edge_mappings` Bảng `tomtom_segments` lưu thông tin gốc của phân đoạn TomTom (ID, hình học LINESTRING,

tên đường, FRC, tốc độ giới hạn, chiều dài). Bảng `segment_edge_mappings` là bảng junction thể hiện mối quan hệ nhiều-nhiều giữa phân đoạn TomTom và cạnh OSM: một phân đoạn TomTom dài có thể được map sang nhiều cạnh OSM, và một cạnh OSM có thể nhận dữ liệu từ nhiều phân đoạn TomTom lân cận. Cột `match_dist_m` ghi khoảng cách snap (mét) để đánh giá chất lượng map matching.

8.9.2.4 Bảng `graph_snapshots` Lưu phiên bản hóa cấu trúc đồ thị. Mỗi khi pipeline map matching được chạy lại (thêm dữ liệu mới hoặc cập nhật OSM), một snapshot mới được tạo. Cột `is_active` xác định phiên bản đồ thị đang được sử dụng bởi mô hình hiện hành. Cột `npz_path` trỏ tới file `.npz` trên S3 hoặc Local Disk chứa ma trận kề và đặc trưng node.

8.9.3 Nhóm bảng dữ liệu giao thông (Traffic Data)

8.9.3.1 Bảng `traffic_observations` Đây là bảng lớn nhất và quan trọng nhất về mặt dữ liệu, được cấu hình là **TimescaleDB hypertable** phân vùng theo cột `observed_at` với kích thước chunk 1 tuần. Mỗi bản ghi lưu trạng thái giao thông tại một cạnh trong một khung giờ 15 phút.

Bảng 11: Cấu trúc bảng `traffic_observations` (TimescaleDB hypertable)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>id</code>	UUID	PK	Định danh duy nhất
<code>edge_id</code>	UUID	FK → edges	Cạnh được quan sát
<code>observed_at</code>	TIMESTAMP	NOT NULL	Thời điểm quan sát
<code>average_speed</code>	FLOAT		Tốc độ trung bình (km/h)
<code>harmonic_avg_speed</code>	FLOAT		Tốc độ điều hòa (km/h)
<code>median_speed</code>	FLOAT		Tốc độ trung vị (km/h)
<code>std_speed</code>	FLOAT		Độ lệch chuẩn tốc độ
<code>avg_travel_time</code>	FLOAT		Thời gian di chuyển TB (s)
<code>travel_time_ratio</code>	FLOAT		Tỉ lệ thời gian/lý tưởng
<code>congestion_index</code>	FLOAT		Chỉ số tắc nghẽn [0,1]
<code>speed_limit_ratio</code>	FLOAT		Tỉ lệ tốc độ/giới hạn
<code>sample_size</code>	INTEGER		Số lượng mẫu xe

Lưu ý triển khai: Sau khi tạo bảng, cần chuyển sang hypertable với câu lệnh:

```
SELECT create_hypertable('traffic_observations', 'observed_at',
    chunk_time_interval => INTERVAL '1 week');
```

Tốc độ ghi ước tính: với $385 \text{ node} \times 24 \text{ khung giờ/ngày} \approx 9.240 \text{ bản ghi/ngày}$ (ở level edge sau map matching còn nhiều hơn). TimescaleDB continuous

aggregate được định nghĩa cho bảng `hourly_traffic_stats` để tính toán thống kê theo giờ mà không cần query toàn bộ bảng gốc.

8.9.3.2 Bảng `hourly_traffic_stats` Bảng aggregate tổng hợp theo giờ, được duy trì tự động bởi TimescaleDB continuous aggregate policy. Khóa chính composite là `(bucket, edge_id)`, trong đó `bucket` là timestamp đã làm tròn theo giờ bằng hàm `time_bucket('1 hour', observed_at)`.

8.9.4 Nhóm bảng học máy và phiên bản mô hình (ML & Model Versioning)

8.9.4.1 Bảng `model_versions` Lưu thông tin về các phiên bản mô hình đã được huấn luyện và triển khai trong hệ thống. Mỗi bản ghi tương ứng với một phiên bản mô hình cụ thể, gắn với một snapshot đồ thị qua cột `graph_snapshot_id`. Khác với thiết kế cũ, các chỉ số đánh giá tổng hợp của mô hình như `rmse`, `mae`, `r2` được lưu trực tiếp trong bảng này, thay vì gom trong cột JSONB `metrics_by_horizon`. Bảng cũng lưu các siêu tham số chính phục vụ tái lập huấn luyện và suy luận như `num_nodes`, `input_dim`, `hidden_dim`, `seq_len`, `pred_len`. Cột `checkpoint_path` trỏ tới tệp checkpoint của mô hình đã huấn luyện trên Object Storage hoặc Local Disk. Tại một thời điểm, chỉ một bản ghi có `is_active = true` được chọn để phục vụ inference.

Bảng 12: Cấu trúc bảng *model_versions*

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	Định danh duy nhất của phiên bản mô hình
model_name	VARCHAR	NOT NULL	Tên mô hình
version	VARCHAR	NOT NULL	Nhãn phiên bản mô hình
graph_snapshot_id	UUID	FK → graph_snapshots	Snapshot đồ thị được dùng để huấn luyện mô hình
num_nodes	INTEGER		Số nút đầu vào của đồ thị
input_dim	INTEGER		Số chiều đặc trưng đầu vào
hidden_dim	INTEGER		Số chiều lớp ẩn của mô hình
seq_len	INTEGER		Độ dài chuỗi đầu vào
pred_len	INTEGER		Số bước dự báo đầu ra
rmse	FLOAT		Sai số RMSE tổng hợp của mô hình
mae	FLOAT		Sai số MAE tổng hợp của mô hình
r2	FLOAT		Hệ số xác định R^2
checkpoint_path	VARCHAR		Đường dẫn tới tệp checkpoint mô hình
is_active	BOOLEAN		Đánh dấu phiên bản đang được dùng cho suy luận
trained_at	TIMESTAMP		Thời điểm hoàn tất huấn luyện

Kết quả thực nghiệm từ `comparison_results.json` cho thấy T-GCN đạt hiệu năng vượt trội so với các baseline tại tất cả 4 horizon, cụ thể tại horizon 1: $RMSE = 0.443$, $MAE = 0.337$, $R^2 = 0.797$; và tại horizon 4: $RMSE = 0.560$, $MAE = 0.406$, $R^2 = 0.725$.

8.9.4.2 Bảng predictions Lưu kết quả dự báo tốc độ từ T-GCN. Mỗi lần chạy prediction job sinh ra $385 \times 4 = 1.540$ bản ghi ($385 \text{ node} \times 4 \text{ horizon}$). Với tần suất 15 phút/lần, bảng này tích lũy khoảng 147.840 bản ghi/ngày. Để kiểm soát kích thước, bảng này cần được cấu hình là **TimescaleDB hypertable** với TTL policy tự động xóa dữ liệu cũ hơn 30 ngày.

Bảng 13: Cấu trúc bảng *predictions*

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	Định danh duy nhất
model_version_id	UUID	FK → model_versions	Mô hình sinh dự báo
node_id	UUID	FK → nodes	Node được dự báo
predicted_at	TIMESTAMP	NOT NULL	Thời điểm chạy inference
target_time	TIMESTAMP	NOT NULL	Thời điểm dự báo tới
horizon_step	INTEGER	NOT NULL	Bước dự báo (1–4)
predicted_speed	FLOAT		Tốc độ dự báo (km/h)
congestion_index	FLOAT		Chỉ số tắc nghẽn dự báo
congestion_label	VARCHAR		Nhãn: free/slow/congested

Index: Cần tạo composite index (node_id, predicted_at, horizon_step) để tối ưu truy vấn từ Prediction Service.

8.9.5 Nhóm bảng phân tích tương quan (Correlation Analysis)

Đây là nhóm bảng phục vụ tính năng cốt lõi của hệ thống: hiển thị mối tương quan giữa các nút giao thông khi người dùng click vào bản đồ.

8.9.5.1 Bảng correlation_snapshots Lưu metadata của mỗi lần chạy tác vụ tính tương quan trong mạng lưới giao thông. Mỗi bản ghi gắn với một phiên bản mô hình thông qua cột model_version_id, đồng thời lưu phương pháp tính tương quan tại cột method và chế độ xử lý tại cột mode. Cột num_nodes cho biết số lượng node tham gia tính toán trong snapshot đó. Các thống kê tổng hợp như mean_corr và std_corr phản ánh phân bố chung của các hệ số tương quan trong toàn mạng. Cột npz_path trỏ tới tệp nhị phân chứa ma trận tương quan hoặc dữ liệu tương quan đã được lưu trên Local Disk. Cột computed_at ghi nhận thời điểm hoàn tất quá trình tính toán snapshot.

Bảng 14: Cấu trúc bảng *correlation_snapshots*

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	Định danh duy nhất
model_version_id	UUID	FK → model_versions	Mô hình sinh dự báo
method	VARCHAR		Phương pháp tính tương quan
mode	VARCHAR		Chế độ chạy correlation job
num_nodes	INTEGER		Số lượng node được đưa vào tính toán
mean_corr	FLOAT		Giá trị trung bình của các hệ số tương quan
std_corr	FLOAT		Độ lệch chuẩn của các hệ số tương quan
npz_path	VARCHAR		Đường dẫn tới tệp lưu dữ liệu tương quan
computed_at	TIMESTAMP		Thời điểm hoàn tất tính toán snapshot

8.9.5.2 Bảng node_correlations Lưu các cặp node và hệ số tương quan tương ứng trong một snapshot cụ thể. Mỗi bản ghi thuộc về một lần tính toán correlation thông qua cột *snapshot_id*, đồng thời tham chiếu đến hai node liên quan qua *node_a_id* và *node_b_id*. Cột *correlation_value* lưu giá trị hệ số tương quan giữa hai node, còn cột *is_adjacent* cho biết hai node này có kề nhau trực tiếp trong đồ thị giao thông hay không. Khác với thiết kế cũ, bảng hiện tại không lưu các cột xếp hạng như *rank_from_a*, *rank_from_b* và cũng không lưu riêng *created_at*; các thông tin này được quy về snapshot hoặc được xử lý ở tầng truy vấn, cache và API khi cần.

Bảng 15: Cấu trúc bảng *node_correlations*

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	Định danh duy nhất
snapshot_id	UUID	FK → correlation_snapshots	Snapshot tương quan mà bản ghi này thuộc về
node_a_id	UUID	FK → nodes	Node thứ nhất
node_b_id	UUID	FK → nodes	Node thứ hai
correlation_value	FLOAT	NOT NULL	Hệ số tương quan giữa hai node
is_adjacent	BOOLEAN		Cho biết hai node có liền kề trong đồ thị hay không

Index cần thiết:

```
CREATE INDEX ON node_correlations (node_a_id, correlation_value DESC);  
CREATE INDEX ON node_correlations (node_b_id, correlation_value DESC);
```

8.9.6 Chiến lược index và tối ưu hóa

Ngoài các index đã đề cập, hệ thống áp dụng các chiến lược tối ưu sau:

- **Partial index cho correlation:** Tạo partial index trên bảng `node_correlations` chỉ cho các bản ghi thuộc snapshot đang active, giảm kích thước index khi có nhiều snapshot lịch sử.
- **BRIN index cho time-series:** Bảng `traffic_observations` sử dụng BRIN index trên cột `observed_at` thay vì B-tree do dữ liệu được ghi tuần tự theo thời gian, BRIN có kích thước nhỏ hơn đáng kể.
- **Connection pooling:** Sử dụng **PgBouncer** để quản lý connection pool giữa FastAPI services và PostgreSQL, tránh tình trạng connection exhaustion khi nhiều request đồng thời.
- **Read replica:** Trong môi trường production, TimescaleDB replica được cấu hình để tách tải đọc (analytics queries từ Dashboard) khỏi tải ghi (pipeline sync), đảm bảo hiệu năng cho cả hai luồng.

8.9.7 Tóm tắt thiết kế cơ sở dữ liệu

Bảng 16: Tổng hợp các bảng trong hệ thống *UTraffic*

Nhóm	Bảng	Chức năng chính
Authentication	<code>users</code> <code>refresh_tokens</code> <code>user_sessions</code>	Tài khoản người dùng Quản lý refresh token Theo dõi phiên đăng nhập
Road Graph	<code>nodes</code> <code>edges</code> <code>tomtom_segments</code> <code>segment_edge_mappings</code> <code>graph_snapshots</code>	Node giao lộ (OSM + PostGIS) Cạnh đường bộ có hướng Phân đoạn TomTom gốc Kết quả map matching Phiên bản hóa đồ thị
Traffic Data	<code>traffic_observations</code> <code>hourly_traffic_stats</code>	Quan sát giao thông (hypertable) Thống kê aggregate theo giờ
ML & Models	<code>model_versions</code> <code>model_horizon_metrics</code> <code>predictions</code>	Phiên bản mô hình T-GCN Metrics theo từng horizon Dự báo tốc độ (hypertable)
Correlation	<code>correlation_snapshots</code> <code>node_correlations</code> <code>node_correlation_cache</code>	Metadata phiên phân tích Ma trận tương quan (lọc) Cache top-20 JSONB per node

8.10 Thiết kế giao diện

8.10.1 Mục tiêu thiết kế

Để giao diện hệ thống đáp ứng tốt yêu cầu của bài toán phân tích tình trạng giao thông trong mạng lưới giao thông đô thị, quá trình thiết kế được xây dựng với một số mục tiêu trọng tâm như sau:

Thứ nhất, giao diện phải đảm bảo tính trực quan và dễ tiếp cận. Dữ liệu giao thông thường có khối lượng lớn, biến động theo thời gian và gắn chặt với yếu tố không gian, do đó thông tin cần được thể hiện bằng những hình thức trực quan như bản đồ, biểu đồ và các chỉ số tổng hợp.

Thứ hai, giao diện phải đảm bảo tính nhất quán trong cách trình bày. Các quy ước về màu sắc, biểu tượng, cách đặt tên chức năng, bố cục các vùng hiển thị và hành vi tương tác cần được sử dụng thống nhất trên toàn bộ hệ thống. Ví dụ, cùng một mức độ ùn tắc thì phải luôn được biểu diễn bằng cùng một nhóm màu ở tất cả các màn hình.

Thứ ba, hệ thống sẽ ưu tiên bản đồ làm phương tiện truyền đạt thông tin. Do bài toán nghiên cứu gắn với không gian đô thị, bản đồ là thành phần phản ánh trực tiếp nhất mối liên hệ giữa dữ liệu và vị trí thực tế. Từ bản đồ, người dùng có thể quan sát phân bố giao thông trên toàn thành phố, phát hiện các cụm ùn tắc và lựa chọn đối tượng để phân tích chi tiết. Các thành phần khác như bộ lọc, biểu đồ hay bảng thống kê nên đóng vai trò hỗ trợ cho bản đồ.

Thứ tư, giao diện phải hỗ trợ tốt tương tác phân tích. Người dùng cần có khả năng lọc dữ liệu theo thời gian, khu vực, đối tượng và lớp thông tin hiển thị.

Thứ năm, thiết kế giao diện phải đảm bảo tính tương thích với kiến trúc hệ thống và hiệu năng khai thác. Do dữ liệu được cung cấp từ nhiều dịch vụ backend khác nhau như dịch vụ giao thông, dịch vụ dự báo và dịch vụ tương quan, giao diện cần tổ chức sao cho việc tải dữ liệu, cập nhật trạng thái và hiển thị kết quả được diễn ra ổn định.

Cuối cùng, giao diện cần hướng tới khả năng mở rộng và bảo trì lâu dài. Kiến trúc giao diện phải cho phép bổ sung thêm các chức năng mới như cảnh báo sớm, so sánh dữ liệu nhiều thời điểm, hiển thị heatmap hoặc gợi ý phương án điều tiết giao thông mà không làm ảnh hưởng lớn đến bố cục và cách tổ chức hiện có.

8.10.2 Trang chủ

Đây là điểm vào đầu tiên của hệ thống, là màn hình sẽ xuất hiện khi người dùng truy cập. Mục đích của trang này là cung cấp cho người dùng một số thông tin cơ bản về trang web này. Cụ thể là giới thiệu ngắn gọn mục đích và một số chức năng chính.

8.10.3 Trang đăng nhập và đăng ký

Đây là hai trang thuộc nhóm giao diện xác thực người dùng của hệ thống. Hai trang này được thiết kế thống nhất về bố cục, màu sắc và cách tổ chức thành phần nhằm tạo trải nghiệm sử dụng liền mạch cho người dùng ngay từ bước đầu truy cập hệ thống.

Màn hình đăng nhập cho phép người dùng đã có tài khoản thực hiện xác thực để truy cập vào các chức năng phân tích và theo dõi giao thông. Trong khi đó, màn hình đăng ký hỗ trợ người dùng mới tạo tài khoản bằng cách cung cấp các thông tin cơ bản như email, mật khẩu và xác nhận mật khẩu. Việc thiết kế đồng bộ hai màn hình này giúp hệ thống đảm bảo đầy đủ luồng xác thực, từ khởi tạo tài khoản cho đến đăng nhập và sử dụng dịch vụ.

Đối với màn hình đăng ký, giao diện cần hỗ trợ kiểm tra tính hợp lệ của dữ liệu đầu vào nhằm hạn chế lỗi trong quá trình tạo tài khoản. Các thông báo phản hồi như thiếu thông tin, sai định dạng email, mật khẩu không đạt yêu cầu hoặc tài khoản đã tồn tại cần được hiển thị rõ ràng và trực tiếp trên giao diện. Sau khi đăng ký thành công, người dùng được điều hướng về màn hình đăng nhập để tiếp tục truy cập hệ thống.

Việc xây dựng đầy đủ nhóm giao diện đăng nhập và đăng ký không chỉ nâng cao tính hoàn chỉnh của hệ thống mà còn phù hợp với định hướng triển khai thực tế, nơi người dùng có thể chủ động tạo tài khoản và sử dụng các chức năng của nền tảng phân tích giao thông.

8.10.4 Trang dữ liệu giao thông

Trang này có chức năng hiển thị trực quan cũng như thống kê dữ liệu giao thông (sau khi đã tiền xử lý). Nó bao gồm hai giao diện con là giao diện bản đồ dữ liệu và giao diện thống kê dữ liệu. Hai giao diện này có thể được chuyển đổi qua lại thông qua một thanh điều hướng dọc ở bên trái giao diện.

Ở chế độ bản đồ dữ liệu, hệ thống hiển thị các điểm dữ liệu giao thông trên nền bản đồ số, giúp người dùng nhận biết vị trí địa lý của các nút giao, đoạn đường hoặc khu vực có dữ liệu giao thông tương ứng. Thông qua màu sắc, ký hiệu hoặc các lớp hiển thị, giao diện cho phép thể hiện các thuộc tính như tốc độ trung bình, mức độ ùn tắc tại từng vị trí trong mạng lưới.

Ở chế độ thống kê dữ liệu, hệ thống cung cấp các biểu đồ, bảng số liệu và chỉ số tổng hợp nhằm phản ánh đặc trưng của tập dữ liệu giao thông theo thời gian hoặc theo khu vực. Người dùng có thể quan sát các xu hướng biến động, so sánh các mốc thời gian khác nhau, cũng như nhận diện các giai đoạn hoặc khu vực có dấu hiệu bất thường.

8.10.5 Trang phân tích tương quan

Trang phân tích tương quan được xây dựng nhằm hỗ trợ người dùng khám phá mối liên hệ giữa các đoạn đường trong mạng lưới giao thông đô thị. Đây chính là trọng tâm, mục tiêu chính mà trang web hướng tới. Trang này giúp người dùng đánh giá hiện tượng lan truyền ùn tắc và mức độ ảnh hưởng qua lại trong mạng lưới.

Về chức năng, trang phân tích tương quan cho phép người dùng lựa chọn một đoạn đường mong muốn làm đối tượng phân tích. Việc lựa chọn có thể được thực hiện thông qua công cụ tìm kiếm theo tên tuyến đường, mã đoạn đường. Sau khi người dùng xác nhận lựa chọn, hệ thống sẽ hiển thị vị trí địa lý của đoạn đường đó lên bản đồ (ngay dưới thanh tìm kiếm) rồi gửi yêu cầu tới dịch vụ phân tích tương quan ở phía backend để thực hiện tính toán trên tập dữ liệu giao thông liên quan. Kết quả trả về được biểu diễn dưới dạng ma trận tương quan, phản ánh mức độ liên hệ giữa các đoạn đường trong tập phân tích.

Giao diện trang này có thể chia thành hai khu vực chính. Khu vực thứ nhất là vùng lựa chọn dữ liệu, nơi người dùng xác định đoạn đường cần phân tích. Khu vực thứ hai là vùng hiển thị kết quả, trong đó ma trận tương quan được thể hiện bằng bảng hoặc heatmap màu để người dùng dễ dàng nhận biết các cặp đoạn đường có mức tương quan cao, trung bình hoặc thấp.

8.10.6 Trang dự đoán lan truyền ùn tắc

Trang dự đoán lan truyền ùn tắc được xây dựng nhằm hỗ trợ người dùng đánh giá khả năng ảnh hưởng của một điểm ùn tắc ban đầu lên các đoạn đường khác trong mạng lưới giao thông.

Về chức năng, trang này cho phép người dùng lựa chọn một đoạn đường làm điểm khởi phát của tình huống ùn tắc giả định hoặc tình huống đang được quan tâm. Việc lựa chọn có thể được thực hiện thông qua ô tìm kiếm hoặc qua danh sách các đoạn đường trong khu vực nghiên cứu. Sau khi người dùng xác nhận đoạn đường cần phân tích, hệ thống sẽ hiển thị vị trí địa lý của đoạn đường đó lên bản đồ (ngay dưới thanh tìm kiếm) rồi gửi yêu cầu đến dịch vụ dự đoán ở phía backend để khai thác mô hình đã huấn luyện và thực hiện suy luận trên dữ liệu hiện có. Kết quả trả về cho biết những đoạn đường nào có khả năng bị ảnh hưởng, mức độ ảnh hưởng tương ứng và thứ tự lan truyền dự kiến trong mạng lưới giao thông.

Về giao diện, trang dự đoán lan truyền ùn tắc có thể được chia thành ba khu vực chính. Khu vực thứ nhất là vùng lựa chọn dữ liệu đầu vào, nơi người dùng xác định đoạn đường cần xét hoặc các tham số liên quan nếu hệ thống cho phép cấu hình thêm. Khu vực thứ hai là vùng hiển thị bản đồ, dùng để trực quan hóa đoạn đường khởi phát và các đoạn

đường có khả năng chịu tác động lan truyền. Khu vực thứ ba là vùng kết quả chi tiết, hiển thị danh sách các đoạn đường bị ảnh hưởng, xác suất hoặc mức độ nguy cơ, cùng với các thông tin bổ trợ như thứ tự lan truyền hoặc khoảng thời gian dự kiến xuất hiện ảnh hưởng.

9 Tổng kết

9.1 Kết quả đạt được

Trong khuôn khổ Đồ án Chuyên ngành, nhóm đã nghiên cứu và hiện thực thành công mô hình phân tích mối tương quan và dự báo sự lan truyền ùn tắc giao thông trên mạng lưới giao thông đô thị. Mô hình cho phép nhận diện mối quan hệ giữa các nút và đoạn đường, đồng thời ước lượng phạm vi ảnh hưởng và thời gian lan truyền của tình trạng ùn tắc trong một khoảng thời gian ngắn kể từ thời điểm quan sát.

Bên cạnh việc xây dựng và đánh giá mô hình phân tích và dự báo, nhóm đã đề xuất thiết kế tổng thể cho một hệ thống hỗ trợ phân tích tình trạng giao thông. Thiết kế hệ thống được mô tả thông qua các sơ đồ UML như use case diagram, activity diagram và sequence diagram, nhằm làm rõ các chức năng chính, luồng xử lý và sự tương tác giữa các thành phần trong hệ thống. Mặc dù các chức năng này mới dừng lại ở mức thiết kế và chưa được hiện thực đầy đủ, các sơ đồ đã thể hiện rõ phương hướng tích hợp mô hình phân tích và dự báo vào hệ thống trong tương lai.

Kết quả đạt được cho thấy mô hình và thiết kế hệ thống đề xuất có tiềm năng hỗ trợ việc phân tích và đánh giá tình trạng giao thông đô thị, đồng thời tạo nền tảng cho việc mở rộng và triển khai các chức năng nâng cao trong các giai đoạn tiếp theo.

9.2 Hướng phát triển

Mặc dù đã đạt được một số kết quả tích cực, dự án này vẫn còn tồn tại những hạn chế nhất định. Đầu tiên là vấn đề liên quan đến dữ liệu giao thông sử dụng trong nghiên cứu. Hiện tại, nhóm chưa thu thập được bộ dữ liệu giao thông đầy đủ cho toàn thành phố Hồ Chí Minh. Dữ liệu đang được sử dụng chủ yếu được lấy từ nền tảng TomTom và chỉ giới hạn trong khoảng thời gian một tháng (tháng 8 năm 2024). Điều này ảnh hưởng đến khả năng phản ánh đầy đủ tính biến động và đặc trưng dài hạn của tình trạng giao thông đô thị.

Bên cạnh đó, mô hình dự báo lan truyền ùn tắc hiện tại chỉ có thể thực hiện dự báo vào những thời điểm hạn chế, cụ thể là trong vòng 15 phút kể từ thời điểm của dữ liệu đầu vào. Trong các nghiên cứu tiếp theo, nhóm dự kiến mở rộng tập dữ liệu cả về phạm vi không gian lẫn thời gian, đồng thời cải tiến mô hình dự báo nhằm kéo dài khoảng thời gian dự báo và nâng cao độ chính xác.

Ngoài ra, một hướng phát triển quan trọng khác là hiện thực hóa hệ thống đã được thiết kế, bao gồm việc tích hợp mô hình phân tích và dự báo vào hệ thống phần mềm hoàn chỉnh, phát triển giao diện trực quan và hỗ trợ người dùng tương tác với kết quả phân



tích. Những hướng phát triển này sẽ góp phần nâng cao tính ứng dụng thực tiễn của hệ thống trong công tác phân tích và hỗ trợ ra quyết định liên quan đến giao thông đô thị.

Kế hoạch thực hiện Đồ án tốt nghiệp

Dựa trên cơ sở kết quả đạt được từ Đồ án Chuyên ngành, nhóm lên kế hoạch thực hiện Đồ án tốt nghiệp trong thời hạn 15 tuần như sau:

Tuần 1–2: Tổng quan và kế thừa kết quả

- Rà soát và hệ thống hóa các kết quả đạt được từ đồ án chuyên ngành.
- Đánh giá ưu điểm và hạn chế của mô hình phân tích và dự báo hiện tại.
- Xác định mục tiêu, phạm vi và nội dung cần mở rộng trong đồ án tốt nghiệp.

Tuần 3–4: Mở rộng và hoàn thiện dữ liệu giao thông

- Khảo sát và tích hợp thêm các nguồn dữ liệu giao thông mới.
- Mở rộng dữ liệu theo phạm vi không gian và thời gian.
- Tiếp tục thực hiện tiền xử lý, chuẩn hóa và tổ chức dữ liệu phục vụ phân tích và dự báo.

Tuần 5–7: Cải tiến mô hình phân tích và dự báo lan truyền ùn tắc

- Nghiên cứu và cải tiến mô hình dự báo lan truyền ùn tắc hiện tại.
- Mở rộng khả năng dự báo về mặt thời gian so với mô hình ban đầu.
- Thực hiện so sánh các mô hình hoặc phương pháp dự báo khác nhau.

Tuần 8–9: Đánh giá và phân tích kết quả mô hình

- Đánh giá mô hình dự báo bằng các thước đo như MAE, RMSE.
- Phân tích khả năng mô hình hóa sự lan truyền ùn tắc trong mạng lưới giao thông.
- Thảo luận và rút ra nhận xét về hiệu quả của các mô hình.

Tuần 10–11: Hiện thực hóa hệ thống hỗ trợ phân tích

- Hiện thực các chức năng chính của hệ thống dựa trên thiết kế đã đề xuất.
- Tích hợp mô hình phân tích và dự báo vào hệ thống.
- Xây dựng giao diện người dùng phục vụ việc trực quan hóa dữ liệu và kết quả.

Tuần 12–13: Hoàn thiện chức năng và trực quan hóa

- Hoàn thiện các chức năng trực quan hóa bằng biểu đồ và bản đồ.
- Bổ sung chức năng xuất báo cáo và lưu kết quả phân tích.
- Kiểm thử hệ thống với các kịch bản dữ liệu khác nhau.

Tuần 14: Đánh giá hệ thống và thực nghiệm

- Đánh giá hiệu năng và tính ổn định của hệ thống.
- Thực hiện các thí nghiệm minh họa và phân tích kết quả.
- Tổng hợp ưu điểm và hạn chế của hệ thống.

Tuần 15: Hoàn thiện báo cáo và chuẩn bị bảo vệ

- Hoàn thiện báo cáo đồ án tốt nghiệp.
- Chuẩn bị slide và nội dung thuyết trình.
- Rà soát và chỉnh sửa toàn bộ tài liệu liên quan.

Tài liệu

- [1] M. Saberi, H. Hamedmoghadam, M. Ashfaq *et al.*, “A simple contagion process describes spreading of traffic jams in urban networks,” *Nature Communications*, vol. 11, no. 1, pp. 15353, 2020.
- [2] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, H. Li, “T-GCN: A temporal graph convolutional network for traffic prediction,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.
- [3] A. M. Nagy, V. Simon, “Improving traffic prediction using congestion propagation patterns in smart cities,” *Advanced Engineering Informatics*, vol. 50, pp. 101343, 2021.
- [4] C. Li, W. Yue, G. Mao, Z. Xu, “Congestion propagation based bottleneck identification in urban road networks,” *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 4827–4837, 2020.
- [5] Y. Zhang, Y. Li, X. Zhou, X. Kong, J. Luo, “Curb-GAN: Conditional urban traffic estimation through spatio-temporal generative adversarial networks,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 842–852, 2020.
- [6] A. Ermagun, S. Chatterjee, D. Levinson, “Using temporal detrending to observe the spatial correlation of traffic,” *PLOS ONE*, vol. 12, no. 5, e0176853, 2017.
- [7] Y. Xu, X. Cai, E. Wang, W. Liu, Y. Yang, F. Yang, “Dynamic traffic correlations based spatio-temporal graph convolutional network for urban traffic prediction,” *Information Sciences*, vol. 621, pp. 580–595, 2023.
- [8] A. Lampo, J. Borge-Holthoefer, S. Gómez, A. Solé-Ribalta, “Emergence of spatial transitions in urban congestion dynamics,” *Applied Network Science*, vol. 6, no. 1, pp. 1–19, 2021.
- [9] J. Chen, Y. Wu, “The propagation of congestion on transportation networks analyzed by the percolation process,” *Mathematics*, vol. 12, no. 20, pp. 3247, 2024.
- [10] E. I. Vlahogianni, M. G. Karlaftis, J. C. Golias, “Short-term traffic forecasting: Where we are and where we’re going,” *Transportation Research Part C*, vol. 43, pp. 3–19, 2014.
- [11] M. Treiber, A. Kesting, *Traffic Flow Dynamics: Data, Models and Simulation*, Springer, 2013.

- [12] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P. Yu, “A comprehensive survey on graph neural networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.
- [13] B. Yu, H. Yin, Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” in *Proceedings of IJCAI*, pp. 3634–3640, 2018.
- [14] J. Kreps, N. Narkhede, J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of NetDB*, 2011.
- [15] T. Akidau, R. Bradshaw, C. Chambers *et al.*, “The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proceedings of VLDB*, vol. 8, no. 12, pp. 1792–1803, 2015.
- [16] Y. Zheng, “Trajectory data mining: An overview,” *ACM Transactions on Intelligent Systems and Technology*, vol. 6, no. 3, pp. 29, 2015.
- [17] U.S. Department of Transportation, *Intelligent Transportation Systems Architecture and Standards*, 2017.

10 Phụ lục

PHỤ LỤC A. DANH MỤC THUẬT NGỮ VÀ KÝ HIỆU

Thuật ngữ / Ký hiệu	Giải thích
TTGT	Tình trạng giao thông
Segment	Đoạn đường trong mạng lưới giao thông
Probe data	Dữ liệu GPS/định vị từ các phương tiện di chuyển
DateRange	Khoảng ngày thu thập dữ liệu giao thông
TimeSet	Tập thời gian trong ngày (giờ cao điểm, thấp điểm)
ERD	Entity Relationship Diagram
Chen notation	Ký pháp ERD dùng entity-relationship-attribute
DTW	Dynamic Time Warping
GCN	Graph Convolutional Network
LSTM	Long Short-Term Memory

PHỤ LỤC B. MÔ TẢ CẤU TRÚC DỮ LIỆU TOMTOM

Dữ liệu giao thông được sử dụng trong dự án được lấy từ nền tảng TomTom dưới dạng JSON. Mỗi bộ dữ liệu (job) mô tả tình trạng giao thông của một khu vực trong một khoảng thời gian xác định.

1. Thông tin Job

- `job_id`: Mã định danh của lần truy xuất dữ liệu.
- `creation_time`: Thời điểm tạo bộ dữ liệu.
- `probeSource`: Nguồn dữ liệu probe dùng để suy diễn thông tin giao thông.
- `map_version`: Phiên bản bản đồ được sử dụng.

2. Mạng lưới giao thông (Network)

- `network_id`: Mã định danh mạng lưới.
- Các đoạn đường (segment) thuộc mạng lưới.

3. Dữ liệu theo thời gian

- `DateRange`: Khoảng ngày quan sát.
- `TimeSet`: Khoảng thời gian trong ngày.

- SegmentTimeResult: Thông tin tốc độ, thời gian di chuyển, mức độ ùn tắc.

PHỤ LỤC C. SƠ ĐỒ ERD (CHEN NOTATION)

Hình dưới đây mô tả sơ đồ ERD của dữ liệu giao thông đầu vào theo ký pháp Chen. Trong đó, *SEGMENT_SHAPE* được mô hình hóa như một thực thể yếu, phụ thuộc vào thực thể SEGMENT.



Hình 17: Sơ đồ ERD dữ liệu giao thông theo Chen notation

PHỤ LỤC D. DANH SÁCH USE CASE & ÁNH XẠ YÊU CẦU

Use Case	Mô tả	Yêu cầu liên quan
UC1	Đăng nhập hệ thống	FR1
UC2	Trực quan hóa dữ liệu	FR6
UC3	Xem kết quả phân tích	FR3, FR4
UC4	Dự báo lan truyền ùn tắc	FR5
UC5	Xuất báo cáo	FR7

PHỤ LỤC E. CÔNG CỤ, THƯ VIỆN VÀ CẤU HÌNH SỬ DỤNG

- Ngôn ngữ: Python 3.10+
- Phân tích dữ liệu: Pandas, NumPy
- Phân tích không gian: GeoPandas, Shapely
- Mạng lưới giao thông: NetworkX
- Học máy: PyTorch, TensorFlow
- Trực quan hóa: Matplotlib, Plotly, Streamlit
- Nguồn dữ liệu: TomTom Traffic API